# Physlib Monthly Report

1–31 October 2024

**Contributors:** Joseph Tooby-Smith

Generated 14 August 2026 from commit `16b3066`.

**Abstract**

Physlib is a community library of formalised physics for the Lean 4 proof assistant: definitions and results from across physics, stated in a formal language and checked by machine. This report is an automatically generated record of one month's additions to it.

It covers 1–31 October 2024 on branch `master` of `leanprover-community/physlib`, between commits `94ceb30` and `16b3066`. Over that period 1 contributor landed 224 commits, changing 22,861 lines across 131 files and adding 889 new Lean declarations, each catalogued with its statement in Section 2. The complete diff is available at GitHub compare.

## Contents

# 1 Introduction

During Oct 2024, 1 contributor submitted 224 commits that changed 22,861 lines (+14,085 / -8,776) across 131 files spanning 16 top-level areas of the repository. Of those, 46 files were newly added and 27 were removed outright. This report catalogues 889 newly added Lean declarations: 514 lemmas, 281 defs, 37 instances, 24 informal lemmas, 12 abbrevs, 9 structures, 9 theorems, 2 inductives, and 1 informal definition.

See the full compare diff and the list of commits on GitHub for the raw source.

## 1.1 Contributors

The following individuals authored commits merged during this reporting period, listed alphabetically.

| Contributor | Lines Changed | Commits |
|---|---|---|
| Joseph Tooby-Smith | 39,114 | 224 |

## 1.2 Reviewers

The following individuals approved pull requests during this reporting period, listed alphabetically.

*No reviewers identified for this month.*

## 1.3 Modified subfolders

| Subfolder | Files | New | Deleted | Additions | Deletions |
|---|---|---|---|---|---|
| `HepLean/Tensors` | 53 | +29 | -24 | +8,268 | -7,521 |
| `HepLean/SpaceTime` | 26 | +14 | -3 | +4,480 | -861 |
| `HepLean/Mathematics` | 2 | +2 | – | +695 | -0 |
| `HepLean/AnomalyCancellation` | 30 | – | – | +213 | -153 |
| `HepLean/FlavorPhysics` | 6 | – | – | +168 | -89 |
| `docs` | 2 | – | – | +42 | -60 |
| `HepLean/BeyondTheStandardModel` | 1 | – | – | +80 | -14 |
| `HepLean.lean` | 1 | – | – | +45 | -27 |
| `lake-manifest.json` | 1 | – | – | +22 | -12 |
| `scripts` | 2 | – | – | +21 | -5 |
| `HepLean/FeynmanDiagrams` | 1 | – | – | +14 | -10 |
| `README.md` | 1 | – | – | +8 | -16 |
| `HepLean/StandardModel` | 2 | – | – | +14 | -5 |
| `.gitpod.yml` | 1 | +1 | – | +12 | -0 |
| `lakefile.toml` | 1 | – | – | +2 | -2 |
| `lean-toolchain` | 1 | – | – | +1 | -1 |

## 1.4 New declarations by kind

| Kind | Count |
|---|---|
| `lemma` | 514 |
| `def` | 281 |

| | |
|---|---|
| instance | 37 |
| informal_lemma | 24 |
| abbrev | 12 |
| structure | 9 |
| theorem | 9 |
| inductive | 2 |
| informal_definition | 1 |

# 2 Details by file

## 2.1 HepLean.BeyondTheStandardModel.TwoHDM.GaugeOrbits

HepLean/BeyondTheStandardModel/TwoHDM/GaugeOrbits.lean on GitHub.

**def fieldCompMatrix** [source]

*The 2 x 2 complex matrices made up of components of the two Higgs fields.*

```
def fieldCompMatrix (Φ1 Φ2 : HiggsField) (x : SpaceTime) : Matrix (Fin 2) (Fin 2) ℂ
```

**lemma prodMatrix_eq_fieldCompMatrix_sq** [source]

*The matrix `prodMatrix Φ1 Φ2 x` is equal to the square of `fieldCompMatrix Φ1 Φ2 x`.*

```
lemma prodMatrix_eq_fieldCompMatrix_sq (Φ1 Φ2 : HiggsField) (x : SpaceTime) :
    prodMatrix Φ1 Φ2 x = fieldCompMatrix Φ1 Φ2 x * (fieldCompMatrix Φ1 Φ2 x).conjTranspose
```

**instance PartialOrder ℂ** [source]

*An instance of `PartialOrder` on `ℂ` defined through `Complex.partialOrder`.*

```
local instance : PartialOrder ℂ
```

**instance NormedAddCommGroup (Matrix (Fin 2) (Fin 2) ℂ)** [source]

*An instance of `NormedAddCommGroup` on `Matrix (Fin 2) (Fin 2) ℂ` defined through `Matrix.normedAddCommGroup`.*

```
local instance : NormedAddCommGroup (Matrix (Fin 2) (Fin 2) ℂ)
```

**instance NormedSpace ℝ (Matrix (Fin 2) (Fin 2) ℂ)** [source]

*An instance of `NormedSpace` on `Matrix (Fin 2) (Fin 2) ℂ` defined through `Matrix.normedSpace`.*

```
local instance : NormedSpace ℝ (Matrix (Fin 2) (Fin 2) ℂ)
```

**lemma prodMatrix_posSemiDef** [source]

*The matrix `prodMatrix` is positive semi-definite.*

```
lemma prodMatrix_posSemiDef (Φ1 Φ2 : HiggsField) (x : SpaceTime) :
    (prodMatrix Φ1 Φ2 x).PosSemidef
```

**informal_lemma prodMatrix_invariant** [source]

```
informal_lemma prodMatrix_invariant where
  math :≈ "The map ``prodMatrix is invariant under the simultanous action of ``gaugeAction
    on the two Higgs fields."
  deps :≈ [``prodMatrix, ``gaugeAction]
```

**informal_lemma prodMatrix_to_higgsField** [source]

```
informal_lemma prodMatrix_to_higgsField where
  math :≈ "Given any smooth map ``f from spacetime to 2 x 2 complex matrices landing on
    positive
    semi-definite matrices, there exist smooth Higgs fields ``Φ1 and ``Φ2 such that
    ``f is equal to ``prodMatrix Φ1 Φ2."
  deps :≈ [``prodMatrix, ``HiggsField, ``prodMatrix_smooth]
  ref :≈ "https://arxiv.org/pdf/hep-ph/0605184"
```

## 2.2 HepLean.Mathematics.Fin

HepLean/Mathematics/Fin.lean on GitHub.

**def predAboveI** [source]

*Given a `i` and `x` in `Fin n.succ.succ` returns an element of `Fin n.succ` subtracting 1 if `i.val ≤ x.val` else casting `x`.*

```
def predAboveI (i x : Fin n.succ.succ) : Fin n.succ
```

**lemma predAboveI_self** [source]

```
lemma predAboveI_self (i : Fin n.succ.succ) : predAboveI i i = ⟨i.val - 1, by omega⟩
```

**lemma predAboveI_succAbove** [source]

```
@[simp]
lemma predAboveI_succAbove (i : Fin n.succ.succ) (x : Fin n.succ) :
    predAboveI i (Fin.succAbove i x) = x
```

**lemma succsAbove_predAboveI** [source]

```
lemma succsAbove_predAboveI {i x : Fin n.succ.succ} (h : i ≠ x) :
    Fin.succAbove i (predAboveI i x) = x
```

**lemma predAboveI_eq_iff** [source]

```
lemma predAboveI_eq_iff {i x : Fin n.succ.succ} (h : i ≠ x) (y : Fin n.succ) :
    y = predAboveI i x ↔ i.succAbove y = x
```

**lemma predAboveI_lt** [source]

```
lemma predAboveI_lt {i x : Fin n.succ.succ} (h : x.val < i.val) :
    predAboveI i x = ⟨x.val, by omega⟩
```

**lemma predAboveI_ge** [source]

```
lemma predAboveI_ge {i x : Fin n.succ.succ} (h : i.val < x.val) :
    predAboveI i x = (x.val - 1, by omega)
```

**lemma succAbove_succAbove_predAboveI** [source]

```
lemma succAbove_succAbove_predAboveI (i : Fin n.succ.succ) (j : Fin n.succ) (x : Fin n) :
    i.succAbove (j.succAbove x) =
    (i.succAbove j).succAbove ((predAboveI (i.succAbove j) i).succAbove x)
```

**def finExtractOne** [source]

*The equivalence between `Fin n.succ` and `Fin 1 ⊕ Fin n` extracting the `i`th component.*

```
def finExtractOne {n : ℕ} (i : Fin n.succ) : Fin n.succ ≃ Fin 1 ⊕ Fin n
```

**lemma finExtractOne_apply_eq** [source]

```
lemma finExtractOne_apply_eq {n : ℕ} (i : Fin n.succ) :
    finExtractOne i i = Sum.inl 0
```

**lemma finExtractOne_symm_inr** [source]

```
lemma finExtractOne_symm_inr {n : ℕ} (i : Fin n.succ) :
    (finExtractOne i).symm ∘ Sum.inr = i.succAbove
```

**lemma finExtractOne_symm_inr_apply** [source]

```
@[simp]
lemma finExtractOne_symm_inr_apply {n : ℕ} (i : Fin n.succ) (x : Fin n) :
    (finExtractOne i).symm (Sum.inr x) = i.succAbove x
```

**lemma finExtractOne_symm_inl_apply** [source]

```
@[simp]
lemma finExtractOne_symm_inl_apply {n : ℕ} (i : Fin n.succ) :
    (finExtractOne i).symm (Sum.inl 0) = i
```

**def finExtractOnPermHom** [source]

*Given an equivalence `Fin n.succ.succ ≃ Fin n.succ.succ`, and an `i : Fin n.succ.succ`, the map `Fin n.succ → Fin n.succ` obtained by dropping `i` and it's image.*

```
def finExtractOnPermHom (i : Fin n.succ.succ) (σ : Fin n.succ.succ ≃ Fin n.succ.succ) :
    Fin n.succ → Fin n.succ
```

**lemma finExtractOnPermHom_inv** [source]

```
lemma finExtractOnPermHom_inv (i : Fin n.succ.succ) (σ : Fin n.succ.succ ≃ Fin n.succ.succ) :
    (finExtractOnPermHom (σ i) σ.symm) ∘ (finExtractOnPermHom i σ) = id
```

**def finExtractOnePerm** [source]

*Given an equivalence `Fin n.succ.succ ≃ Fin n.succ.succ`, and an `i : Fin n.succ.succ`, the equivalence `Fin n.succ ≃ Fin n.succ` obtained by dropping `i` and it's image.*

```
def finExtractOnePerm (i : Fin n.succ.succ) (σ : Fin n.succ.succ ≃ Fin n.succ.succ) :
    Fin n.succ ≃ Fin n.succ where
```

**def finExtractTwo** [source]

*The equivalence of types `Fin n.succ.succ ≃ (Fin 1 ⊕ Fin 1) ⊕ Fin n` extracting the `i` and `(i.succAbove j)`.*

```
def finExtractTwo {n : ℕ} (i : Fin n.succ.succ) (j : Fin n.succ) :
    Fin n.succ.succ ≃ (Fin 1 ⊕ Fin 1) ⊕ Fin n
```

**lemma finExtractTwo_apply_fst** [source]

```
@[simp]
lemma finExtractTwo_apply_fst {n : ℕ} (i : Fin n.succ.succ) (j : Fin n.succ) :
    finExtractTwo i j i = Sum.inl (Sum.inl 0)
```

**lemma finExtractTwo_symm_inr** [source]

```
lemma finExtractTwo_symm_inr {n : ℕ} (i : Fin n.succ.succ) (j : Fin n.succ) :
    (finExtractTwo i j).symm ∘ Sum.inr = i.succAbove ∘ j.succAbove
```

**lemma finExtractTwo_symm_inr_apply** [source]

```
@[simp]
lemma finExtractTwo_symm_inr_apply {n : ℕ} (i : Fin n.succ.succ) (j : Fin n.succ) (x : Fin n) :
    (finExtractTwo i j).symm (Sum.inr x) = i.succAbove (j.succAbove x)
```

**lemma finExtractTwo_symm_inl_inr_apply** [source]

```
@[simp]
lemma finExtractTwo_symm_inl_inr_apply {n : ℕ} (i : Fin n.succ.succ) (j : Fin n.succ) :
    (finExtractTwo i j).symm (Sum.inl (Sum.inr 0)) = i.succAbove j
```

**lemma finExtractTwo_symm_inl_inl_apply** [source]

```
@[simp]
lemma finExtractTwo_symm_inl_inl_apply {n : ℕ} (i : Fin n.succ.succ) (j : Fin n.succ) :
    (finExtractTwo i j).symm (Sum.inl (Sum.inl 0)) = i
```

**lemma finExtractTwo_apply_snd** [source]

```
@[simp]
lemma finExtractTwo_apply_snd {n : ℕ} (i : Fin n.succ.succ) (j : Fin n.succ) :
    finExtractTwo i j (i.succAbove j) = Sum.inl (Sum.inr 0)
```

## 2.3 HepLean.Mathematics.PiTensorProduct

HepLean/Mathematics/PiTensorProduct.lean on GitHub.

**lemma induction_tmul** [source]

```
lemma induction_tmul {f g : ((⨂[R] i : ι1, s1 i) ⊗[R] ⨂[R] i : ι2, s2 i) →ₗ[R] M}
    (h : ∀ p q, f (PiTensorProduct.tprod R p ⊗ₜ[R] PiTensorProduct.tprod R q)
    = g (PiTensorProduct.tprod R p ⊗ₜ[R] PiTensorProduct.tprod R q)) : f = g
```

**lemma induction_assoc** [source]

```
lemma induction_assoc
    {f g : ((⨂[R] i : ι1, s1 i) ⊗[R] (⨂[R] i : ι2, s2 i) ⊗[R] ⨂[R] i : ι3, s3 i) →ₗ[R] M}
    (h : ∀ p q m, f (PiTensorProduct.tprod R p ⊗ₜ[R]
    PiTensorProduct.tprod R q ⊗ₜ[R] PiTensorProduct.tprod R m)
    = g (PiTensorProduct.tprod R p ⊗ₜ[R] PiTensorProduct.tprod R q
    ⊗ₜ[R] PiTensorProduct.tprod R m)) : f = g
```

**lemma induction_assoc'** [source]

```
lemma induction_assoc'
    {f g : (((⨂[R] i : ι1, s1 i) ⊗[R] (⨂[R] i : ι2, s2 i)) ⊗[R] ⨂[R] i : ι3, s3 i) →ₗ[R] M}
    (h : ∀ p q m, f ((PiTensorProduct.tprod R p ⊗ₜ[R] PiTensorProduct.tprod R q) ⊗ₜ[R]
    PiTensorProduct.tprod R m) = g ((PiTensorProduct.tprod R p ⊗ₜ[R] PiTensorProduct.tprod R q)
    ⊗ₜ[R] PiTensorProduct.tprod R m)) : f = g
```

**lemma induction_tmul_mod** [source]

```
lemma induction_tmul_mod
    {f g : ((⨂[R] i : ι1, s1 i) ⊗[R] N) →ₗ[R] M}
    (h : ∀ p m, f (PiTensorProduct.tprod R p ⊗ₜ[R] m) = g (PiTensorProduct.tprod R p ⊗ₜ[R] m))
    :
    f = g
```

**lemma induction_mod_tmul** [source]

```
lemma induction_mod_tmul
    {f g : (N ⊗[R] ⨂[R] i : ι1, s1 i) →ₗ[R] M}
    (h : ∀ m p, f (m ⊗ₜ[R] PiTensorProduct.tprod R p) = g (m ⊗ₜ[R] PiTensorProduct.tprod R p))
    :
    f = g
```

**instance (i : ι1 ⊕ ι2) → AddCommMonoid ((fun i => Sum.elim s1 s2 i) i)** [source]

```
instance : (i : ι1 ⊕ ι2) → AddCommMonoid ((fun i => Sum.elim s1 s2 i) i)
```

**instance (i : ι1 ⊕ ι2) → Module R ((fun i => Sum.elim s1 s2 i) i)** [source]

```
instance : (i : ι1 ⊕ ι2) → Module R ((fun i => Sum.elim s1 s2 i) i)
```

**def pureInl** [source]

*Takes a map `(i : ι1 ⊕ ι2) → Sum.elim s1 s2 i` to the underlying map `(i : ι1) → s1 i` .*

```
def pureInl (f : (i : ι1 ⊕ ι2) → Sum.elim s1 s2 i) : (i : ι1) → s1 i
```

**def pureInr** [source]

*Takes a map `(i : ι1 ⊕ ι2) → Sum.elim s1 s2 i` to the underlying map `(i : ι2) → s2 i` .*

```
def pureInr (f : (i : ι1 ⊕ ι2) → Sum.elim s1 s2 i) : (i : ι2) → s2 i
```

**lemma pureInl_update_left** [source]

```
lemma pureInl_update_left [DecidableEq ι1] (f : (i : ι1 ⊕ ι2) → Sum.elim s1 s2 i) (x : ι1)
    (v1 : s1 x) : pureInl (Function.update f (Sum.inl x) v1) =
    Function.update (pureInl f) x v1
```

**lemma pureInr_update_left** [source]

```
lemma pureInr_update_left (f : (i : ι1 ⊕ ι2) → Sum.elim s1 s2 i) (x : ι1)
    (v2 : s1 x) :
    pureInr (Function.update f (Sum.inl x) v2) = (pureInr f)
```

**lemma pureInr_update_right** [source]

```
lemma pureInr_update_right [DecidableEq ι2] (f : (i : ι1 ⊕ ι2) → Sum.elim s1 s2 i) (x : ι2)
    (v2 : s2 x) : pureInr (Function.update f (Sum.inr x) v2) =
    Function.update (pureInr f) x v2
```

**lemma pureInl_update_right** [source]

```
lemma pureInl_update_right (f : (i : ι1 ⊕ ι2) → Sum.elim s1 s2 i) (x : ι2)
    (v1 : s2 x) :
    pureInl (Function.update f (Sum.inr x) v1) = (pureInl f)
```

**def domCoprod** [source]

*The multilinear map from `(Sum.elim s1 s2)` to `((⨂[R] i : ι1, s1 i) ⊗[R] ⨂[R] i : ι2, s2 i)` defined by splitting elements of `(Sum.elim s1 s2)` into two parts.*

```
def domCoprod :
    MultilinearMap R (Sum.elim s1 s2) ((⨂[R] i : ι1, s1 i) ⊗[R] ⨂[R] i : ι2, s2 i) where
```

**def tmulSymm** [source]

*Expand `PiTensorProduct` on sums into a `TensorProduct` of two factors.*

```
def tmulSymm : (⨂[R] i : ι1 ⊕ ι2, (Sum.elim s1 s2) i) →ₗ[R]
    ((⨂[R] i : ι1, s1 i) ⊗[R] ⨂[R] i : ι2, s2 i)
```

**def elimPureTensor** [source]

*Produces a map `(i : ι1 ⊕ ι2) → Sum.elim s1 s2 i` from a map `(i : ι1) → s1 i` and a map `q : (i : ι2) → s2 i`.*

```
def elimPureTensor (p : (i : ι1) → s1 i) (q : (i : ι2) → s2 i) : (i : ι1 ⊕ ι2) → Sum.elim s1 s2
    i
```

**lemma elimPureTensor_update_right** [source]

```
lemma elimPureTensor_update_right (p : (i : ι1) → s1 i) (q : (i : ι2) → s2 i)
    (y : ι2) (r : s2 y) : elimPureTensor p (Function.update q y r) =
    Function.update (elimPureTensor p q) (Sum.inr y) r
```

**lemma elimPureTensor_update_left** [source]

```
@[simp]
lemma elimPureTensor_update_left (p : (i : ι1) → s1 i) (q : (i : ι2) → s2 i)
    (x : ι1) (r : s1 x) : elimPureTensor (Function.update p x r) q =
    Function.update (elimPureTensor p q) (Sum.inl x) r
```

**def elimPureTensorMulLin** [source]

*The multilinear map valued in multilinear maps defined by combining `(i : ι1) → s1 i` and `q : (i : ι2) → s2 i` into a PiTensorProduct.*

```
def elimPureTensorMulLin : MultilinearMap R s1
    (MultilinearMap R s2 (⨂[R] i : ι1 ⊕ ι2, (Sum.elim s1 s2) i)) where
```

**def tmul** [source]

*Collapse a `TensorProduct` of `PiTensorProduct` into a `PiTensorProduct`.*

```
def tmul : ((⨂[R] i : ι1, s1 i) ⊗[R] ⨂[R] i : ι2, s2 i) →ₗ[R]
    ⨂[R] i : ι1 ⊕ ι2, (Sum.elim s1 s2) i
```

**def tmulEquiv** [source]

*THe equivalence formed by combining a `TensorProduct` into a `PiTensorProduct`.*

```
def tmulEquiv : ((⨂[R] i : ι1, s1 i) ⊗[R] ⨂[R] i : ι2, s2 i) ≃ₗ[R]
    ⨂[R] i : ι1 ⊕ ι2, (Sum.elim s1 s2) i
```

**lemma tmulEquiv_tmul_tprod** [source]

```
@[simp]
lemma tmulEquiv_tmul_tprod (p : (i : ι1) → s1 i) (q : (i : ι2) → s2 i) :
    tmulEquiv ((PiTensorProduct.tprod R) p ⊗ₜ[R] (PiTensorProduct.tprod R) q) =
    (PiTensorProduct.tprod R) (elimPureTensor p q)
```

## 2.4 HepLean.SpaceTime.LorentzGroup.Basic

HepLean/SpaceTime/LorentzGroup/Basic.lean on GitHub.

**def toComplex** [source]

*The monoid homomorphisms taking the lorentz group to complex matrices.*

```
def toComplex : LorentzGroup d →* Matrix (Fin 1 ⊕ Fin d) (Fin 1 ⊕ Fin d) ℂ where
```

**instance Invertible (toComplex M)** [source]

```
instance (M : LorentzGroup d) : Invertible (toComplex M) where
```

**lemma toComplex_inv** [source]

```
lemma toComplex_inv (Λ : LorentzGroup d) : (toComplex Λ)⁻¹ = toComplex Λ⁻¹
```

**lemma toComplex_mul_minkowskiMatrix_mul_transpose** [source]

```
@[simp]
lemma toComplex_mul_minkowskiMatrix_mul_transpose (Λ : LorentzGroup d) :
    toComplex Λ * minkowskiMatrix.map ofReal * (toComplex Λ)ᵀ = minkowskiMatrix.map ofReal
```

**lemma toComplex_transpose_mul_minkowskiMatrix_mul_self** [source]

```
@[simp]
lemma toComplex_transpose_mul_minkowskiMatrix_mul_self (Λ : LorentzGroup d) :
    (toComplex Λ)ᵀ * minkowskiMatrix.map ofReal * toComplex Λ = minkowskiMatrix.map ofReal
```

**lemma toComplex_mulVec_ofReal** [source]

```
lemma toComplex_mulVec_ofReal (v : Fin 1 ⊕ Fin d → ℝ) (Λ : LorentzGroup d) :
    toComplex Λ *ᵥ (ofReal ∘ v) = ofReal ∘ (Λ *ᵥ v)
```

## 2.5 HepLean.SpaceTime.LorentzVector.AsSelfAdjointMatrix

HepLean/SpaceTime/LorentzVector/AsSelfAdjointMatrix.lean on GitHub.

**lemma toSelfAdjointMatrix_apply** [source]

```
lemma toSelfAdjointMatrix_apply (x : LorentzVector 3) : toSelfAdjointMatrix x =
    x (Sum.inl 0) • ⟨PauliMatrix.σ0, PauliMatrix.σ0_selfAdjoint⟩
    - x (Sum.inr 0) • ⟨PauliMatrix.σ1, PauliMatrix.σ1_selfAdjoint⟩
    - x (Sum.inr 1) • ⟨PauliMatrix.σ2, PauliMatrix.σ2_selfAdjoint⟩
    - x (Sum.inr 2) • ⟨PauliMatrix.σ3, PauliMatrix.σ3_selfAdjoint⟩
```

**lemma toSelfAdjointMatrix_apply_coe** [source]

```
lemma toSelfAdjointMatrix_apply_coe (x : LorentzVector 3) : (toSelfAdjointMatrix x).1 =
    x (Sum.inl 0) • PauliMatrix.σ0
    - x (Sum.inr 0) • PauliMatrix.σ1
    - x (Sum.inr 1) • PauliMatrix.σ2
    - x (Sum.inr 2) • PauliMatrix.σ3
```

**lemma toSelfAdjointMatrix_stdBasis** [source]

```
lemma toSelfAdjointMatrix_stdBasis (i : Fin 1 ⊕ Fin 3) :
    toSelfAdjointMatrix (LorentzVector.stdBasis i) = PauliMatrix.σSAL i
```

**lemma toSelfAdjointMatrix_symm_basis** [source]

```
@[simp]
lemma toSelfAdjointMatrix_symm_basis (i : Fin 1 ⊕ Fin 3) :
    toSelfAdjointMatrix.symm (PauliMatrix.σSAL i) = (LorentzVector.stdBasis i)
```

## 2.6 HepLean.SpaceTime.LorentzVector.Complex.Basic

HepLean/SpaceTime/LorentzVector/Complex/Basic.lean on GitHub.

**def complexContr** [source]

*The representation of `SL(2, ℂ)` on complex vectors corresponding to contravariant Lorentz vectors. In index notation these have an up index $\psi^i$.*

```
def complexContr : Rep ℂ SL(2, ℂ)
```

**def complexCo** [source]

*The representation of `SL(2, ℂ)` on complex vectors corresponding to contravariant Lorentz vectors. In index notation these have a down index $\psi^i$.*

```
def complexCo : Rep ℂ SL(2, ℂ)
```

**def complexContrBasis** [source]

*The standard basis of complex contravariant Lorentz vectors.*

```
def complexContrBasis : Basis (Fin 1 ⊕ Fin 3) ℂ complexContr
```

**lemma complexContrBasis_toFin13ℂ** [source]

```
@[simp]
lemma complexContrBasis_toFin13ℂ (i :Fin 1 ⊕ Fin 3) :
    (complexContrBasis i).toFin13ℂ = Pi.single i 1
```

**lemma complexContrBasis_ρ_apply** [source]

```
@[simp]
lemma complexContrBasis_ρ_apply (M : SL(2,ℂ)) (i j : Fin 1 ⊕ Fin 3) :
    (LinearMap.toMatrix complexContrBasis complexContrBasis) (complexContr.ρ M) i j =
    (LorentzGroup.toComplex (SL2C.toLorentzGroup M)) i j
```

**lemma complexContrBasis_ρ_val** [source]

```
lemma complexContrBasis_ρ_val (M : SL(2,ℂ)) (v : complexContr) :
    ((complexContr.ρ M) v).val =
    LorentzGroup.toComplex (SL2C.toLorentzGroup M) *ᵥ v.val
```

**def complexContrBasisFin4** [source]

*The standard basis of complex contravariant Lorentz vectors indexed by `Fin 4`.*

```
def complexContrBasisFin4 : Basis (Fin 4) ℂ complexContr
```

**def complexCoBasis** [source]

*The standard basis of complex covariant Lorentz vectors.*

```
def complexCoBasis : Basis (Fin 1 ⊕ Fin 3) ℂ complexCo
```

**lemma complexCoBasis_toFin13ℂ** [source]

```
@[simp]
lemma complexCoBasis_toFin13ℂ (i :Fin 1 ⊕ Fin 3) : (complexCoBasis i).toFin13ℂ = Pi.single i 1
```

**lemma complexCoBasis_ρ_apply** [source]

```
@[simp]
lemma complexCoBasis_ρ_apply (M : SL(2,ℂ)) (i j : Fin 1 ⊕ Fin 3) :
    (LinearMap.toMatrix complexCoBasis complexCoBasis) (complexCo.ρ M) i j =
    (LorentzGroup.toComplex (SL2C.toLorentzGroup M))⁻¹ᵀ i j
```

**def complexCoBasisFin4** [source]

*The standard basis of complex covariant Lorentz vectors indexed by `Fin 4`.*

```
def complexCoBasisFin4 : Basis (Fin 4) ℂ complexCo
```

**def inclCongrRealLorentz** [source]

*The semilinear map including real Lorentz vectors into complex contravariant lorentz vectors.*

```
def inclCongrRealLorentz : LorentzVector 3 →ₛₗ[Complex.ofReal] complexContr where
```

**lemma inclCongrRealLorentz_val** [source]

```
lemma inclCongrRealLorentz_val (v : LorentzVector 3) :
    (inclCongrRealLorentz v).val = ofReal ∘ v
```

**lemma complexContrBasis_of_real** [source]

```
lemma complexContrBasis_of_real (i : Fin 1 ⊕ Fin 3) :
    (complexContrBasis i) = inclCongrRealLorentz (LorentzVector.stdBasis i)
```

**lemma inclCongrRealLorentz_ρ** [source]

```
lemma inclCongrRealLorentz_ρ (M : SL(2, ℂ)) (v : LorentzVector 3) :
    (complexContr.ρ M) (inclCongrRealLorentz v) =
    inclCongrRealLorentz (SL2C.repLorentzVector M v)
```

**lemma SL2CRep_ρ_basis** [source]

```
lemma SL2CRep_ρ_basis (M : SL(2, ℂ)) (i : Fin 1 ⊕ Fin 3) :
    (complexContr.ρ M) (complexContrBasis i) =
    ∑ j, (SL2C.toLorentzGroup M).1 j i •
    complexContrBasis j
```

## 2.7 HepLean.SpaceTime.LorentzVector.Complex.Contraction

HepLean/SpaceTime/LorentzVector/Complex/Contraction.lean on GitHub.

**def contrCoContrBi** [source]

*The bi-linear map corresponding to contraction of a contravariant Lorentz vector with a covariant Lorentz vector.*

```
def contrCoContrBi : complexContr →ₗ[ℂ] complexCo →ₗ[ℂ] ℂ where
```

**def contrContrCoBi** [source]

*The bi-linear map corresponding to contraction of a covariant Lorentz vector with a contravariant Lorentz vector.*

```
def contrContrCoBi : complexCo →ₗ[ℂ] complexContr →ₗ[ℂ] ℂ where
```

**def contrCoContraction** [source]

*The linear map from complexContr ⊗ complexCo to ℂ given by summing over components of contravariant Lorentz vector and covariant Lorentz vector in the standard basis (i.e. the dot product). In terms of index notation this is the contraction is $\psi^i \varphi_i$.*

```
def contrCoContraction : complexContr ⊗ complexCo ⟶ 𝟙_ (Rep ℂ SL(2,ℂ)) where
```

**lemma contrCoContraction_hom_tmul** [source]

```
lemma contrCoContraction_hom_tmul (ψ : complexContr) (φ : complexCo) :
    contrCoContraction.hom (ψ ⊗ₜ φ) = ψ.toFin13ℂ ⬝ᵥ φ.toFin13ℂ
```

**lemma contrCoContraction_basis** [source]

```
lemma contrCoContraction_basis (i j : Fin 4) :
    contrCoContraction.hom (complexContrBasisFin4 i ⊗ₜ complexCoBasisFin4 j) =
    if i.1 = j.1 then (1 : ℂ) else 0
```

**lemma contrCoContraction_basis'** [source]

```
lemma contrCoContraction_basis' (i j : Fin 1 ⊕ Fin 3) :
    contrCoContraction.hom (complexContrBasis i ⊗ₜ complexCoBasis j) =
    if i = j then (1 : ℂ) else 0
```

**def coContrContraction** [source]

*The linear map from complexCo ⊗ complexContr to ℂ given by summing over components of covariant Lorentz vector and contravariant Lorentz vector in the standard basis (i.e. the dot product). In terms of index notation this is the contraction is $\varphi_i \psi^i$.*

```
def coContrContraction : complexCo ⊗ complexContr ⟶ 𝟙_ (Rep ℂ SL(2,ℂ)) where
```

**lemma coContrContraction_hom_tmul** [source]

```
lemma coContrContraction_hom_tmul (φ : complexCo) (ψ : complexContr) :
    coContrContraction.hom (φ ⊗ₜ ψ) = φ.toFin13ℂ ⬝ᵥ ψ.toFin13ℂ
```

**lemma coContrContraction_basis** [source]

```
lemma coContrContraction_basis (i j : Fin 4) :
    coContrContraction.hom (complexCoBasisFin4 i ⊗ₜ complexContrBasisFin4 j) =
    if i.1 = j.1 then (1 : ℂ) else 0
```

**lemma coContrContraction_basis'** [source]

```
lemma coContrContraction_basis' (i j : Fin 1 ⊕ Fin 3) :
    coContrContraction.hom (complexCoBasis i ⊗ₜ complexContrBasis j) =
    if i = j then (1 : ℂ) else 0
```

**lemma contrCoContraction_tmul_symm** [source]

```
lemma contrCoContraction_tmul_symm (φ : complexContr) (ψ : complexCo) :
    contrCoContraction.hom (φ ⊗ₜ ψ) = coContrContraction.hom (ψ ⊗ₜ φ)
```

**lemma coContrContraction_tmul_symm** [source]

```
lemma coContrContraction_tmul_symm (φ : complexCo) (ψ : complexContr) :
    coContrContraction.hom (φ ⊗ₜ ψ) = contrCoContraction.hom (ψ ⊗ₜ φ)
```

## 2.8 HepLean.SpaceTime.LorentzVector.Complex.Metric

HepLean/SpaceTime/LorentzVector/Complex/Metric.lean on GitHub.

**def contrMetricVal** [source]

*The metric $\eta^{aa}$ as an element of `(complexContr ⊗ complexContr).V`.*

```
def contrMetricVal : (complexContr ⊗ complexContr).V
```

**lemma contrMetricVal_expand_tmul** [source]

*The expansion of `contrMetricVal` into basis vectors.*

```
lemma contrMetricVal_expand_tmul : contrMetricVal =
    complexContrBasis (Sum.inl 0) ⊗ₜ[ℂ] complexContrBasis (Sum.inl 0)
    - complexContrBasis (Sum.inr 0) ⊗ₜ[ℂ] complexContrBasis (Sum.inr 0)
```

```
    - complexContrBasis (Sum.inr 1) ⊗ₜ[ℂ] complexContrBasis (Sum.inr 1)
    - complexContrBasis (Sum.inr 2) ⊗ₜ[ℂ] complexContrBasis (Sum.inr 2)
```

**def contrMetric** [source]

*The metric $\eta^{aa}$ as a morphism `𝟙_ (Rep ℂ SL(2,ℂ)) ⟶ complexContr ⊗ complexContr`, making its invariance under the action of `SL(2,ℂ)`.*

```
def contrMetric : 𝟙_ (Rep ℂ SL(2,ℂ)) ⟶ complexContr ⊗ complexContr where
```

**lemma contrMetric_apply_one** [source]

```
lemma contrMetric_apply_one : contrMetric.hom (1 : ℂ) = contrMetricVal
```

**def coMetricVal** [source]

*The metric $\eta_{ii}$ as an element of `(complexCo ⊗ complexCo).V`.*

```
def coMetricVal : (complexCo ⊗ complexCo).V
```

**lemma coMetricVal_expand_tmul** [source]

*The expansion of `coMetricVal` into basis vectors.*

```
lemma coMetricVal_expand_tmul : coMetricVal =
    complexCoBasis (Sum.inl 0) ⊗ₜ[ℂ] complexCoBasis (Sum.inl 0)
    - complexCoBasis (Sum.inr 0) ⊗ₜ[ℂ] complexCoBasis (Sum.inr 0)
    - complexCoBasis (Sum.inr 1) ⊗ₜ[ℂ] complexCoBasis (Sum.inr 1)
    - complexCoBasis (Sum.inr 2) ⊗ₜ[ℂ] complexCoBasis (Sum.inr 2)
```

**def coMetric** [source]

*The metric $\eta_{ii}$ as a morphism `𝟙_ (Rep ℂ SL(2,ℂ)) ⟶ complexCo ⊗ complexCo`, making its invariance under the action of `SL(2,ℂ)`.*

```
def coMetric : 𝟙_ (Rep ℂ SL(2,ℂ)) ⟶ complexCo ⊗ complexCo where
```

**lemma coMetric_apply_one** [source]

```
lemma coMetric_apply_one : coMetric.hom (1 : ℂ) = coMetricVal
```

**lemma contrCoContraction_apply_metric** [source]

```
lemma contrCoContraction_apply_metric : (β_ complexContr complexCo).hom.hom
    ((complexContr.V ◁ (λ_ complexCo.V).hom)
    ((complexContr.V ◁ contrCoContraction.hom ▷ complexCo.V)
    ((complexContr.V ◁ (α_ complexContr.V complexCo.V complexCo.V).inv)
    ((α_ complexContr.V complexContr.V (complexCo.V ⊗ complexCo.V)).hom
    (contrMetric.hom (1 : ℂ) ⊗ₜ[ℂ] coMetric.hom (1 : ℂ))))) =
    coContrUnit.hom (1 : ℂ)
```

**lemma coContrContraction_apply_metric** [source]

```
lemma coContrContraction_apply_metric : (β_ complexCo complexContr).hom.hom
    ((complexCo.V ◁ (λ_ complexContr.V).hom)
    ((complexCo.V ◁ coContrContraction.hom ▷ complexContr.V)
    ((complexCo.V ◁ (α_ complexCo.V complexContr.V complexContr.V).inv)
    ((α_ complexCo.V complexCo.V (complexContr.V ⊗ complexContr.V)).hom
    (coMetric.hom (1 : ℂ) ⊗ₜ[ℂ] contrMetric.hom (1 : ℂ))))) =
    contrCoUnit.hom (1 : ℂ)
```

## 2.9 HepLean.SpaceTime.LorentzVector.Complex.Two

HepLean/SpaceTime/LorentzVector/Complex/Two.lean on GitHub.

**def contrContrToMatrix** [source]

*Equivalence of `complexContr ⊗ complexContr` to `4 x 4` complex matrices.*

```
def contrContrToMatrix : (complexContr ⊗ complexContr).V ≃ₗ[ℂ]
    Matrix (Fin 1 ⊕ Fin 3) (Fin 1 ⊕ Fin 3) ℂ
```

**lemma contrContrToMatrix_symm_expand_tmul** [source]

*Expanding `contrContrToMatrix` in terms of the standard basis.*

```
lemma contrContrToMatrix_symm_expand_tmul (M : Matrix (Fin 1 ⊕ Fin 3) (Fin 1 ⊕ Fin 3) ℂ) :
    contrContrToMatrix.symm M =
    ∑ i, ∑ j, M i j • (complexContrBasis i ⊗ₜ[ℂ] complexContrBasis j)
```

**def coCoToMatrix** [source]

*Equivalence of `complexCo ⊗ complexCo` to `4 x 4` complex matrices.*

```
def coCoToMatrix : (complexCo ⊗ complexCo).V ≃ₗ[ℂ]
    Matrix (Fin 1 ⊕ Fin 3) (Fin 1 ⊕ Fin 3) ℂ
```

**lemma coCoToMatrix_symm_expand_tmul** [source]

*Expanding `coCoToMatrix` in terms of the standard basis.*

```
lemma coCoToMatrix_symm_expand_tmul (M : Matrix (Fin 1 ⊕ Fin 3) (Fin 1 ⊕ Fin 3) ℂ) :
    coCoToMatrix.symm M = ∑ i, ∑ j, M i j • (complexCoBasis i ⊗ₜ[ℂ] complexCoBasis j)
```

**def contrCoToMatrix** [source]

*Equivalence of `complexContr ⊗ complexCo` to `4 x 4` complex matrices.*

```
def contrCoToMatrix : (complexContr ⊗ complexCo).V ≃ₗ[ℂ]
    Matrix (Fin 1 ⊕ Fin 3) (Fin 1 ⊕ Fin 3) ℂ
```

**lemma contrCoToMatrix_symm_expand_tmul** [source]

*Expansion of `contrCoToMatrix` in terms of the standard basis.*

```
lemma contrCoToMatrix_symm_expand_tmul (M : Matrix (Fin 1 ⊕ Fin 3) (Fin 1 ⊕ Fin 3) ℂ) :
    contrCoToMatrix.symm M = ∑ i, ∑ j, M i j • (complexContrBasis i ⊗ₜ[ℂ] complexCoBasis j)
```

**def coContrToMatrix** [source]

*Equivalence of `complexCo ⊗ complexContr` to `4 x 4` complex matrices.*

```
def coContrToMatrix : (complexCo ⊗ complexContr).V ≃ₗ[ℂ]
    Matrix (Fin 1 ⊕ Fin 3) (Fin 1 ⊕ Fin 3) ℂ
```

**lemma coContrToMatrix_symm_expand_tmul** [source]

*Expansion of `coContrToMatrix` in terms of the standard basis.*

```
lemma coContrToMatrix_symm_expand_tmul (M : Matrix (Fin 1 ⊕ Fin 3) (Fin 1 ⊕ Fin 3) ℂ) :
    coContrToMatrix.symm M = ∑ i, ∑ j, M i j • (complexCoBasis i ⊗ₜ[ℂ] complexContrBasis j)
```

**lemma contrContrToMatrix_ρ** [source]

```
lemma contrContrToMatrix_ρ (v : (complexContr ⊗ complexContr).V) (M : SL(2,ℂ)) :
    contrContrToMatrix (TensorProduct.map (complexContr.ρ M) (complexContr.ρ M) v) =
    (LorentzGroup.toComplex (SL2C.toLorentzGroup M)) * contrContrToMatrix v *
    (LorentzGroup.toComplex (SL2C.toLorentzGroup M))ᵀ
```

**lemma coCoToMatrix_ρ** [source]

```
lemma coCoToMatrix_ρ (v : (complexCo ⊗ complexCo).V) (M : SL(2,ℂ)) :
    coCoToMatrix (TensorProduct.map (complexCo.ρ M) (complexCo.ρ M) v) =
    (LorentzGroup.toComplex (SL2C.toLorentzGroup M))⁻¹ᵀ * coCoToMatrix v *
    (LorentzGroup.toComplex (SL2C.toLorentzGroup M))⁻¹
```

**lemma contrCoToMatrix_ρ** [source]

```
lemma contrCoToMatrix_ρ (v : (complexContr ⊗ complexCo).V) (M : SL(2,ℂ)) :
    contrCoToMatrix (TensorProduct.map (complexContr.ρ M) (complexCo.ρ M) v) =
    (LorentzGroup.toComplex (SL2C.toLorentzGroup M)) * contrCoToMatrix v *
    (LorentzGroup.toComplex (SL2C.toLorentzGroup M))⁻¹
```

**lemma coContrToMatrix_ρ** [source]

```
lemma coContrToMatrix_ρ (v : (complexCo ⊗ complexContr).V) (M : SL(2,ℂ)) :
    coContrToMatrix (TensorProduct.map (complexCo.ρ M) (complexContr.ρ M) v) =
    (LorentzGroup.toComplex (SL2C.toLorentzGroup M))⁻¹ᵀ * coContrToMatrix v *
    (LorentzGroup.toComplex (SL2C.toLorentzGroup M))ᵀ
```

**lemma contrContrToMatrix_ρ_symm** [source]

```
lemma contrContrToMatrix_ρ_symm (v : Matrix (Fin 1 ⊕ Fin 3) (Fin 1 ⊕ Fin 3) ℂ) (M : SL(2,ℂ)) :
    TensorProduct.map (complexContr.ρ M) (complexContr.ρ M) (contrContrToMatrix.symm v) =
    contrContrToMatrix.symm ((LorentzGroup.toComplex (SL2C.toLorentzGroup M)) * v *
    (LorentzGroup.toComplex (SL2C.toLorentzGroup M))ᵀ)
```

**lemma coCoToMatrix_ρ_symm** [source]

```
lemma coCoToMatrix_ρ_symm (v : Matrix (Fin 1 ⊕ Fin 3) (Fin 1 ⊕ Fin 3) ℂ) (M : SL(2,ℂ)) :
    TensorProduct.map (complexCo.ρ M) (complexCo.ρ M) (coCoToMatrix.symm v) =
    coCoToMatrix.symm ((LorentzGroup.toComplex (SL2C.toLorentzGroup M))⁻¹ᵀ * v *
    (LorentzGroup.toComplex (SL2C.toLorentzGroup M))⁻¹)
```

**lemma contrCoToMatrix_ρ_symm** [source]

```
lemma contrCoToMatrix_ρ_symm (v : Matrix (Fin 1 ⊕ Fin 3) (Fin 1 ⊕ Fin 3) ℂ) (M : SL(2,ℂ)) :
    TensorProduct.map (complexContr.ρ M) (complexCo.ρ M) (contrCoToMatrix.symm v) =
    contrCoToMatrix.symm ((LorentzGroup.toComplex (SL2C.toLorentzGroup M)) * v *
    (LorentzGroup.toComplex (SL2C.toLorentzGroup M))⁻¹)
```

**lemma coContrToMatrix_ρ_symm** [source]

```
lemma coContrToMatrix_ρ_symm (v : Matrix (Fin 1 ⊕ Fin 3) (Fin 1 ⊕ Fin 3) ℂ) (M : SL(2,ℂ)) :
    TensorProduct.map (complexCo.ρ M) (complexContr.ρ M) (coContrToMatrix.symm v) =
    coContrToMatrix.symm ((LorentzGroup.toComplex (SL2C.toLorentzGroup M))⁻¹ᵀ * v *
    (LorentzGroup.toComplex (SL2C.toLorentzGroup M))ᵀ)
```

## 2.10 HepLean.SpaceTime.LorentzVector.Complex.Unit

HepLean/SpaceTime/LorentzVector/Complex/Unit.lean on GitHub.

**def contrCoUnitVal** [source]

*The contra-co unit for complex lorentz vectors. Usually denoted $\delta^i{}_i$.*

```
def contrCoUnitVal : (complexContr ⊗ complexCo).V
```

**lemma contrCoUnitVal_expand_tmul** [source]

*Expansion of `contrCoUnitVal` into basis.*

```
lemma contrCoUnitVal_expand_tmul : contrCoUnitVal =
    complexContrBasis (Sum.inl 0) ⊗ₜ[ℂ] complexCoBasis (Sum.inl 0)
    + complexContrBasis (Sum.inr 0) ⊗ₜ[ℂ] complexCoBasis (Sum.inr 0)
    + complexContrBasis (Sum.inr 1) ⊗ₜ[ℂ] complexCoBasis (Sum.inr 1)
    + complexContrBasis (Sum.inr 2) ⊗ₜ[ℂ] complexCoBasis (Sum.inr 2)
```

**def contrCoUnit** [source]

*The contra-co unit for complex lorentz vectors as a morphism `𝟙_ (Rep ℂ SL(2,ℂ)) ⟶ complexContr ⊗ complexCo`, manifesting the invaraince under the `SL(2, ℂ)` action.*

```
def contrCoUnit : 𝟙_ (Rep ℂ SL(2,ℂ)) ⟶ complexContr ⊗ complexCo where
```

**lemma contrCoUnit_apply_one** [source]

```
lemma contrCoUnit_apply_one : contrCoUnit.hom (1 : ℂ) = contrCoUnitVal
```

**def coContrUnitVal** [source]

*The co-contra unit for complex lorentz vectors. Usually denoted $\delta_i{}^i$.*

```
def coContrUnitVal : (complexCo ⊗ complexContr).V
```

**lemma coContrUnitVal_expand_tmul** [source]

*Expansion of `coContrUnitVal` into basis.*

```
lemma coContrUnitVal_expand_tmul : coContrUnitVal =
    complexCoBasis (Sum.inl 0) ⊗ₜ[ℂ] complexContrBasis (Sum.inl 0)
    + complexCoBasis (Sum.inr 0) ⊗ₜ[ℂ] complexContrBasis (Sum.inr 0)
    + complexCoBasis (Sum.inr 1) ⊗ₜ[ℂ] complexContrBasis (Sum.inr 1)
    + complexCoBasis (Sum.inr 2) ⊗ₜ[ℂ] complexContrBasis (Sum.inr 2)
```

**def coContrUnit** [source]

*The co-contra unit for complex lorentz vectors as a morphism `𝟙_ (Rep ℂ SL(2,ℂ)) ⟶ complexCo ⊗ complexContr`, manifesting the invaraince under the `SL(2, ℂ)` action.*

```
def coContrUnit : 𝟙_ (Rep ℂ SL(2,ℂ)) ⟶ complexCo ⊗ complexContr where
```

**lemma coContrUnit_apply_one** [source]

```
lemma coContrUnit_apply_one : coContrUnit.hom (1 : ℂ) = coContrUnitVal
```

**lemma contr_contrCoUnit** [source]

*Contraction on the right with `contrCoUnit` does nothing.*

```
lemma contr_contrCoUnit (x : complexCo) :
    (λ_ complexCo).hom.hom
    ((coContrContraction ▷ complexCo).hom
    ((α_ _ _ complexCo).inv.hom
    (x ⊗ₜ[ℂ] contrCoUnit.hom (1 : ℂ)))) = x
```

**lemma contr_coContrUnit** [source]

*Contraction on the right with `coContrUnit`.*

```
lemma contr_coContrUnit (x : complexContr) :
    (λ_ complexContr).hom.hom
    ((contrCoContraction ▷ complexContr).hom
    ((α_ _ _ complexContr).inv.hom
    (x ⊗ₜ[ℂ] coContrUnit.hom (1 : ℂ)))) = x
```

**lemma contrCoUnit_symm** [source]

```
lemma contrCoUnit_symm :
    (contrCoUnit.hom (1 : ℂ)) = (complexContr ◁ 𝟙 _).hom ((β_ complexCo complexContr).hom.hom
    (coContrUnit.hom (1 : ℂ)))
```

**lemma coContrUnit_symm** [source]

```
lemma coContrUnit_symm :
    (coContrUnit.hom (1 : ℂ)) = (complexCo ◁ ⅄ _).hom ((β_ complexContr complexCo).hom.hom
    (contrCoUnit.hom (1 : ℂ)))
```

## 2.11 HepLean.SpaceTime.LorentzVector.Modules

HepLean/SpaceTime/LorentzVector/Modules.lean on GitHub.

**structure ContrℂModule** [source]

*The module for contravariant (up-index) complex Lorentz vectors.*

```
structure ContrℂModule where
  val : Fin 1 ⊕ Fin 3 → ℂ
```

**def ContrℂModule.toFin13ℂFun** [source]

*The equivalence between `ContrℂModule` and `Fin 1 ⊕ Fin 3 → ℂ`.*

```
def toFin13ℂFun : ContrℂModule ≃ (Fin 1 ⊕ Fin 3 → ℂ) where
```

**instance AddCommMonoid ContrℂModule** [source]

*The instance of `AddCommMonoid` on `ContrℂModule` defined via its equivalence with `Fin 1 ⊕ Fin 3 → ℂ`.*

```
instance : AddCommMonoid ContrℂModule
```

**instance AddCommGroup ContrℂModule** [source]

*The instance of `AddCommGroup` on `ContrℂModule` defined via its equivalence with `Fin 1 ⊕ Fin 3 → ℂ`.*

```
instance : AddCommGroup ContrℂModule
```

**instance Module ℂ ContrℂModule** [source]

*The instance of `Module` on `ContrℂModule` defined via its equivalence with `Fin 1 ⊕ Fin 3 → ℂ`.*

```
instance : Module ℂ ContrℂModule
```

**lemma ext** [source]

```
@[ext]
lemma ext (ψ ψ' : ContrℂModule) (h : ψ.val = ψ'.val) : ψ = ψ'
```

**lemma val_add** [source]

```
@[simp]
lemma val_add (ψ ψ' : ContrℂModule) : (ψ + ψ').val = ψ.val + ψ'.val
```

**lemma val_smul** [source]

```
@[simp]
lemma val_smul (r : ℂ) (ψ : ContrℂModule) : (r • ψ).val = r • ψ.val
```

**def ContrℂModule.toFin13ℂEquiv** [source]

*The linear equivalence between `ContrℂModule` and `(Fin 1 ⊕ Fin 3 → ℂ)`.*

```
@[simps!]
def toFin13ℂEquiv : ContrℂModule ≃ₗ[ℂ] (Fin 1 ⊕ Fin 3 → ℂ) where
```

**abbrev ContrℂModule.toFin13ℂ** [source]

*The underlying element of `Fin 1 ⊕ Fin 3 → ℂ` of a element in `ContrℂModule` defined through the linear equivalence `toFin13ℂEquiv`.*

```
abbrev toFin13ℂ (ψ : ContrℂModule)
```

**def ContrℂModule.lorentzGroupRep** [source]

*The representation of the Lorentz group on `ContrℂModule`.*

```
def lorentzGroupRep : Representation ℂ (LorentzGroup 3) ContrℂModule where
```

**def ContrℂModule.SL2CRep** [source]

*The representation of the SL(2, ℂ) on `ContrℂModule` induced by the representation of the Lorentz group.*

```
def SL2CRep : Representation ℂ SL(2, ℂ) ContrℂModule
```

**structure CoℂModule** [source]

*The module for covariant (up-index) complex Lorentz vectors.*

```
structure CoℂModule where
  val : Fin 1 ⊕ Fin 3 → ℂ
```

**def CoℂModule.toFin13ℂFun** [source]

*The equivalence between `CoℂModule` and `Fin 1 ⊕ Fin 3 → ℂ`.*

```
def toFin13ℂFun : CoℂModule ≃ (Fin 1 ⊕ Fin 3 → ℂ) where
```

**instance AddCommMonoid CoℂModule** [source]

*The instance of `AddCommMonoid` on `CoℂModule` defined via its equivalence with `Fin 1 ⊕ Fin 3 → ℂ`.*

```
instance : AddCommMonoid CoℂModule
```

**instance AddCommGroup CoℂModule** [source]

*The instance of `AddCommGroup` on `CoℂModule` defined via its equivalence with `Fin 1 ⊕ Fin 3 → ℂ`.*

```
instance : AddCommGroup CoℂModule
```

**instance Module ℂ CoℂModule** [source]

*The instance of `Module` on `CoℂModule` defined via its equivalence with `Fin 1 ⊕ Fin 3 → ℂ`.*

```
instance : Module ℂ CoℂModule
```

**def CoℂModule.toFin13ℂEquiv** [source]

*The linear equivalence between `CoℂModule` and `(Fin 1 ⊕ Fin 3 → ℂ)`.*

```
@[simps!]
def toFin13ℂEquiv : CoℂModule ≃ₗ[ℂ] (Fin 1 ⊕ Fin 3 → ℂ) where
```

**abbrev CoℂModule.toFin13ℂ** [source]

*The underlying element of `Fin 1 ⊕ Fin 3 → ℂ` of a element in `CoℂModule` defined through the linear equivalence `toFin13ℂEquiv`.*

```
abbrev toFin13ℂ (ψ : CoℂModule)
```

**def CoℂModule.lorentzGroupRep** [source]

*The representation of the Lorentz group on `CoℂModule`.*

```
def lorentzGroupRep : Representation ℂ (LorentzGroup 3) CoℂModule where
```

**def CoℂModule.SL2CRep** [source]

*The representation of the SL(2, ℂ) on `ContrℂModule` induced by the representation of the Lorentz group.*

```
def SL2CRep : Representation ℂ SL(2, ℂ) CoℂModule
```

## 2.12 HepLean.SpaceTime.MinkowskiMetric

HepLean/SpaceTime/MinkowskiMetric.lean on GitHub.

**lemma inl_0_inl_0** [source]

```
lemma inl_0_inl_0 : @minkowskiMatrix d (Sum.inl 0) (Sum.inl 0) = 1
```

**lemma inr_i_inr_i** [source]

```
lemma inr_i_inr_i (i : Fin d) : @minkowskiMatrix d (Sum.inr i) (Sum.inr i) = -1
```

**lemma dual_apply** [source]

```
lemma dual_apply (μ ν : Fin 1 ⊕ Fin d) :
    dual Λ μ ν = η μ μ * Λ ν μ * η ν ν
```

**lemma dual_apply_minkowskiMatrix** [source]

```
lemma dual_apply_minkowskiMatrix (μ ν : Fin 1 ⊕ Fin d) :
    dual Λ μ ν * η ν ν = η μ μ * Λ ν μ
```

## 2.13 HepLean.SpaceTime.PauliMatrices.AsTensor

HepLean/SpaceTime/PauliMatrices/AsTensor.lean on GitHub.

**def asTensor** [source]

*The tensor σ^μ^a^{dot a} based on the Pauli-matrices as an element of complexContr ⊗ leftHanded ⊗ rightHanded.*

```
def asTensor : (complexContr ⊗ leftHanded ⊗ rightHanded).V
```

**lemma asTensor_expand_complexContrBasis** [source]

*The expansion of asTensor into complexContrBasis basis vectors .*

```
lemma asTensor_expand_complexContrBasis : asTensor =
    complexContrBasis (Sum.inl 0) ⊗ₜ leftRightToMatrix.symm (σSA (Sum.inl 0))
    + complexContrBasis (Sum.inr 0) ⊗ₜ leftRightToMatrix.symm (σSA (Sum.inr 0))
    + complexContrBasis (Sum.inr 1) ⊗ₜ leftRightToMatrix.symm (σSA (Sum.inr 1))
    + complexContrBasis (Sum.inr 2) ⊗ₜ leftRightToMatrix.symm (σSA (Sum.inr 2))
```

**lemma leftRightToMatrix_σSA_inl_0_expand** [source]

*The expansion of the pauli matrix $\sigma_0$ in terms of a basis of tensor product vectors.*

```
lemma leftRightToMatrix_σSA_inl_0_expand : leftRightToMatrix.symm (σSA (Sum.inl 0)) =
    leftBasis 0 ⊗ₜ rightBasis 0 + leftBasis 1 ⊗ₜ rightBasis 1
```

**lemma leftRightToMatrix_σSA_inr_0_expand** [source]

*The expansion of the pauli matrix $\sigma_1$ in terms of a basis of tensor product vectors.*

```
lemma leftRightToMatrix_σSA_inr_0_expand : leftRightToMatrix.symm (σSA (Sum.inr 0)) =
    leftBasis 0 ⊗ₜ rightBasis 1 + leftBasis 1 ⊗ₜ rightBasis 0
```

**lemma leftRightToMatrix_σSA_inr_1_expand** [source]

*The expansion of the pauli matrix $\sigma_2$ in terms of a basis of tensor product vectors.*

```
lemma leftRightToMatrix_σSA_inr_1_expand : leftRightToMatrix.symm (σSA (Sum.inr 1)) =
    -(I • leftBasis 0 ⊗ₜ[ℂ] rightBasis 1) + I • leftBasis 1 ⊗ₜ[ℂ] rightBasis 0
```

**lemma leftRightToMatrix_σSA_inr_2_expand** [source]

*The expansion of the pauli matrix σ₃ in terms of a basis of tensor product vectors.*

```
lemma leftRightToMatrix_σSA_inr_2_expand : leftRightToMatrix.symm (σSA (Sum.inr 2)) =
    leftBasis 0 ⊗ₜ rightBasis 0 - leftBasis 1 ⊗ₜ rightBasis 1
```

**lemma asTensor_expand** [source]

*The expansion of `asTensor` into complexContrBasis basis of tensor product vectors.*

```
lemma asTensor_expand : asTensor =
    complexContrBasis (Sum.inl 0) ⊗ₜ (leftBasis 0 ⊗ₜ rightBasis 0)
    + complexContrBasis (Sum.inl 0) ⊗ₜ (leftBasis 1 ⊗ₜ rightBasis 1)
    + complexContrBasis (Sum.inr 0) ⊗ₜ (leftBasis 0 ⊗ₜ rightBasis 1)
    + complexContrBasis (Sum.inr 0) ⊗ₜ (leftBasis 1 ⊗ₜ rightBasis 0)
    - I • complexContrBasis (Sum.inr 1) ⊗ₜ (leftBasis 0 ⊗ₜ rightBasis 1)
    + I • complexContrBasis (Sum.inr 1) ⊗ₜ (leftBasis 1 ⊗ₜ rightBasis 0)
    + complexContrBasis (Sum.inr 2) ⊗ₜ (leftBasis 0 ⊗ₜ rightBasis 0)
    - complexContrBasis (Sum.inr 2) ⊗ₜ (leftBasis 1 ⊗ₜ rightBasis 1)
```

**def asConsTensor** [source]

*The tensor `σ^μ^a^{dot a}` based on the Pauli-matrices as a morphism, `𝓚_ (Rep ℂ SL(2,ℂ)) ⟶ complexContr ⊗ leftHanded ⊗ rightHanded` manifesting the invariance under the `SL(2,ℂ)` action.*

```
def asConsTensor : 𝓚_ (Rep ℂ SL(2,ℂ)) ⟶ complexContr ⊗ leftHanded ⊗ rightHanded where
```

**lemma asConsTensor_apply_one** [source]

```
lemma asConsTensor_apply_one : asConsTensor.hom (1 : ℂ) = asTensor
```

## 2.14 HepLean.SpaceTime.PauliMatrices.Basic

HepLean/SpaceTime/PauliMatrices/Basic.lean on GitHub.

**def σ0** [source]

*The zeroth Pauli-matrix as a `2 x 2` complex matrix.*

```
def σ0 : Matrix (Fin 2) (Fin 2) ℂ
```

**def σ1** [source]

*The first Pauli-matrix as a `2 x 2` complex matrix.*

```
def σ1 : Matrix (Fin 2) (Fin 2) ℂ
```

**def σ2** [source]

*The second Pauli-matrix as a `2 x 2` complex matrix.*

```
def σ2 : Matrix (Fin 2) (Fin 2) ℂ
```

**def σ3** [source]

*The third Pauli-matrix as a `2 x 2` complex matrix.*

```
def σ3 : Matrix (Fin 2) (Fin 2) ℂ
```

**lemma σ0_selfAdjoint** [source]

```
@[simp]
lemma σ0_selfAdjoint : σ0^H = σ0
```

**lemma σ1_selfAdjoint** [source]

```
@[simp]
lemma σ1_selfAdjoint : σ1^H = σ1
```

**lemma σ2_selfAdjoint** [source]

```
@[simp]
lemma σ2_selfAdjoint : σ2^H = σ2
```

**lemma σ3_selfAdjoint** [source]

```
@[simp]
lemma σ3_selfAdjoint : σ3^H = σ3
```

**lemma σ0_σ0_trace** [source]

```
@[simp]
lemma σ0_σ0_trace : Matrix.trace (σ0 * σ0) = 2
```

**lemma σ0_σ1_trace** [source]

```
@[simp]
lemma σ0_σ1_trace : Matrix.trace (σ0 * σ1) = 0
```

**lemma σ0_σ2_trace** [source]

```
@[simp]
lemma σ0_σ2_trace : Matrix.trace (σ0 * σ2) = 0
```

**lemma σ0_σ3_trace** [source]

```
@[simp]
lemma σ0_σ3_trace : Matrix.trace (σ0 * σ3) = 0
```

**lemma σ1_σ0_trace** [source]

```
@[simp]
lemma σ1_σ0_trace : Matrix.trace (σ1 * σ0) = 0
```

**lemma σ1_σ1_trace** [source]

```
@[simp]
lemma σ1_σ1_trace : Matrix.trace (σ1 * σ1) = 2
```

**lemma σ1_σ2_trace** [source]

```
@[simp]
lemma σ1_σ2_trace : Matrix.trace (σ1 * σ2) = 0
```

**lemma σ1_σ3_trace** [source]

```
@[simp]
lemma σ1_σ3_trace : Matrix.trace (σ1 * σ3) = 0
```

**lemma σ2_σ0_trace** [source]

```
@[simp]
lemma σ2_σ0_trace : Matrix.trace (σ2 * σ0) = 0
```

**lemma σ2_σ1_trace** [source]

```
@[simp]
lemma σ2_σ1_trace : Matrix.trace (σ2 * σ1) = 0
```

**lemma σ2_σ2_trace** [source]

```
@[simp]
lemma σ2_σ2_trace : Matrix.trace (σ2 * σ2) = 2
```

**lemma σ2_σ3_trace** [source]

```
@[simp]
lemma σ2_σ3_trace : Matrix.trace (σ2 * σ3) = 0
```

**lemma σ3_σ0_trace** [source]

```
@[simp]
lemma σ3_σ0_trace : Matrix.trace (σ3 * σ0) = 0
```

**lemma σ3_σ1_trace** [source]

```
@[simp]
lemma σ3_σ1_trace : Matrix.trace (σ3 * σ1) = 0
```

**lemma σ3_σ2_trace** [source]

```
@[simp]
lemma σ3_σ2_trace : Matrix.trace (σ3 * σ2) = 0
```

**lemma σ3_σ3_trace** [source]

```
@[simp]
lemma σ3_σ3_trace : Matrix.trace (σ3 * σ3) = 2
```

## 2.15 HepLean.SpaceTime.PauliMatrices.SelfAdjoint

HepLean/SpaceTime/PauliMatrices/SelfAdjoint.lean on GitHub.

**lemma selfAdjoint_trace_σ0_real** [source]

```
lemma selfAdjoint_trace_σ0_real (A : selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)) :
    (Matrix.trace (σ0 * A.1)).re = Matrix.trace (σ0 * A.1)
```

**lemma selfAdjoint_trace_σ1_real** [source]

```
lemma selfAdjoint_trace_σ1_real (A : selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)) :
    (Matrix.trace (σ1 * A.1)).re = Matrix.trace (σ1 * A.1)
```

**lemma selfAdjoint_trace_σ2_real** [source]

```
lemma selfAdjoint_trace_σ2_real (A : selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)) :
    (Matrix.trace (σ2 * A.1)).re = Matrix.trace (σ2 * A.1)
```

**lemma selfAdjoint_trace_σ3_real** [source]

```
lemma selfAdjoint_trace_σ3_real (A : selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)) :
    (Matrix.trace (σ3 * A.1)).re = Matrix.trace (σ3 * A.1)
```

**lemma selfAdjoint_ext_complex** [source]

```
lemma selfAdjoint_ext_complex {A B : selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)}
    (h0 : ((Matrix.trace (PauliMatrix.σ0 * A.1)) = (Matrix.trace (PauliMatrix.σ0 * B.1))))
    (h1 : Matrix.trace (PauliMatrix.σ1 * A.1) = Matrix.trace (PauliMatrix.σ1 * B.1))
    (h2 : Matrix.trace (PauliMatrix.σ2 * A.1) = Matrix.trace (PauliMatrix.σ2 * B.1))
    (h3 : Matrix.trace (PauliMatrix.σ3 * A.1) = Matrix.trace (PauliMatrix.σ3 * B.1)) : A = B
```

**lemma selfAdjoint_ext** [source]

```
lemma selfAdjoint_ext {A B : selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)}
    (h0 : ((Matrix.trace (PauliMatrix.σ0 * A.1))).re = ((Matrix.trace (PauliMatrix.σ0 * B.1))).
    re)
    (h1 : ((Matrix.trace (PauliMatrix.σ1 * A.1))).re = ((Matrix.trace (PauliMatrix.σ1 * B.1))).
    re)
    (h2 : ((Matrix.trace (PauliMatrix.σ2 * A.1))).re = ((Matrix.trace (PauliMatrix.σ2 * B.1))).
    re)
    (h3 : ((Matrix.trace (PauliMatrix.σ3 * A.1))).re = ((Matrix.trace (PauliMatrix.σ3 * B.1))).
    re) :
    A = B
```

**def σSA'** [source]

*An auxillary function which on `i : Fin 1 ⊕ Fin 3` returns the corresponding Pauli-matrix as a self-adjoint matrix.*

```
def σSA' (i : Fin 1 ⊕ Fin 3) : selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)
```

**lemma σSA_linearly_independent** [source]

```
lemma σSA_linearly_independent : LinearIndependent ℝ σSA'
```

**lemma σSA_span** [source]

```
lemma σSA_span : ⊤ ≤ Submodule.span ℝ (Set.range σSA')
```

**def σSA** [source]

*The basis of `selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)` formed by Pauli matrices.*

```
def σSA : Basis (Fin 1 ⊕ Fin 3) ℝ (selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ))
```

**def σSAL'** [source]

*An auxillary function which on `i : Fin 1 ⊕ Fin 3` returns the corresponding Pauli-matrix as a self-adjoint matrix with a minus sign for `Sum.inr _`.*

```
def σSAL' (i : Fin 1 ⊕ Fin 3) : selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)
```

**lemma σSAL_linearly_independent** [source]

```
lemma σSAL_linearly_independent : LinearIndependent ℝ σSAL'
```

**lemma σSAL_span** [source]

```
lemma σSAL_span : ⊤ ≤ Submodule.span ℝ (Set.range σSAL')
```

**def σSAL** [source]

*The basis of `selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)` formed by Pauli matrices where the `1, 2, 3` pauli matrices are negated.*

```
def σSAL : Basis (Fin 1 ⊕ Fin 3) ℝ (selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ))
```

**lemma σSAL_decomp** [source]

```
lemma σSAL_decomp (M : selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)) :
    M = (1/2 * (Matrix.trace (σ0 * M.1)).re) • σSAL (Sum.inl 0)
    + (-1/2 * (Matrix.trace (σ1 * M.1)).re) • σSAL (Sum.inr 0)
    + (-1/2 * (Matrix.trace (σ2 * M.1)).re) • σSAL (Sum.inr 1)
    + (-1/2 * (Matrix.trace (σ3 * M.1)).re) • σSAL (Sum.inr 2)
```

**lemma σSAL_repr_inl_0** [source]

```
@[simp]
lemma σSAL_repr_inl_0 (M : selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)) :
    σSAL.repr M (Sum.inl 0) = 1 / 2 * Matrix.trace (σ0 * M.1)
```

**lemma σSAL_repr_inr_0** [source]

```
@[simp]
lemma σSAL_repr_inr_0 (M : selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)) :
    σSAL.repr M (Sum.inr 0) = - 1 / 2 * Matrix.trace (σ1 * M.1)
```

**lemma σSAL_repr_inr_1** [source]

```
@[simp]
lemma σSAL_repr_inr_1 (M : selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)) :
    σSAL.repr M (Sum.inr 1) = - 1 / 2 * Matrix.trace (σ2 * M.1)
```

**lemma σSAL_repr_inr_2** [source]

```
@[simp]
lemma σSAL_repr_inr_2 (M : selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)) :
    σSAL.repr M (Sum.inr 2) = - 1 / 2 * Matrix.trace (σ3 * M.1)
```

**lemma σSA_minkowskiMetric_σSAL** [source]

```
lemma σSA_minkowskiMetric_σSAL (i : Fin 1 ⊕ Fin 3) :
    (σSA i) = minkowskiMatrix i i • (σSAL i)
```

## 2.16 HepLean.SpaceTime.SL2C.Basic

HepLean/SpaceTime/SL2C/Basic.lean on GitHub.

**lemma inverse_coe** [source]

```
lemma inverse_coe (M : SL(2, ℂ)) : M.1⁻¹ = (M⁻¹).1
```

**lemma transpose_coe** [source]

```
lemma transpose_coe (M : SL(2, ℂ)) : M.1ᵀ = (M.transpose).1
```

**lemma toLorentzGroup_eq_σSAL** [source]

```
lemma toLorentzGroup_eq_σSAL (M : SL(2, ℂ)) :
    toLorentzGroup M = LinearMap.toMatrix
    PauliMatrix.σSAL PauliMatrix.σSAL (repSelfAdjointMatrix M)
```

**lemma toLorentzGroup_eq_stdBasis** [source]

```
lemma toLorentzGroup_eq_stdBasis (M : SL(2, ℂ)) :
    toLorentzGroup M = LinearMap.toMatrix LorentzVector.stdBasis LorentzVector.stdBasis
    (repLorentzVector M)
```

**lemma repLorentzVector_apply_eq_mulVec** [source]

```
lemma repLorentzVector_apply_eq_mulVec (v : LorentzVector 3) :
    SL2C.repLorentzVector M v = (SL2C.toLorentzGroup M).1 *ᵥ v
```

**lemma repSelfAdjointMatrix_basis** [source]

```
lemma repSelfAdjointMatrix_basis (i : Fin 1 ⊕ Fin 3) :
    SL2C.repSelfAdjointMatrix M (PauliMatrix.σSAL i) =
    ∑ j, (toLorentzGroup M).1 j i •
    PauliMatrix.σSAL j
```

**lemma repSelfAdjointMatrix_σSA** [source]

```
lemma repSelfAdjointMatrix_σSA (i : Fin 1 ⊕ Fin 3) :
    SL2C.repSelfAdjointMatrix M (PauliMatrix.σSA i) =
    ∑ j, (toLorentzGroup M⁻¹).1 i j • PauliMatrix.σSA j
```

**lemma repLorentzVector_stdBasis** [source]

```
lemma repLorentzVector_stdBasis (i : Fin 1 ⊕ Fin 3) :
    SL2C.repLorentzVector M (LorentzVector.stdBasis i) =
    ∑ j, (toLorentzGroup M).1 j i • LorentzVector.stdBasis j
```

## 2.17 HepLean.SpaceTime.WeylFermion.Basic

HepLean/SpaceTime/WeylFermion/Basic.lean on GitHub.

**def leftHanded** [source]

*The vector space ℂ^2 carrying the fundamental representation of SL(2,C). In index notation corresponds to a Weyl fermion with indices ψ^a.*

```
def leftHanded : Rep ℂ SL(2,ℂ)
```

**def leftBasis** [source]

*The standard basis on left-handed Weyl fermions.*

```
def leftBasis : Basis (Fin 2) ℂ leftHanded
```

**lemma leftBasis_ρ_apply** [source]

```
@[simp]
lemma leftBasis_ρ_apply (M : SL(2,ℂ)) (i j : Fin 2) :
    (LinearMap.toMatrix leftBasis leftBasis) (leftHanded.ρ M) i j = M.1 i j
```

**lemma leftBasis_toFin2ℂ** [source]

```
@[simp]
lemma leftBasis_toFin2ℂ (i : Fin 2) : (leftBasis i).toFin2ℂ = Pi.single i 1
```

**def altLeftHanded** [source]

*The vector space ℂ^2 carrying the representation of SL(2,C) given by $M \to (M^{-1})^T$. In index notation corresponds to a Weyl fermion with indices ψ_a.*

```
def altLeftHanded : Rep ℂ SL(2,ℂ)
```

**def altLeftBasis** [source]

*The standard basis on alt-left-handed Weyl fermions.*

```
def altLeftBasis : Basis (Fin 2) ℂ altLeftHanded
```

**lemma altLeftBasis_toFin2ℂ** [source]

```
@[simp]
lemma altLeftBasis_toFin2ℂ (i : Fin 2) : (altLeftBasis i).toFin2ℂ = Pi.single i 1
```

**lemma altLeftBasis_ρ_apply** [source]

```
@[simp]
lemma altLeftBasis_ρ_apply (M : SL(2,ℂ)) (i j : Fin 2) :
    (LinearMap.toMatrix altLeftBasis altLeftBasis) (altLeftHanded.ρ M) i j = (M.1⁻¹)ᵀ i j
```

**def rightHanded** [source]

*The vector space ℂ^2 carrying the conjugate representation of SL(2,C). In index notation corresponds to a Weyl fermion with indices ψ^{dot a}.*

```
def rightHanded : Rep ℂ SL(2,ℂ)
```

**def rightBasis** [source]

*The standard basis on right-handed Weyl fermions.*

```
def rightBasis : Basis (Fin 2) ℂ rightHanded
```

**lemma rightBasis_toFin2ℂ** [source]

```
@[simp]
lemma rightBasis_toFin2ℂ (i : Fin 2) : (rightBasis i).toFin2ℂ = Pi.single i 1
```

**lemma rightBasis_ρ_apply** [source]

```
@[simp]
lemma rightBasis_ρ_apply (M : SL(2,ℂ)) (i j : Fin 2) :
    (LinearMap.toMatrix rightBasis rightBasis) (rightHanded.ρ M) i j = (M.1.map star) i j
```

**def altRightHanded** [source]

*The vector space ℂ^2 carrying the representation of SL(2,C) given by $M \to (M^{-1})^\dagger$. In index notation this corresponds to a Weyl fermion with index ψ_{dot a}.*

```
def altRightHanded : Rep ℂ SL(2,ℂ)
```

**def altRightBasis** [source]

*The standard basis on alt-right-handed Weyl fermions.*

```
def altRightBasis : Basis (Fin 2) ℂ altRightHanded
```

**lemma altRightBasis_toFin2ℂ** [source]

```
@[simp]
lemma altRightBasis_toFin2ℂ (i : Fin 2) : (altRightBasis i).toFin2ℂ = Pi.single i 1
```

**lemma altRightBasis_ρ_apply** [source]

```
@[simp]
lemma altRightBasis_ρ_apply (M : SL(2,ℂ)) (i j : Fin 2) :
    (LinearMap.toMatrix altRightBasis altRightBasis) (altRightHanded.ρ M) i j =
    ((M.1⁻¹).conjTranspose) i j
```

**def leftHandedToAlt** [source]

*The morphism between the representation `leftHanded` and the representation `altLeftHanded` defined by multiplying an element of `leftHanded` by the matrix `ε^a₀a₁ = !![0, 1; -1, 0]`.*

```
def leftHandedToAlt : leftHanded ⟶ altLeftHanded where
```

**lemma leftHandedToAlt_hom_apply** [source]

```
lemma leftHandedToAlt_hom_apply (ψ : leftHanded) :
    leftHandedToAlt.hom ψ =
    AltLeftHandedModule.toFin2ℂEquiv.symm (!![0, 1; -1, 0] *ᵥ ψ.toFin2ℂ)
```

**def leftHandedAltTo** [source]

*The morphism from `altLeftHanded` to `leftHanded` defined by multiplying an element of altLeftHandedWeyl by the matrix `ε_a₁a₂ = !![0, -1; 1, 0]`.*

```
def leftHandedAltTo : altLeftHanded ⟶ leftHanded where
```

**lemma leftHandedAltTo_hom_apply** [source]

```
lemma leftHandedAltTo_hom_apply (ψ : altLeftHanded) :
    leftHandedAltTo.hom ψ =
    LeftHandedModule.toFin2ℂEquiv.symm (!![0, -1; 1, 0] *ᵥ ψ.toFin2ℂ)
```

**def leftHandedAltEquiv** [source]

*The equivalence between the representation `leftHanded` and the representation `altLeftHanded` defined by multiplying an element of `leftHanded` by the matrix `ε^a₀a₁ = !![0, 1; -1, 0]`.*

```
def leftHandedAltEquiv : leftHanded ≅ altLeftHanded where
```

**lemma leftHandedAltEquiv_hom_hom_apply** [source]

```
lemma leftHandedAltEquiv_hom_hom_apply (ψ : leftHanded) :
    leftHandedAltEquiv.hom.hom ψ =
    AltLeftHandedModule.toFin2ℂEquiv.symm (!![0, 1; -1, 0] *ᵥ ψ.toFin2ℂ)
```

**lemma leftHandedAltEquiv_inv_hom_apply** [source]

```
lemma leftHandedAltEquiv_inv_hom_apply (ψ : altLeftHanded) :
    leftHandedAltEquiv.inv.hom ψ =
    LeftHandedModule.toFin2ℂEquiv.symm (!![0, -1; 1, 0] *ᵥ ψ.toFin2ℂ)
```

## 2.18 HepLean.SpaceTime.WeylFermion.Contraction

HepLean/SpaceTime/WeylFermion/Contraction.lean on GitHub.

**def leftAltBi** [source]

*The bi-linear map corresponding to contraction of a left-handed Weyl fermion with a alt-left-handed Weyl fermion.*

```
def leftAltBi : leftHanded →ₗ[ℂ] altLeftHanded →ₗ[ℂ] ℂ where
```

**def altLeftBi** [source]

*The bi-linear map corresponding to contraction of a alt-left-handed Weyl fermion with a left-handed Weyl fermion.*

```
def altLeftBi : altLeftHanded →ₗ[ℂ] leftHanded →ₗ[ℂ] ℂ where
```

**def rightAltBi** [source]

*The bi-linear map corresponding to contraction of a right-handed Weyl fermion with a alt-right-handed Weyl fermion.*

```
def rightAltBi : rightHanded →ₗ[ℂ] altRightHanded →ₗ[ℂ] ℂ where
```

**def altRightBi** [source]

*The bi-linear map corresponding to contraction of a alt-right-handed Weyl fermion with a right-handed Weyl fermion.*

```
def altRightBi : altRightHanded →ₗ[ℂ] rightHanded →ₗ[ℂ] ℂ where
```

**def leftAltContraction** [source]

*The linear map from leftHandedWeyl ⊗ altLeftHandedWeyl to ℂ given by summing over components of leftHandedWeyl and altLeftHandedWeyl in the standard basis (i.e. the dot product). Physically, the contraction of a left-handed Weyl fermion with a alt-left-handed Weyl fermion. In index notation this is ψ^a φ_a.*

```
def leftAltContraction : leftHanded ⊗ altLeftHanded ⟶ 𝟙_ (Rep ℂ SL(2,ℂ)) where
```

**lemma leftAltContraction_hom_tmul** [source]

```
lemma leftAltContraction_hom_tmul (ψ : leftHanded) (φ : altLeftHanded) :
    leftAltContraction.hom (ψ ⊗ₜ φ) = ψ.toFin2ℂ ⬝ᵥ φ.toFin2ℂ
```

**lemma leftAltContraction_basis** [source]

```
lemma leftAltContraction_basis (i j : Fin 2) :
    leftAltContraction.hom (leftBasis i ⊗ₜ altLeftBasis j) = if i.1 = j.1 then (1 : ℂ) else 0
```

**def altLeftContraction** [source]

*The linear map from altLeftHandedWeyl ⊗ leftHandedWeyl to ℂ given by summing over components of altLeftHandedWeyl and leftHandedWeyl in the standard basis (i.e. the dot product). Physically, the contraction of a alt-left-handed Weyl fermion with a left-handed Weyl fermion. In index notation this is φ_a ψ^a.*

```
def altLeftContraction : altLeftHanded ⊗ leftHanded ⟶ 𝟙_ (Rep ℂ SL(2,ℂ)) where
```

**lemma altLeftContraction_hom_tmul** [source]

```
lemma altLeftContraction_hom_tmul (φ : altLeftHanded) (ψ : leftHanded) :
    altLeftContraction.hom (φ ⊗ₜ ψ) = φ.toFin2ℂ ⬝ᵥ ψ.toFin2ℂ
```

**lemma altLeftContraction_basis** [source]

```
lemma altLeftContraction_basis (i j : Fin 2) :
    altLeftContraction.hom (altLeftBasis i ⊗ₜ leftBasis j) = if i.1 = j.1 then (1 : ℂ) else 0
```

**def rightAltContraction** [source]

*The linear map from rightHandedWeyl ⊗ altRightHandedWeyl to ℂ given by summing over components of rightHandedWeyl and altRightHandedWeyl in the standard basis (i.e. the dot product). The contraction of a right-handed Weyl fermion with a left-handed Weyl fermion. In index notation this is ψ^{dot a} φ_{dot a}.*

```
def rightAltContraction : rightHanded ⊗ altRightHanded ⟶ 𝟙_ (Rep ℂ SL(2,ℂ)) where
```

**lemma rightAltContraction_hom_tmul** [source]

```
lemma rightAltContraction_hom_tmul (ψ : rightHanded) (φ : altRightHanded) :
    rightAltContraction.hom (ψ ⊗ₜ φ) = ψ.toFin2ℂ ⬝ᵥ φ.toFin2ℂ
```

**lemma rightAltContraction_basis** [source]

```
lemma rightAltContraction_basis (i j : Fin 2) :
    rightAltContraction.hom (rightBasis i ⊗ₜ altRightBasis j) =
    if i.1 = j.1 then (1 : ℂ) else 0
```

**def altRightContraction** [source]

*The linear map from altRightHandedWeyl ⊗ rightHandedWeyl to ℂ given by summing over components of altRightHandedWeyl and rightHandedWeyl in the standard basis (i.e. the dot product). The contraction of a right-handed Weyl fermion with a left-handed Weyl fermion. In index notation this is φ_{dot a} ψ^{dot a}.*

```
def altRightContraction : altRightHanded ⊗ rightHanded ⟶ 𝟙_ (Rep ℂ SL(2,ℂ)) where
```

**lemma altRightContraction_hom_tmul** [source]

```
lemma altRightContraction_hom_tmul (φ : altRightHanded) (ψ : rightHanded) :
    altRightContraction.hom (φ ⊗ₜ ψ) = φ.toFin2ℂ ⬝ᵥ ψ.toFin2ℂ
```

**lemma altRightContraction_basis** [source]

```
lemma altRightContraction_basis (i j : Fin 2) :
    altRightContraction.hom (altRightBasis i ⊗ₜ rightBasis j) =
    if i.1 = j.1 then (1 : ℂ) else 0
```

**lemma leftAltContraction_tmul_symm** [source]

```
lemma leftAltContraction_tmul_symm (ψ : leftHanded) (φ : altLeftHanded) :
    leftAltContraction.hom (ψ ⊗ₜ[ℂ] φ) = altLeftContraction.hom (φ ⊗ₜ[ℂ] ψ)
```

**lemma altLeftContraction_tmul_symm** [source]

```
lemma altLeftContraction_tmul_symm (φ : altLeftHanded) (ψ : leftHanded) :
    altLeftContraction.hom (φ ⊗ₜ[ℂ] ψ) = leftAltContraction.hom (ψ ⊗ₜ[ℂ] φ)
```

**lemma rightAltContraction_tmul_symm** [source]

```
lemma rightAltContraction_tmul_symm (ψ : rightHanded) (φ : altRightHanded) :
    rightAltContraction.hom (ψ ⊗ₜ[ℂ] φ) = altRightContraction.hom (φ ⊗ₜ[ℂ] ψ)
```

**lemma altRightContraction_tmul_symm** [source]

```
lemma altRightContraction_tmul_symm (φ : altRightHanded) (ψ : rightHanded) :
    altRightContraction.hom (φ ⊗ₜ[ℂ] ψ) = rightAltContraction.hom (ψ ⊗ₜ[ℂ] φ)
```

## 2.19 HepLean.SpaceTime.WeylFermion.Metric

HepLean/SpaceTime/WeylFermion/Metric.lean on GitHub.

**def metricRaw** [source]

*The raw 2x2 matrix corresponding to the metric for fermions.*

```
def metricRaw : Matrix (Fin 2) (Fin 2) ℂ
```

**lemma comm_metricRaw** [source]

```
lemma comm_metricRaw (M : SL(2,ℂ)) : M.1 * metricRaw = metricRaw * (M.1⁻¹)ᵀ
```

**lemma metricRaw_comm** [source]

```
lemma metricRaw_comm (M : SL(2,ℂ)) : metricRaw * M.1 = (M.1⁻¹)ᵀ * metricRaw
```

**lemma star_comm_metricRaw** [source]

```
lemma star_comm_metricRaw (M : SL(2,ℂ)) : M.1.map star * metricRaw = metricRaw * ((M.1)⁻¹)ᴴ
```

**lemma metricRaw_comm_star** [source]

```
lemma metricRaw_comm_star (M : SL(2,ℂ)) : metricRaw * M.1.map star = ((M.1)⁻¹)ᴴ * metricRaw
```

**def leftMetricVal** [source]

*The metric $\varepsilon^{aa}$ as an element of `(leftHanded ⊗ leftHanded).V`.*

```
def leftMetricVal : (leftHanded ⊗ leftHanded).V
```

**lemma leftMetricVal_expand_tmul** [source]

*Expansion of `leftMetricVal` into the left basis.*

```
lemma leftMetricVal_expand_tmul : leftMetricVal =
    - leftBasis 0 ⊗ₜ[ℂ] leftBasis 1 + leftBasis 1 ⊗ₜ[ℂ] leftBasis 0
```

**def leftMetric** [source]

*The metric $\varepsilon^{aa}$ as a morphism `↑_ (Rep ℂ SL(2,ℂ)) ⟶ leftHanded ⊗ leftHanded`, making manifest its invariance under the action of `SL(2,ℂ)`.*

```
def leftMetric : ↑_ (Rep ℂ SL(2,ℂ)) ⟶ leftHanded ⊗ leftHanded where
```

**lemma leftMetric_apply_one** [source]

```
lemma leftMetric_apply_one : leftMetric.hom (1 : ℂ) = leftMetricVal
```

**def altLeftMetricVal** [source]

*The metric $\varepsilon_{aa}$ as an element of `(altLeftHanded ⊗ altLeftHanded).V`.*

```
def altLeftMetricVal : (altLeftHanded ⊗ altLeftHanded).V
```

**lemma altLeftMetricVal_expand_tmul** [source]

*Expansion of `altLeftMetricVal` into the left basis.*

```
lemma altLeftMetricVal_expand_tmul : altLeftMetricVal =
    altLeftBasis 0 ⊗ₜ[ℂ] altLeftBasis 1 - altLeftBasis 1 ⊗ₜ[ℂ] altLeftBasis 0
```

**def altLeftMetric** [source]

*The metric $\varepsilon_{aa}$ as a morphism $𝟙 \longrightarrow$ (Rep ℂ SL(2,ℂ)) $\longrightarrow$ altLeftHanded ⊗ altLeftHanded, making manifest its invariance under the action of SL(2,ℂ).*

```
def altLeftMetric : 𝟙_ (Rep ℂ SL(2,ℂ)) ⟶ altLeftHanded ⊗ altLeftHanded where
```

**lemma altLeftMetric_apply_one** [source]

```
lemma altLeftMetric_apply_one : altLeftMetric.hom (1 : ℂ) = altLeftMetricVal
```

**def rightMetricVal** [source]

*The metric ε^{dot a}^{dot a} as an element of (rightHanded ⊗ rightHanded).V.*

```
def rightMetricVal : (rightHanded ⊗ rightHanded).V
```

**lemma rightMetricVal_expand_tmul** [source]

*Expansion of rightMetricVal into the left basis.*

```
lemma rightMetricVal_expand_tmul : rightMetricVal =
    - rightBasis 0 ⊗ₜ[ℂ] rightBasis 1 + rightBasis 1 ⊗ₜ[ℂ] rightBasis 0
```

**def rightMetric** [source]

*The metric ε^{dot a}^{dot a} as a morphism $𝟙 \longrightarrow$ (Rep ℂ SL(2,ℂ)) $\longrightarrow$ rightHanded ⊗ rightHanded, making manifest its invariance under the action of SL(2,ℂ).*

```
def rightMetric : 𝟙_ (Rep ℂ SL(2,ℂ)) ⟶ rightHanded ⊗ rightHanded where
```

**lemma rightMetric_apply_one** [source]

```
lemma rightMetric_apply_one : rightMetric.hom (1 : ℂ) = rightMetricVal
```

**def altRightMetricVal** [source]

*The metric ε_{dot a}_{dot a} as an element of (altRightHanded ⊗ altRightHanded).V.*

```
def altRightMetricVal : (altRightHanded ⊗ altRightHanded).V
```

**lemma altRightMetricVal_expand_tmul** [source]

*Expansion of rightMetricVal into the left basis.*

```
lemma altRightMetricVal_expand_tmul : altRightMetricVal =
    altRightBasis 0 ⊗ₜ[ℂ] altRightBasis 1 - altRightBasis 1 ⊗ₜ[ℂ] altRightBasis 0
```

**def altRightMetric** [source]

*The metric ε_{dot a}_{dot a} as a morphism $𝟙 \longrightarrow$ (Rep ℂ SL(2,ℂ)) $\longrightarrow$ altRightHanded ⊗ altRightHanded, making manifest its invariance under the action of SL(2,ℂ).*

```
def altRightMetric : 𝟙_ (Rep ℂ SL(2,ℂ)) ⟶ altRightHanded ⊗ altRightHanded where
```

**lemma altRightMetric_apply_one** [source]

```
lemma altRightMetric_apply_one : altRightMetric.hom (1 : ℂ) = altRightMetricVal
```

**lemma leftAltContraction_apply_metric** [source]

```
lemma leftAltContraction_apply_metric : (β_ leftHanded altLeftHanded).hom.hom
    ((leftHanded.V ◁ (λ_ altLeftHanded.V).hom)
    ((leftHanded.V ◁ leftAltContraction.hom ▷ altLeftHanded.V)
    ((leftHanded.V ◁ (α_ leftHanded.V altLeftHanded.V altLeftHanded.V).inv)
    ((α_ leftHanded.V leftHanded.V (altLeftHanded.V ⊗ altLeftHanded.V)).hom
    (leftMetric.hom (1 : ℂ) ⊗ₜ[ℂ] altLeftMetric.hom (1 : ℂ))))) =
    altLeftLeftUnit.hom (1 : ℂ)
```

**lemma altLeftContraction_apply_metric** [source]

```
lemma altLeftContraction_apply_metric : (β_ altLeftHanded leftHanded).hom.hom
    ((altLeftHanded.V ◁ (λ_ leftHanded.V).hom)
    ((altLeftHanded.V ◁ altLeftContraction.hom ▷ leftHanded.V)
    ((altLeftHanded.V ◁ (α_ altLeftHanded.V leftHanded.V leftHanded.V).inv)
    ((α_ altLeftHanded.V altLeftHanded.V (leftHanded.V ⊗ leftHanded.V)).hom
    (altLeftMetric.hom (1 : ℂ) ⊗ₜ[ℂ] leftMetric.hom (1 : ℂ))))) =
    leftAltLeftUnit.hom (1 : ℂ)
```

**lemma rightAltContraction_apply_metric** [source]

```
lemma rightAltContraction_apply_metric : (β_ rightHanded altRightHanded).hom.hom
    ((rightHanded.V ◁ (λ_ altRightHanded.V).hom)
    ((rightHanded.V ◁ rightAltContraction.hom ▷ altRightHanded.V)
    ((rightHanded.V ◁ (α_ rightHanded.V altRightHanded.V altRightHanded.V).inv)
    ((α_ rightHanded.V rightHanded.V (altRightHanded.V ⊗ altRightHanded.V)).hom
    (rightMetric.hom (1 : ℂ) ⊗ₜ[ℂ] altRightMetric.hom (1 : ℂ))))) =
    altRightRightUnit.hom (1 : ℂ)
```

**lemma altRightContraction_apply_metric** [source]

```
lemma altRightContraction_apply_metric : (β_ altRightHanded rightHanded).hom.hom
    ((altRightHanded.V ◁ (λ_ rightHanded.V).hom)
    ((altRightHanded.V ◁ altRightContraction.hom ▷ rightHanded.V)
    ((altRightHanded.V ◁ (α_ altRightHanded.V rightHanded.V rightHanded.V).inv)
    ((α_ altRightHanded.V altRightHanded.V (rightHanded.V ⊗ rightHanded.V)).hom
    (altRightMetric.hom (1 : ℂ) ⊗ₜ[ℂ] rightMetric.hom (1 : ℂ))))) =
    rightAltRightUnit.hom (1 : ℂ)
```

## 2.20 HepLean.SpaceTime.WeylFermion.Modules

HepLean/SpaceTime/WeylFermion/Modules.lean on GitHub.

**structure LeftHandedModule** [source]

*The module in which left handed fermions live. This is equivalent to `Fin 2 → ℂ`.*

```
structure LeftHandedModule where
  val : Fin 2 → ℂ
```

**def LeftHandedModule.toFin2ℂFun** [source]

*The equivalence between `LeftHandedModule` and `Fin 2 → ℂ`.*

```
def toFin2ℂFun : LeftHandedModule ≃ (Fin 2 → ℂ) where
```

**instance AddCommMonoid LeftHandedModule** [source]

*The instance of `AddCommMonoid` on `LeftHandedModule` defined via its equivalence with `Fin 2 → ℂ`.*

```
instance : AddCommMonoid LeftHandedModule
```

**instance AddCommGroup LeftHandedModule** [source]

*The instance of `AddCommGroup` on `LeftHandedModule` defined via its equivalence with `Fin 2 → ℂ`.*

```
instance : AddCommGroup LeftHandedModule
```

**instance Module ℂ LeftHandedModule** [source]

*The instance of `Module` on `LeftHandedModule` defined via its equivalence with `Fin 2 → ℂ`.*

```
instance : Module ℂ LeftHandedModule
```

**def LeftHandedModule.toFin2ℂEquiv** [source]

*The linear equivalence between `LeftHandedModule` and `(Fin 2 → ℂ)`.*

```
@[simps!]
def toFin2ℂEquiv : LeftHandedModule ≃ₗ[ℂ] (Fin 2 → ℂ) where
```

**abbrev LeftHandedModule.toFin2ℂ** [source]

*The underlying element of `Fin 2 → ℂ` of a element in `LeftHandedModule` defined through the linear equivalence `toFin2ℂEquiv`.*

```
abbrev toFin2ℂ (ψ : LeftHandedModule)
```

**structure AltLeftHandedModule** [source]

*The module in which alt-left handed fermions live. This is equivalent to `Fin 2 → ℂ`.*

```
structure AltLeftHandedModule where
  val : Fin 2 → ℂ
```

**def AltLeftHandedModule.toFin2ℂFun** [source]

*The equivalence between `AltLeftHandedModule` and `Fin 2 → ℂ`.*

```
def toFin2ℂFun : AltLeftHandedModule ≃ (Fin 2 → ℂ) where
```

**instance AddCommMonoid AltLeftHandedModule** [source]

*The instance of `AddCommMonoid` on `AltLeftHandedModule` defined via its equivalence with `Fin 2 → ℂ`.*

```
instance : AddCommMonoid AltLeftHandedModule
```

**instance AddCommGroup AltLeftHandedModule** [source]

*The instance of `AddCommGroup` on `AltLeftHandedModule` defined via its equivalence with `Fin 2 → ℂ`.*

```
instance : AddCommGroup AltLeftHandedModule
```

**instance Module ℂ AltLeftHandedModule** [source]

*The instance of `Module` on `AltLeftHandedModule` defined via its equivalence with `Fin 2 → ℂ`.*

```
instance : Module ℂ AltLeftHandedModule
```

**def AltLeftHandedModule.toFin2ℂEquiv** [source]

*The linear equivalence between `AltLeftHandedModule` and `(Fin 2 → ℂ)`.*

```
@[simps!]
def toFin2ℂEquiv : AltLeftHandedModule ≃ₗ[ℂ] (Fin 2 → ℂ) where
```

**abbrev AltLeftHandedModule.toFin2ℂ** [source]

*The underlying element of `Fin 2 → ℂ` of a element in `AltLeftHandedModule` defined through the linear equivalence `toFin2ℂEquiv`.*

```
abbrev toFin2ℂ (ψ : AltLeftHandedModule)
```

**structure RightHandedModule** [source]

*The module in which right handed fermions live. This is equivalent to `Fin 2 → ℂ`.*

```
structure RightHandedModule where
  val : Fin 2 → ℂ
```

**def RightHandedModule.toFin2ℂFun** [source]

*The equivalence between `RightHandedModule` and `Fin 2 → ℂ`.*

```
def toFin2ℂFun : RightHandedModule ≃ (Fin 2 → ℂ) where
```

**instance AddCommMonoid RightHandedModule** [source]

*The instance of `AddCommMonoid` on `RightHandedModule` defined via its equivalence with `Fin 2 → ℂ`.*

```
instance : AddCommMonoid RightHandedModule
```

**instance AddCommGroup RightHandedModule** [source]

*The instance of `AddCommGroup` on `RightHandedModule` defined via its equivalence with `Fin 2 → ℂ`.*

```
instance : AddCommGroup RightHandedModule
```

**instance Module ℂ RightHandedModule** [source]

*The instance of `Module` on `RightHandedModule` defined via its equivalence with `Fin 2 → ℂ`.*

```
instance : Module ℂ RightHandedModule
```

**def RightHandedModule.toFin2ℂEquiv** [source]

*The linear equivalence between `RightHandedModule` and `(Fin 2 → ℂ)`.*

```
@[simps!]
def toFin2ℂEquiv : RightHandedModule ≃ₗ[ℂ] (Fin 2 → ℂ) where
```

**abbrev RightHandedModule.toFin2ℂ** [source]

*The underlying element of `Fin 2 → ℂ` of a element in `RightHandedModule` defined through the linear equivalence `toFin2ℂEquiv`.*

```
abbrev toFin2ℂ (ψ : RightHandedModule)
```

**structure AltRightHandedModule** [source]

*The module in which alt-right handed fermions live. This is equivalent to `Fin 2 → ℂ`.*

```
structure AltRightHandedModule where
  val : Fin 2 → ℂ
```

**def AltRightHandedModule.toFin2ℂFun** [source]

*The equivalence between `AltRightHandedModule` and `Fin 2 → ℂ`.*

```
def toFin2ℂFun : AltRightHandedModule ≃ (Fin 2 → ℂ) where
```

**instance AddCommMonoid AltRightHandedModule** [source]

*The instance of `AddCommMonoid` on `AltRightHandedModule` defined via its equivalence with `Fin 2 → ℂ`.*

```
instance : AddCommMonoid AltRightHandedModule
```

**instance AddCommGroup AltRightHandedModule** [source]

*The instance of `AddCommGroup` on `AltRightHandedModule` defined via its equivalence with `Fin 2 → ℂ`.*

```
instance : AddCommGroup AltRightHandedModule
```

**instance Module ℂ AltRightHandedModule** [source]

*The instance of `Module` on `AltRightHandedModule` defined via its equivalence with `Fin 2 → ℂ`.*

```
instance : Module ℂ AltRightHandedModule
```

**def AltRightHandedModule.toFin2ℂEquiv** [source]

*The linear equivalence between `AltRightHandedModule` and `(Fin 2 → ℂ)`.*

```
@[simps!]
def toFin2ℂEquiv : AltRightHandedModule ≃ₗ[ℂ] (Fin 2 → ℂ) where
```

**abbrev AltRightHandedModule.toFin2ℂ** [source]

*The underlying element of `Fin 2 → ℂ` of a element in `AltRightHandedModule` defined through the linear equivalence `toFin2ℂEquiv`.*

```
abbrev toFin2ℂ (ψ : AltRightHandedModule)
```

## 2.21 HepLean.SpaceTime.WeylFermion.Two

HepLean/SpaceTime/WeylFermion/Two.lean on GitHub.

**def leftLeftToMatrix** [source]

*Equivalence of `leftHanded ⊗ leftHanded` to `2 x 2` complex matrices.*

```
def leftLeftToMatrix : (leftHanded ⊗ leftHanded).V ≃ₗ[ℂ] Matrix (Fin 2) (Fin 2) ℂ
```

**lemma leftLeftToMatrix_symm_expand_tmul** [source]

*Expanding `leftLeftToMatrix` in terms of the standard basis.*

```
lemma leftLeftToMatrix_symm_expand_tmul (M : Matrix (Fin 2) (Fin 2) ℂ) :
    leftLeftToMatrix.symm M = ∑ i, ∑ j, M i j • (leftBasis i ⊗ₜ[ℂ] leftBasis j)
```

**def altLeftaltLeftToMatrix** [source]

*Equivalence of `altLeftHanded ⊗ altLeftHanded` to `2 x 2` complex matrices.*

```
def altLeftaltLeftToMatrix : (altLeftHanded ⊗ altLeftHanded).V ≃ₗ[ℂ] Matrix (Fin 2) (Fin 2) ℂ
```

**lemma altLeftaltLeftToMatrix_symm_expand_tmul** [source]

*Expanding `altLeftaltLeftToMatrix` in terms of the standard basis.*

```
lemma altLeftaltLeftToMatrix_symm_expand_tmul (M : Matrix (Fin 2) (Fin 2) ℂ) :
    altLeftaltLeftToMatrix.symm M = ∑ i, ∑ j, M i j • (altLeftBasis i ⊗ₜ[ℂ] altLeftBasis j)
```

**def leftAltLeftToMatrix** [source]

*Equivalence of `leftHanded ⊗ altLeftHanded` to `2 x 2` complex matrices.*

```
def leftAltLeftToMatrix : (leftHanded ⊗ altLeftHanded).V ≃ₗ[ℂ] Matrix (Fin 2) (Fin 2) ℂ
```

**lemma leftAltLeftToMatrix_symm_expand_tmul** [source]

*Expanding `leftAltLeftToMatrix` in terms of the standard basis.*

```
lemma leftAltLeftToMatrix_symm_expand_tmul (M : Matrix (Fin 2) (Fin 2) ℂ) :
    leftAltLeftToMatrix.symm M = ∑ i, ∑ j, M i j • (leftBasis i ⊗ₜ[ℂ] altLeftBasis j)
```

**def altLeftLeftToMatrix** [source]

*Equivalence of `altLeftHanded ⊗ leftHanded` to `2 x 2` complex matrices.*

```
def altLeftLeftToMatrix : (altLeftHanded ⊗ leftHanded).V ≃ₗ[ℂ] Matrix (Fin 2) (Fin 2) ℂ
```

**lemma altLeftLeftToMatrix_symm_expand_tmul** [source]

*Expanding `altLeftLeftToMatrix` in terms of the standard basis.*

```
lemma altLeftLeftToMatrix_symm_expand_tmul (M : Matrix (Fin 2) (Fin 2) ℂ) :
    altLeftLeftToMatrix.symm M = ∑ i, ∑ j, M i j • (altLeftBasis i ⊗ₜ[ℂ] leftBasis j)
```

**def rightRightToMatrix** [source]

*Equivalence of `rightHanded ⊗ rightHanded` to `2 x 2` complex matrices.*

```
def rightRightToMatrix : (rightHanded ⊗ rightHanded).V ≃ₗ[ℂ] Matrix (Fin 2) (Fin 2) ℂ
```

**lemma rightRightToMatrix_symm_expand_tmul** [source]

*Expanding `rightRightToMatrix` in terms of the standard basis.*

```
lemma rightRightToMatrix_symm_expand_tmul (M : Matrix (Fin 2) (Fin 2) ℂ) :
    rightRightToMatrix.symm M = ∑ i, ∑ j, M i j • (rightBasis i ⊗ₜ[ℂ] rightBasis j)
```

**def altRightAltRightToMatrix** [source]

*Equivalence of `altRightHanded ⊗ altRightHanded` to `2 x 2` complex matrices.*

```
def altRightAltRightToMatrix : (altRightHanded ⊗ altRightHanded).V ≃ₗ[ℂ] Matrix (Fin 2) (Fin 2)
    ℂ
```

**lemma altRightAltRightToMatrix_symm_expand_tmul** [source]

*Expanding `altRightAltRightToMatrix` in terms of the standard basis.*

```
lemma altRightAltRightToMatrix_symm_expand_tmul (M : Matrix (Fin 2) (Fin 2) ℂ) :
    altRightAltRightToMatrix.symm M =
    ∑ i, ∑ j, M i j • (altRightBasis i ⊗ₜ[ℂ] altRightBasis j)
```

**def rightAltRightToMatrix** [source]

*Equivalence of `rightHanded ⊗ altRightHanded` to `2 x 2` complex matrices.*

```
def rightAltRightToMatrix : (rightHanded ⊗ altRightHanded).V ≃ₗ[ℂ] Matrix (Fin 2) (Fin 2) ℂ
```

**lemma rightAltRightToMatrix_symm_expand_tmul** [source]

*Expanding `rightAltRightToMatrix` in terms of the standard basis.*

```
lemma rightAltRightToMatrix_symm_expand_tmul (M : Matrix (Fin 2) (Fin 2) ℂ) :
    rightAltRightToMatrix.symm M = ∑ i, ∑ j, M i j • (rightBasis i ⊗ₜ[ℂ] altRightBasis j)
```

**def altRightRightToMatrix** [source]

*Equivalence of `altRightHanded ⊗ rightHanded` to `2 x 2` complex matrices.*

```
def altRightRightToMatrix : (altRightHanded ⊗ rightHanded).V ≃ₗ[ℂ] Matrix (Fin 2) (Fin 2) ℂ
```

**lemma altRightRightToMatrix_symm_expand_tmul** [source]

*Expanding `altRightRightToMatrix` in terms of the standard basis.*

```
lemma altRightRightToMatrix_symm_expand_tmul (M : Matrix (Fin 2) (Fin 2) ℂ) :
    altRightRightToMatrix.symm M = ∑ i, ∑ j, M i j • (altRightBasis i ⊗ₜ[ℂ] rightBasis j)
```

**def altLeftAltRightToMatrix** [source]

*Equivalence of `altLeftHanded ⊗ altRightHanded` to `2 x 2` complex matrices.*

```
def altLeftAltRightToMatrix : (altLeftHanded ⊗ altRightHanded).V ≃ₗ[ℂ] Matrix (Fin 2) (Fin 2) ℂ
```

**lemma altLeftAltRightToMatrix_symm_expand_tmul** [source]

*Expanding `altLeftAltRightToMatrix` in terms of the standard basis.*

```
lemma altLeftAltRightToMatrix_symm_expand_tmul (M : Matrix (Fin 2) (Fin 2) ℂ) :
    altLeftAltRightToMatrix.symm M = ∑ i, ∑ j, M i j • (altLeftBasis i ⊗ₜ[ℂ] altRightBasis j)
```

**def leftRightToMatrix** [source]

*Equivalence of `leftHanded ⊗ rightHanded` to `2 x 2` complex matrices.*

```
def leftRightToMatrix : (leftHanded ⊗ rightHanded).V ≃ₗ[ℂ] Matrix (Fin 2) (Fin 2) ℂ
```

**lemma leftRightToMatrix_symm_expand_tmul** [source]

*Expanding `leftRightToMatrix` in terms of the standard basis.*

```
lemma leftRightToMatrix_symm_expand_tmul (M : Matrix (Fin 2) (Fin 2) ℂ) :
    leftRightToMatrix.symm M = ∑ i, ∑ j, M i j • (leftBasis i ⊗ₜ[ℂ] rightBasis j)
```

**lemma leftLeftToMatrix_ρ** [source]

*The group action of `SL(2,ℂ)` on `leftHanded ⊗ leftHanded` is equivalent to `M.1 * leftLeftToMatrix v * (M.1)ᵀ`.*

```
lemma leftLeftToMatrix_ρ (v : (leftHanded ⊗ leftHanded).V) (M : SL(2,ℂ)) :
    leftLeftToMatrix (TensorProduct.map (leftHanded.ρ M) (leftHanded.ρ M) v) =
    M.1 * leftLeftToMatrix v * (M.1)ᵀ
```

**lemma altLeftaltLeftToMatrix_ρ** [source]

*The group action of `SL(2,ℂ)` on `altLeftHanded ⊗ altLeftHanded` is equivalent to `(M.1⁻¹)ᵀ * leftLeftToMatrix v * (M.1⁻¹)`.*

```
lemma altLeftaltLeftToMatrix_ρ (v : (altLeftHanded ⊗ altLeftHanded).V) (M : SL(2,ℂ)) :
    altLeftaltLeftToMatrix (TensorProduct.map (altLeftHanded.ρ M) (altLeftHanded.ρ M) v) =
    (M.1⁻¹)ᵀ * altLeftaltLeftToMatrix v * (M.1⁻¹)
```

**lemma leftAltLeftToMatrix_ρ** [source]

*The group action of `SL(2,ℂ)` on `leftHanded ⊗ altLeftHanded` is equivalent to `M.1 * leftAltLeftToMatrix v * (M.1⁻¹)`.*

```
lemma leftAltLeftToMatrix_ρ (v : (leftHanded ⊗ altLeftHanded).V) (M : SL(2,ℂ)) :
    leftAltLeftToMatrix (TensorProduct.map (leftHanded.ρ M) (altLeftHanded.ρ M) v) =
    M.1 * leftAltLeftToMatrix v * (M.1⁻¹)
```

**lemma altLeftLeftToMatrix_ρ** [source]

*The group action of `SL(2,ℂ)` on `altLeftHanded ⊗ leftHanded` is equivalent to `(M.1⁻¹)ᵀ * leftAltLeftToMatrix v * (M.1)ᵀ`.*

```
lemma altLeftLeftToMatrix_ρ (v : (altLeftHanded ⊗ leftHanded).V) (M : SL(2,ℂ)) :
    altLeftLeftToMatrix (TensorProduct.map (altLeftHanded.ρ M) (leftHanded.ρ M) v) =
    (M.1⁻¹)ᵀ * altLeftLeftToMatrix v * (M.1)ᵀ
```

**lemma rightRightToMatrix_ρ** [source]

*The group action of `SL(2,ℂ)` on `rightHanded ⊗ rightHanded` is equivalent to `(M.1.map star) * rightRightToMatrix v * ((M.1.map star))ᵀ`.*

```
lemma rightRightToMatrix_ρ (v : (rightHanded ⊗ rightHanded).V) (M : SL(2,ℂ)) :
    rightRightToMatrix (TensorProduct.map (rightHanded.ρ M) (rightHanded.ρ M) v) =
    (M.1.map star) * rightRightToMatrix v * ((M.1.map star))ᵀ
```

**lemma altRightAltRightToMatrix_ρ** [source]

*The group action of `SL(2,ℂ)` on `altRightHanded ⊗ altRightHanded` is equivalent to `((M.1⁻¹).conjTranspose * rightRightToMatrix v * (((M.1⁻¹).conjTranspose)ᵀ`.*

```
lemma altRightAltRightToMatrix_ρ (v : (altRightHanded ⊗ altRightHanded).V) (M : SL(2,ℂ)) :
    altRightAltRightToMatrix (TensorProduct.map (altRightHanded.ρ M) (altRightHanded.ρ M) v) =
    ((M.1⁻¹).conjTranspose) * altRightAltRightToMatrix v * (((M.1⁻¹).conjTranspose)ᵀ)
```

**lemma rightAltRightToMatrix_ρ** [source]

*The group action of `SL(2,ℂ)` on `rightHanded ⊗ altRightHanded` is equivalent to `(M.1.map star) * rightAltRightToMatrix v * (((M.1⁻¹).conjTranspose)^T)`.*

```
lemma rightAltRightToMatrix_ρ (v : (rightHanded ⊗ altRightHanded).V) (M : SL(2,ℂ)) :
    rightAltRightToMatrix (TensorProduct.map (rightHanded.ρ M) (altRightHanded.ρ M) v) =
    (M.1.map star) * rightAltRightToMatrix v * (((M.1⁻¹).conjTranspose)^T)
```

**lemma altRightRightToMatrix_ρ** [source]

*The group action of `SL(2,ℂ)` on `altRightHanded ⊗ rightHanded` is equivalent to `((M.1⁻¹).conjTranspose * rightAltRightToMatrix v * ((M.1.map star)).T`.*

```
lemma altRightRightToMatrix_ρ (v : (altRightHanded ⊗ rightHanded).V) (M : SL(2,ℂ)) :
    altRightRightToMatrix (TensorProduct.map (altRightHanded.ρ M) (rightHanded.ρ M) v) =
    ((M.1⁻¹).conjTranspose) * altRightRightToMatrix v * (M.1.map star)^T
```

**lemma altLeftAltRightToMatrix_ρ** [source]

```
lemma altLeftAltRightToMatrix_ρ (v : (altLeftHanded ⊗ altRightHanded).V) (M : SL(2,ℂ)) :
    altLeftAltRightToMatrix (TensorProduct.map (altLeftHanded.ρ M) (altRightHanded.ρ M) v) =
    (M.1⁻¹)^T * altLeftAltRightToMatrix v * ((M.1⁻¹).conjTranspose)^T
```

**lemma leftRightToMatrix_ρ** [source]

```
lemma leftRightToMatrix_ρ (v : (leftHanded ⊗ rightHanded).V) (M : SL(2,ℂ)) :
    leftRightToMatrix (TensorProduct.map (leftHanded.ρ M) (rightHanded.ρ M) v) =
    M.1 * leftRightToMatrix v * (M.1)^H
```

**lemma leftLeftToMatrix_ρ_symm** [source]

```
lemma leftLeftToMatrix_ρ_symm (v : Matrix (Fin 2) (Fin 2) ℂ) (M : SL(2,ℂ)) :
    TensorProduct.map (leftHanded.ρ M) (leftHanded.ρ M) (leftLeftToMatrix.symm v) =
    leftLeftToMatrix.symm (M.1 * v * (M.1)^T)
```

**lemma altLeftaltLeftToMatrix_ρ_symm** [source]

```
lemma altLeftaltLeftToMatrix_ρ_symm (v : Matrix (Fin 2) (Fin 2) ℂ) (M : SL(2,ℂ)) :
    TensorProduct.map (altLeftHanded.ρ M) (altLeftHanded.ρ M) (altLeftaltLeftToMatrix.symm v) =
    altLeftaltLeftToMatrix.symm ((M.1⁻¹)^T * v * (M.1⁻¹))
```

**lemma leftAltLeftToMatrix_ρ_symm** [source]

```
lemma leftAltLeftToMatrix_ρ_symm (v : Matrix (Fin 2) (Fin 2) ℂ) (M : SL(2,ℂ)) :
    TensorProduct.map (leftHanded.ρ M) (altLeftHanded.ρ M) (leftAltLeftToMatrix.symm v) =
    leftAltLeftToMatrix.symm (M.1 * v * (M.1⁻¹))
```

**lemma altLeftLeftToMatrix_ρ_symm** [source]

```
lemma altLeftLeftToMatrix_ρ_symm (v : Matrix (Fin 2) (Fin 2) ℂ) (M : SL(2,ℂ)) :
    TensorProduct.map (altLeftHanded.ρ M) (leftHanded.ρ M) (altLeftLeftToMatrix.symm v) =
    altLeftLeftToMatrix.symm ((M.1⁻¹)^T * v * (M.1)^T)
```

**lemma rightRightToMatrix_ρ_symm** [source]

```
lemma rightRightToMatrix_ρ_symm (v : Matrix (Fin 2) (Fin 2) ℂ) (M : SL(2,ℂ)) :
    TensorProduct.map (rightHanded.ρ M) (rightHanded.ρ M) (rightRightToMatrix.symm v) =
    rightRightToMatrix.symm ((M.1.map star) * v * ((M.1.map star))ᵀ)
```

**lemma altRightAltRightToMatrix_ρ_symm** [source]

```
lemma altRightAltRightToMatrix_ρ_symm (v : Matrix (Fin 2) (Fin 2) ℂ) (M : SL(2,ℂ)) :
    TensorProduct.map (altRightHanded.ρ M) (altRightHanded.ρ M) (altRightAltRightToMatrix.symm
    v) =
    altRightAltRightToMatrix.symm (((M.1⁻¹).conjTranspose) * v * ((M.1⁻¹).conjTranspose)ᵀ)
```

**lemma rightAltRightToMatrix_ρ_symm** [source]

```
lemma rightAltRightToMatrix_ρ_symm (v : Matrix (Fin 2) (Fin 2) ℂ) (M : SL(2,ℂ)) :
    TensorProduct.map (rightHanded.ρ M) (altRightHanded.ρ M) (rightAltRightToMatrix.symm v) =
    rightAltRightToMatrix.symm ((M.1.map star) * v * (((M.1⁻¹).conjTranspose)ᵀ))
```

**lemma altRightRightToMatrix_ρ_symm** [source]

```
lemma altRightRightToMatrix_ρ_symm (v : Matrix (Fin 2) (Fin 2) ℂ) (M : SL(2,ℂ)) :
    TensorProduct.map (altRightHanded.ρ M) (rightHanded.ρ M) (altRightRightToMatrix.symm v) =
    altRightRightToMatrix.symm (((M.1⁻¹).conjTranspose) * v * (M.1.map star)ᵀ)
```

**lemma altLeftAltRightToMatrix_ρ_symm** [source]

```
lemma altLeftAltRightToMatrix_ρ_symm (v : Matrix (Fin 2) (Fin 2) ℂ) (M : SL(2,ℂ)) :
    TensorProduct.map (altLeftHanded.ρ M) (altRightHanded.ρ M) (altLeftAltRightToMatrix.symm v)
     =
    altLeftAltRightToMatrix.symm ((M.1⁻¹)ᵀ * v * ((M.1⁻¹).conjTranspose)ᵀ)
```

**lemma leftRightToMatrix_ρ_symm** [source]

```
lemma leftRightToMatrix_ρ_symm (v : Matrix (Fin 2) (Fin 2) ℂ) (M : SL(2,ℂ)) :
    TensorProduct.map (leftHanded.ρ M) (rightHanded.ρ M) (leftRightToMatrix.symm v) =
    leftRightToMatrix.symm (M.1 * v * (M.1)ᴴ)
```

**lemma altLeftAltRightToMatrix_ρ_symm_selfAdjoint** [source]

```
lemma altLeftAltRightToMatrix_ρ_symm_selfAdjoint (v : Matrix (Fin 2) (Fin 2) ℂ)
    (hv : IsSelfAdjoint v) (M : SL(2,ℂ)) :
    TensorProduct.map (altLeftHanded.ρ M) (altRightHanded.ρ M) (altLeftAltRightToMatrix.symm v)
     =
    altLeftAltRightToMatrix.symm
    (SL2C.repSelfAdjointMatrix (M.transpose⁻¹) ⟨v, hv⟩)
```

**lemma leftRightToMatrix_ρ_symm_selfAdjoint** [source]

```
lemma leftRightToMatrix_ρ_symm_selfAdjoint (v : Matrix (Fin 2) (Fin 2) ℂ)
    (hv : IsSelfAdjoint v) (M : SL(2,ℂ)) :
    TensorProduct.map (leftHanded.ρ M) (rightHanded.ρ M) (leftRightToMatrix.symm v) =
    leftRightToMatrix.symm
```

```
    (SL2C.repSelfAdjointMatrix M ⟨v, hv⟩)
```

## 2.22 HepLean.SpaceTime.WeylFermion.Unit

HepLean/SpaceTime/WeylFermion/Unit.lean on GitHub.

**def leftAltLeftUnitVal** [source]

*The left-alt-left unit $\delta^{a}{}_{a}$ as an element of `(leftHanded ⊗ altLeftHanded).V`.*

```
def leftAltLeftUnitVal : (leftHanded ⊗ altLeftHanded).V
```

**lemma leftAltLeftUnitVal_expand_tmul** [source]

*Expansion of `leftAltLeftUnitVal` into the basis.*

```
lemma leftAltLeftUnitVal_expand_tmul : leftAltLeftUnitVal =
    leftBasis 0 ⊗ₜ[ℂ] altLeftBasis 0 + leftBasis 1 ⊗ₜ[ℂ] altLeftBasis 1
```

**def leftAltLeftUnit** [source]

*The left-alt-left unit $\delta^{a}{}_{a}$ as a morphism `↥_ (Rep ℂ SL(2,ℂ)) ⟶ leftHanded ⊗ altLeftHanded`, manifesting the invariance under the `SL(2,ℂ)` action.*

```
def leftAltLeftUnit : ↥_ (Rep ℂ SL(2,ℂ)) ⟶ leftHanded ⊗ altLeftHanded where
```

**lemma leftAltLeftUnit_apply_one** [source]

```
lemma leftAltLeftUnit_apply_one : leftAltLeftUnit.hom (1 : ℂ) = leftAltLeftUnitVal
```

**def altLeftLeftUnitVal** [source]

*The alt-left-left unit $\delta_{a}{}^{a}$ as an element of `(altLeftHanded ⊗ leftHanded).V`.*

```
def altLeftLeftUnitVal : (altLeftHanded ⊗ leftHanded).V
```

**lemma altLeftLeftUnitVal_expand_tmul** [source]

*Expansion of `altLeftLeftUnitVal` into the basis.*

```
lemma altLeftLeftUnitVal_expand_tmul : altLeftLeftUnitVal =
    altLeftBasis 0 ⊗ₜ[ℂ] leftBasis 0 + altLeftBasis 1 ⊗ₜ[ℂ] leftBasis 1
```

**def altLeftLeftUnit** [source]

*The alt-left-left unit $\delta_{a}{}^{a}$ as a morphism `↥_ (Rep ℂ SL(2,ℂ)) ⟶ altLeftHanded ⊗ leftHanded`, manifesting the invariance under the `SL(2,ℂ)` action.*

```
def altLeftLeftUnit : ↥_ (Rep ℂ SL(2,ℂ)) ⟶ altLeftHanded ⊗ leftHanded where
```

**lemma altLeftLeftUnit_apply_one** [source]

```
lemma altLeftLeftUnit_apply_one : altLeftLeftUnit.hom (1 : ℂ) = altLeftLeftUnitVal
```

**def rightAltRightUnitVal** [source]

*The right-alt-right unit `δ^{dot a}_{dot a}` as an element of `(rightHanded ⊗ altRightHanded).V`.*

```
def rightAltRightUnitVal : (rightHanded ⊗ altRightHanded).V
```

**lemma rightAltRightUnitVal_expand_tmul** [source]

*Expansion of `rightAltRightUnitVal` into the basis.*

```
lemma rightAltRightUnitVal_expand_tmul : rightAltRightUnitVal =
    rightBasis 0 ⊗ₜ[ℂ] altRightBasis 0 + rightBasis 1 ⊗ₜ[ℂ] altRightBasis 1
```

**def rightAltRightUnit** [source]

*The right-alt-right unit `δ^{dot a}_{dot a}` as a morphism `𝓚_ (Rep ℂ SL(2,ℂ)) ⟶ rightHanded ⊗ altRightHanded`, manifesting the invariance under the `SL(2,ℂ)` action.*

```
def rightAltRightUnit : 𝓚_ (Rep ℂ SL(2,ℂ)) ⟶ rightHanded ⊗ altRightHanded where
```

**lemma rightAltRightUnit_apply_one** [source]

```
lemma rightAltRightUnit_apply_one : rightAltRightUnit.hom (1 : ℂ) = rightAltRightUnitVal
```

**def altRightRightUnitVal** [source]

*The alt-right-right unit `δ_{dot a}^{dot a}` as an element of `(rightHanded ⊗ altRightHanded).V`.*

```
def altRightRightUnitVal : (altRightHanded ⊗ rightHanded).V
```

**lemma altRightRightUnitVal_expand_tmul** [source]

*Expansion of `altRightRightUnitVal` into the basis.*

```
lemma altRightRightUnitVal_expand_tmul : altRightRightUnitVal =
    altRightBasis 0 ⊗ₜ[ℂ] rightBasis 0 + altRightBasis 1 ⊗ₜ[ℂ] rightBasis 1
```

**def altRightRightUnit** [source]

*The alt-right-right unit `δ_{dot a}^{dot a}` as a morphism `𝓚_ (Rep ℂ SL(2,ℂ)) ⟶ altRightHanded ⊗ rightHanded`, manifesting the invariance under the `SL(2,ℂ)` action.*

```
def altRightRightUnit : 𝓚_ (Rep ℂ SL(2,ℂ)) ⟶ altRightHanded ⊗ rightHanded where
```

**lemma altRightRightUnit_apply_one** [source]

```
lemma altRightRightUnit_apply_one : altRightRightUnit.hom (1 : ℂ) = altRightRightUnitVal
```

**lemma contr_altLeftLeftUnit** [source]

*Contraction on the right with `altLeftLeftUnit` does nothing.*

```
lemma contr_altLeftLeftUnit (x : leftHanded) :
    (λ_ leftHanded).hom.hom
    (((leftAltContraction) ▷ leftHanded).hom
    ((α_ _ _ leftHanded).inv.hom
    (x ⊗ₜ[ℂ] altLeftLeftUnit.hom (1 : ℂ)))) = x
```

**lemma contr_leftAltLeftUnit** [source]

*Contraction on the right with `leftAltLeftUnit` does nothing.*

```
lemma contr_leftAltLeftUnit (x : altLeftHanded) :
    (λ_ altLeftHanded).hom.hom
    (((altLeftContraction) ▷ altLeftHanded).hom
    ((α_ _ _ altLeftHanded).inv.hom
    (x ⊗ₜ[ℂ] leftAltLeftUnit.hom (1 : ℂ)))) = x
```

**lemma contr_altRightRightUnit** [source]

*Contraction on the right with `altRightRightUnit` does nothing.*

```
lemma contr_altRightRightUnit (x : rightHanded) :
    (λ_ rightHanded).hom.hom
    (((rightAltContraction) ▷ rightHanded).hom
    ((α_ _ _ rightHanded).inv.hom
    (x ⊗ₜ[ℂ] altRightRightUnit.hom (1 : ℂ)))) = x
```

**lemma contr_rightAltRightUnit** [source]

*Contraction on the right with `rightAltRightUnit` does nothing.*

```
lemma contr_rightAltRightUnit (x : altRightHanded) :
    (λ_ altRightHanded).hom.hom
    (((altRightContraction) ▷ altRightHanded).hom
    ((α_ _ _ altRightHanded).inv.hom
    (x ⊗ₜ[ℂ] rightAltRightUnit.hom (1 : ℂ)))) = x
```

**lemma altLeftLeftUnit_symm** [source]

```
lemma altLeftLeftUnit_symm :
    (altLeftLeftUnit.hom (1 : ℂ)) = (altLeftHanded ◁ 𝟙 _).hom
    ((β_ leftHanded altLeftHanded).hom.hom (leftAltLeftUnit.hom (1 : ℂ)))
```

**lemma leftAltLeftUnit_symm** [source]

```
lemma leftAltLeftUnit_symm :
    (leftAltLeftUnit.hom (1 : ℂ)) = (leftHanded ◁ 𝟙 _).hom ((β_ altLeftHanded leftHanded).hom.
    hom
    (altLeftLeftUnit.hom (1 : ℂ)))
```

**lemma altRightRightUnit_symm** [source]

```
lemma altRightRightUnit_symm :
    (altRightRightUnit.hom (1 : ℂ)) = (altRightHanded ◁ 𝟙 _).hom
    ((β_ rightHanded altRightHanded).hom.hom (rightAltRightUnit.hom (1 : ℂ)))
```

**lemma rightAltRightUnit_symm** [source]

```
lemma rightAltRightUnit_symm :
    (rightAltRightUnit.hom (1 : ℂ)) = (rightHanded ◁ 𝟙 _).hom
    ((β_ altRightHanded rightHanded).hom.hom (altRightRightUnit.hom (1 : ℂ)))
```

## 2.23 HepLean.StandardModel.HiggsBoson.PointwiseInnerProd

HepLean/StandardModel/HiggsBoson/PointwiseInnerProd.lean on GitHub.

**lemma innerProd_expand'** [source]

*Expands the inner product on Higgs fields in terms of complex components of the Higgs fields.*

```
lemma innerProd_expand' (φ1 φ2 : HiggsField) (x : SpaceTime) :
    ⟪φ1, φ2⟫_H x = conj (φ1 x 0) * φ2 x 0 + conj (φ1 x 1) * φ2 x 1
```

## 2.24 HepLean.Tensors.ComplexLorentz.Basic

HepLean/Tensors/ComplexLorentz/Basic.lean on GitHub.

**inductive Color** [source]

*The colors associated with complex representations of SL(2, ℂ) of intrest to physics.*

```
inductive Color
  | upL : Color
  | downL : Color
  | upR : Color
  | downR : Color
  | up : Color
  | down : Color
```

**instance DecidableEq Color** [source]

```
instance : DecidableEq Color
```

**def complexLorentzTensor** [source]

*The tensor structure for complex Lorentz tensors.*

```
def complexLorentzTensor : TensorSpecies where
```

**instance DecidableEq complexLorentzTensor.C** [source]

```
instance : DecidableEq complexLorentzTensor.C
```

**lemma basis_contr** [source]

```
lemma basis_contr (c : complexLorentzTensor.C) (i : Fin (complexLorentzTensor.repDim c))
    (j : Fin (complexLorentzTensor.repDim (complexLorentzTensor.τ c))) :
    complexLorentzTensor.castToField
    ((complexLorentzTensor.contr.app {as := c}).hom
```

```
    (complexLorentzTensor.basis c i ⊗ₜ complexLorentzTensor.basis (complexLorentzTensor.τ c) j)
    ) =
    if i.val = j.val then 1 else 0
```

**instance DecidableEq (OverColor.mk c).left** [source]

```
instance {n : ℕ} {c : Fin n → complexLorentzTensor.C} :
    DecidableEq (OverColor.mk c).left
```

**instance Fintype (OverColor.mk c).left** [source]

```
instance {n : ℕ} {c : Fin n → complexLorentzTensor.C} :
    Fintype (OverColor.mk c).left
```

**instance Decidable (σ = σ')** [source]

```
instance {n m : ℕ} {c : Fin n → complexLorentzTensor.C}
    {c1 : Fin m → complexLorentzTensor.C} (σ σ' : OverColor.mk c ⟶ OverColor.mk c1) :
    Decidable (σ = σ')
```

## 2.25 HepLean.Tensors.ComplexLorentz.Basis

HepLean/Tensors/ComplexLorentz/Basis.lean on GitHub.

**def basisVector** [source]

*Basis vectors for complex Lorentz tensors.*

```
def basisVector {n : ℕ} (c : Fin n → complexLorentzTensor.C)
    (b : Π j, Fin (complexLorentzTensor.repDim (c j))) :
    complexLorentzTensor.F.obj (OverColor.mk c)
```

**lemma perm_basisVector_cast** [source]

```
lemma perm_basisVector_cast {n m : ℕ} {c : Fin n → complexLorentzTensor.C}
    {c1 : Fin m → complexLorentzTensor.C}
    (σ : OverColor.mk c ⟶ OverColor.mk c1) (i : Fin m) :
    complexLorentzTensor.repDim (c ((OverColor.Hom.toEquiv σ).symm i)) =
    complexLorentzTensor.repDim (c1 i)
```

**lemma basis_eq_FDiscrete** [source]

```
lemma basis_eq_FDiscrete {n : ℕ} (c : Fin n → complexLorentzTensor.C)
    (b : Π j, Fin (complexLorentzTensor.repDim (c j))) (i : Fin n)
    (h : { as := c i } = { as := c1 }) :
    (complexLorentzTensor.FDiscrete.map (eqToHom h)).hom
    (complexLorentzTensor.basis (c i) (b i)) =
    (complexLorentzTensor.basis c1 (Fin.cast (by simp_all) (b i)))
```

**lemma perm_basisVector** [source]

```
lemma perm_basisVector {n m : ℕ} {c : Fin n → complexLorentzTensor.C}
    {c1 : Fin m → complexLorentzTensor.C} (σ : OverColor.mk c ⟶ OverColor.mk c1)
```

```
    (b : Π j, Fin (complexLorentzTensor.repDim (c j))) :
    (perm σ (tensorNode (basisVector c b))).tensor =
    (basisVector c1 (fun i => Fin.cast (perm_basisVector_cast σ i)
    (b ((OverColor.Hom.toEquiv σ).symm i))))
```

**lemma perm_basisVector_tree** [source]

```
lemma perm_basisVector_tree {n m : ℕ} {c : Fin n → complexLorentzTensor.C}
    {c1 : Fin m → complexLorentzTensor.C} (σ : OverColor.mk c ⟶ OverColor.mk c1)
    (b : Π j, Fin (complexLorentzTensor.repDim (c j))) :
    (perm σ (tensorNode (basisVector c b))).tensor =
    (tensorNode (basisVector c1 (fun i => Fin.cast (perm_basisVector_cast σ i)
    (b ((OverColor.Hom.toEquiv σ).symm i))))).tensor
```

**def contrBasisVectorMul** [source]

*The scalar determining if contracting two basis vectors together gives zero or not.*

```
def contrBasisVectorMul {n : ℕ} {c : Fin n.succ.succ → complexLorentzTensor.C}
    (i : Fin n.succ.succ) (j : Fin n.succ)
    (b : Π k, Fin (complexLorentzTensor.repDim (c k))) : ℂ
```

**lemma contrBasisVectorMul_neg** [source]

```
lemma contrBasisVectorMul_neg {n : ℕ} {c : Fin n.succ.succ → complexLorentzTensor.C}
    {i : Fin n.succ.succ} {j : Fin n.succ} {b : Π k, Fin (complexLorentzTensor.repDim (c k))}
    (h : ¬ (b i).val = (b (i.succAbove j)).val) :
    contrBasisVectorMul i j b = 0
```

**lemma contrBasisVectorMul_pos** [source]

```
lemma contrBasisVectorMul_pos {n : ℕ} {c : Fin n.succ.succ → complexLorentzTensor.C}
    {i : Fin n.succ.succ} {j : Fin n.succ} {b : Π k, Fin (complexLorentzTensor.repDim (c k))}
    (h : (b i).val = (b (i.succAbove j)).val) :
    contrBasisVectorMul i j b = 1
```

**lemma contr_basisVector** [source]

```
lemma contr_basisVector {n : ℕ} {c : Fin n.succ.succ → complexLorentzTensor.C}
    {i : Fin n.succ.succ} {j : Fin n.succ} {h : c (i.succAbove j) = complexLorentzTensor.τ (c i
    )}
    (b : Π k, Fin (complexLorentzTensor.repDim (c k))) :
    (contr i j h (tensorNode (basisVector c b))).tensor = (contrBasisVectorMul i j b) •
    basisVector (c ∘ Fin.succAbove i ∘ Fin.succAbove j)
    (fun k => b (i.succAbove (j.succAbove k)))
```

**lemma contr_basisVector_tree** [source]

```
lemma contr_basisVector_tree {n : ℕ} {c : Fin n.succ.succ → complexLorentzTensor.C}
    {i : Fin n.succ.succ} {j : Fin n.succ} {h : c (i.succAbove j) = complexLorentzTensor.τ (c i
    )}
    (b : Π k, Fin (complexLorentzTensor.repDim (c k))) :
    (contr i j h (tensorNode (basisVector c b))).tensor =
    (smul (contrBasisVectorMul i j b) (tensorNode
```

```
    (basisVector (c ∘ Fin.succAbove i ∘ Fin.succAbove j)
    (fun k => b (i.succAbove (j.succAbove k))))))).tensor
```

### lemma contr_basisVector_tree_pos [source]

```
lemma contr_basisVector_tree_pos {n : ℕ} {c : Fin n.succ.succ → complexLorentzTensor.C}
    {i : Fin n.succ.succ} {j : Fin n.succ} {h : c (i.succAbove j) = complexLorentzTensor.τ (c i
    )}
    (b : Π k, Fin (complexLorentzTensor.repDim (c k)))
    (hn : (b i).val = (b (i.succAbove j)).val := by decide) :
    (contr i j h (tensorNode (basisVector c b))).tensor =
    ((tensorNode (basisVector (c ∘ Fin.succAbove i ∘ Fin.succAbove j)
    (fun k => b (i.succAbove (j.succAbove k))))))).tensor
```

### lemma contr_basisVector_tree_neg [source]

```
lemma contr_basisVector_tree_neg {n : ℕ} {c : Fin n.succ.succ → complexLorentzTensor.C}
    {i : Fin n.succ.succ} {j : Fin n.succ} {h : c (i.succAbove j) = complexLorentzTensor.τ (c i
    )}
    (b : Π k, Fin (complexLorentzTensor.repDim (c k)))
    (hn : ¬ (b i).val = (b (i.succAbove j)).val := by decide) :
    (contr i j h (tensorNode (basisVector c b))).tensor =
    (tensorNode 0).tensor
```

### def prodBasisVecEquiv [source]

*Equivalence of Fin types appearing in the product of two basis vectors.*

```
def prodBasisVecEquiv {n m : ℕ} {c : Fin n → complexLorentzTensor.C}
    {c1 : Fin m → complexLorentzTensor.C} (i : Fin n ⊕ Fin m) :
    Sum.elim (fun i => Fin (complexLorentzTensor.repDim (c i))) (fun i =>
      Fin (complexLorentzTensor.repDim (c1 i)))
    i ≃ Fin (complexLorentzTensor.repDim ((Sum.elim c c1 i)))
```

### lemma prod_basisVector [source]

```
lemma prod_basisVector {n m : ℕ} {c : Fin n → complexLorentzTensor.C}
    {c1 : Fin m → complexLorentzTensor.C}
    (b : Π k, Fin (complexLorentzTensor.repDim (c k)))
    (b1 : Π k, Fin (complexLorentzTensor.repDim (c1 k))) :
    (prod (tensorNode (basisVector c b)) (tensorNode (basisVector c1 b1))).tensor =
    basisVector (Sum.elim c c1 ∘ finSumFinEquiv.symm) (fun i =>
    prodBasisVecEquiv (finSumFinEquiv.symm i)
    ((HepLean.PiTensorProduct.elimPureTensor b b1) (finSumFinEquiv.symm i)))
```

### lemma prod_basisVector_tree [source]

```
lemma prod_basisVector_tree {n m : ℕ} {c : Fin n → complexLorentzTensor.C}
    {c1 : Fin m → complexLorentzTensor.C}
    (b : Π k, Fin (complexLorentzTensor.repDim (c k)))
    (b1 : Π k, Fin (complexLorentzTensor.repDim (c1 k))) :
    (prod (tensorNode (basisVector c b)) (tensorNode (basisVector c1 b1))).tensor =
    (tensorNode (basisVector (Sum.elim c c1 ∘ finSumFinEquiv.symm) (fun i =>
    prodBasisVecEquiv (finSumFinEquiv.symm i)
    ((HepLean.PiTensorProduct.elimPureTensor b b1) (finSumFinEquiv.symm i))))).tensor
```

**lemma eval_basisVector** [source]

```
lemma eval_basisVector {n : ℕ} {c : Fin n.succ → complexLorentzTensor.C}
    {i : Fin n.succ} (j : Fin (complexLorentzTensor.repDim (c i)))
    (b : Π k, Fin (complexLorentzTensor.repDim (c k))) :
    (eval i j (tensorNode (basisVector c b))).tensor = (if j = b i then (1 : ℂ) else 0) •
    basisVector (c ∘ Fin.succAbove i) (fun k => b (i.succAbove k))
```

## 2.26 HepLean.Tensors.ComplexLorentz.Bispinors.Basic

HepLean/Tensors/ComplexLorentz/Bispinors/Basic.lean on GitHub.

**def contrBispinorUp** [source]

*A bispinor $p^{aa}$ created from a lorentz vector p^μ.*

```
def contrBispinorUp (p : complexContr)
```

**def contrBispinorDown** [source]

*A bispinor $p_{aa}$ created from a lorentz vector p^μ.*

```
def contrBispinorDown (p : complexContr)
```

**def coBispinorUp** [source]

*A bispinor $p^{aa}$ created from a lorentz vector p_μ.*

```
def coBispinorUp (p : complexCo)
```

**def coBispinorDown** [source]

*A bispinor $p_{aa}$ created from a lorentz vector p_μ.*

```
def coBispinorDown (p : complexCo)
```

**lemma tensorNode_contrBispinorUp** [source]

*The definitional tensor node relation for `contrBispinorUp`.*

```
lemma tensorNode_contrBispinorUp (p : complexContr) :
    {contrBispinorUp p | α β}ᵀ.tensor = {pauliCo | μ α β ⊗ p | μ}ᵀ.tensor
```

**lemma tensorNode_contrBispinorDown** [source]

*The definitional tensor node relation for `contrBispinorDown`.*

```
lemma tensorNode_contrBispinorDown (p : complexContr) :
    {contrBispinorDown p | α β}ᵀ.tensor =
    {εL' | α α' ⊗ εR' | β β' ⊗ contrBispinorUp p | α β}ᵀ.tensor
```

**lemma tensorNode_coBispinorUp** [source]

*The definitional tensor node relation for `coBispinorUp`.*

```
lemma tensorNode_coBispinorUp (p : complexCo) :
    {coBispinorUp p | α β}ᵀ.tensor = {pauliContr | μ α β ⊗ p | μ}ᵀ.tensor
```

**lemma tensorNode_coBispinorDown** [source]

*The definitional tensor node relation for `coBispinorDown`.*

```
lemma tensorNode_coBispinorDown (p : complexCo) :
    {coBispinorDown p | α β}ᵀ.tensor =
    {εL' | α α' ⊗ εR' | β β' ⊗ coBispinorUp p | α β}ᵀ.tensor
```

**informal_lemma contrBispinorUp_eq_metric_contr_contrBispinorDown** [source]

```
informal_lemma contrBispinorUp_eq_metric_contr_contrBispinorDown where
  math :≈ "{contrBispinorUp p | α β = εL | α α' ⊗ εR | β β'⊗ contrBispinorDown p | α' β' }ᵀ"
  proof :≈ "Expand `contrBispinorDown` and use fact that metrics contract to the identity."
  deps :≈ [``contrBispinorUp, ``contrBispinorDown, ``leftMetric, ``rightMetric]
```

**informal_lemma coBispinorUp_eq_metric_contr_coBispinorDown** [source]

```
informal_lemma coBispinorUp_eq_metric_contr_coBispinorDown where
  math :≈ "{coBispinorUp p | α β = εL | α α' ⊗ εR | β β'⊗ coBispinorDown p | α' β' }ᵀ"
  proof :≈ "Expand `coBispinorDown` and use fact that metrics contract to the identity."
  deps :≈ [``coBispinorUp, ``coBispinorDown, ``leftMetric, ``rightMetric]
```

**lemma contrBispinorDown_expand** [source]

```
lemma contrBispinorDown_expand (p : complexContr) :
    {contrBispinorDown p | α β}ᵀ.tensor =
    {εL' | α α' ⊗ εR' | β β' ⊗
    (pauliCo | μ α β ⊗ p | μ)}ᵀ.tensor
```

**lemma coBispinorDown_expand** [source]

```
lemma coBispinorDown_expand (p : complexCo) :
    {coBispinorDown p | α β}ᵀ.tensor =
    {εL' | α α' ⊗ εR' | β β' ⊗
    (pauliContr | μ α β ⊗ p | μ)}ᵀ.tensor
```

**lemma contrBispinorDown_eq_pauliCoDown_contr** [source]

```
lemma contrBispinorDown_eq_pauliCoDown_contr (p : complexContr) :
    {contrBispinorDown p | α β = pauliCoDown | μ α β ⊗ p | μ}ᵀ
```

**lemma coBispinorDown_eq_pauliContrDown_contr** [source]

```
lemma coBispinorDown_eq_pauliContrDown_contr (p : complexCo) :
    {coBispinorDown p | α β = pauliContrDown | μ α β ⊗ p | μ}ᵀ
```

## 2.27 HepLean.Tensors.ComplexLorentz.Lemmas

HepLean/Tensors/ComplexLorentz/Lemmas.lean on GitHub.

**lemma antiSymm_contr_symm** [source]

```
lemma antiSymm_contr_symm {A : complexLorentzTensor.F.obj (OverColor.mk ![Color.up, Color.up])}
    {S : complexLorentzTensor.F.obj (OverColor.mk ![Color.down, Color.down])}
    (hA : {A | μ ν = - (A | ν μ)}ᵀ) (hs : {S | μ ν = S | ν μ}ᵀ) :
    {A | μ ν ⊗ S | μ ν = - A | μ ν ⊗ S | μ ν}ᵀ
```

## 2.28 HepLean.Tensors.ComplexLorentz.Metrics.Basic

HepLean/Tensors/ComplexLorentz/Metrics/Basic.lean on GitHub.

**def coMetric** [source]

*The metric $\eta_{ii}$ as a complex Lorentz tensor.*

```
def coMetric
```

**def contrMetric** [source]

*The metric $\eta^{ii}$ as a complex Lorentz tensor.*

```
def contrMetric
```

**def leftMetric** [source]

*The metric $\varepsilon^{aa}$ as a complex Lorentz tensor.*

```
def leftMetric
```

**def rightMetric** [source]

*The metric ε^{dot a}^{dot a} as a complex Lorentz tensor.*

```
def rightMetric
```

**def altLeftMetric** [source]

*The metric $\varepsilon_{aa}$ as a complex Lorentz tensor.*

```
def altLeftMetric
```

**def altRightMetric** [source]

*The metric ε_{dot a}_{dot a} as a complex Lorentz tensor.*

```
def altRightMetric
```

**lemma tensorNode_coMetric** [source]

*The definitional tensor node relation for `coMetric`.*

```
lemma tensorNode_coMetric : {η' | μ ν}ᵀ.tensor = {Lorentz.coMetric | μ ν}ᵀ.tensor
```

**lemma tensorNode_contrMetric** [source]

*The definitional tensor node relation for `contrMetric`.*

```
lemma tensorNode_contrMetric : {η | μ ν}ᵀ.tensor = {Lorentz.contrMetric | μ ν}ᵀ.tensor
```

**lemma tensorNode_leftMetric** [source]

*The definitional tensor node relation for `leftMetric`.*

```
lemma tensorNode_leftMetric : {εL | α α'}ᵀ.tensor = {Fermion.leftMetric | α α'}ᵀ.tensor
```

**lemma tensorNode_rightMetric** [source]

*The definitional tensor node relation for `rightMetric`.*

```
lemma tensorNode_rightMetric : {εR | β β'}ᵀ.tensor = {Fermion.rightMetric | β β'}ᵀ.tensor
```

**lemma tensorNode_altLeftMetric** [source]

*The definitional tensor node relation for `altLeftMetric`.*

```
lemma tensorNode_altLeftMetric :
    {εL' | α α'}ᵀ.tensor = {Fermion.altLeftMetric | α α'}ᵀ.tensor
```

**lemma tensorNode_altRightMetric** [source]

*The definitional tensor node relation for `altRightMetric`.*

```
lemma tensorNode_altRightMetric :
    {εR' | β β'}ᵀ.tensor = {Fermion.altRightMetric | β β'}ᵀ.tensor
```

**lemma action_coMetric** [source]

*The tensor `coMetric` is invariant under the action of `SL(2,ℂ)`.*

```
lemma action_coMetric (g : SL(2,ℂ)) : {g •ₐ η' | μ ν}ᵀ.tensor =
    {η' | μ ν}ᵀ.tensor
```

**lemma action_contrMetric** [source]

*The tensor `contrMetric` is invariant under the action of `SL(2,ℂ)`.*

```
lemma action_contrMetric (g : SL(2,ℂ)) : {g •ₐ η | μ ν}ᵀ.tensor =
    {η | μ ν}ᵀ.tensor
```

**lemma action_leftMetric** [source]

*The tensor `leftMetric` is invariant under the action of `SL(2,ℂ)`.*

```
lemma action_leftMetric (g : SL(2,ℂ)) : {g •ₐ εL | α α'}^T.tensor =
    {εL | α α'}^T.tensor
```

**lemma action_rightMetric** [source]

*The tensor `rightMetric` is invariant under the action of `SL(2,ℂ)`.*

```
lemma action_rightMetric (g : SL(2,ℂ)) : {g •ₐ εR | β β'}^T.tensor =
    {εR | β β'}^T.tensor
```

**lemma action_altLeftMetric** [source]

*The tensor `altLeftMetric` is invariant under the action of `SL(2,ℂ)`.*

```
lemma action_altLeftMetric (g : SL(2,ℂ)) : {g •ₐ εL' | α α'}^T.tensor =
    {εL' | α α'}^T.tensor
```

**lemma action_altRightMetric** [source]

*The tensor `altRightMetric` is invariant under the action of `SL(2,ℂ)`.*

```
lemma action_altRightMetric (g : SL(2,ℂ)) : {g •ₐ εR' | β β'}^T.tensor =
    {εR' | β β'}^T.tensor
```

## 2.29 HepLean.Tensors.ComplexLorentz.Metrics.Basis

HepLean/Tensors/ComplexLorentz/Metrics/Basis.lean on GitHub.

**lemma coMetric_basis_expand** [source]

*The expansion of the Lorentz covariant metric in terms of basis vectors.*

```
lemma coMetric_basis_expand : {η' | μ ν}^T.tensor =
    basisVector ![Color.down, Color.down] (fun _ => 0)
    - basisVector ![Color.down, Color.down] (fun _ => 1)
    - basisVector ![Color.down, Color.down] (fun _ => 2)
    - basisVector ![Color.down, Color.down] (fun _ => 3)
```

**lemma coMetric_basis_expand_tree** [source]

*Provides the explicit expansion of the co-metric tensor in terms of the basis elements, as a tensor tree.*

```
lemma coMetric_basis_expand_tree : {η' | μ ν}^T.tensor =
    (TensorTree.add (tensorNode (basisVector ![Color.down, Color.down] (fun _ => 0))) <|
    TensorTree.add (smul (-1) (tensorNode (basisVector ![Color.down, Color.down] (fun _ => 1)))
    ) <|
    TensorTree.add (smul (-1) (tensorNode (basisVector ![Color.down, Color.down] (fun _ => 2)))
    ) <|
    (smul (-1) (tensorNode (basisVector ![Color.down, Color.down] (fun _ => 3))))).tensor
```

**lemma contrMatrix_basis_expand** [source]

*The expansion of the Lorentz contrvariant metric in terms of basis vectors.*

```
lemma contrMatrix_basis_expand : {η | μ ν}^T.tensor =
    basisVector ![Color.up, Color.up] (fun _ => 0)
    - basisVector ![Color.up, Color.up] (fun _ => 1)
    - basisVector ![Color.up, Color.up] (fun _ => 2)
    - basisVector ![Color.up, Color.up] (fun _ => 3)
```

**lemma contrMatrix_basis_expand_tree** [source]

```
lemma contrMatrix_basis_expand_tree : {η | μ ν}^T.tensor =
    (TensorTree.add (tensorNode (basisVector ![Color.up, Color.up] (fun _ => 0))) <|
    TensorTree.add (smul (-1) (tensorNode (basisVector ![Color.up, Color.up] (fun _ => 1)))) <|
    TensorTree.add (smul (-1) (tensorNode (basisVector ![Color.up, Color.up] (fun _ => 2)))) <|
    (smul (-1) (tensorNode (basisVector ![Color.up, Color.up] (fun _ => 3))))).tensor
```

**lemma leftMetric_expand** [source]

```
lemma leftMetric_expand : {εL | α β}^T.tensor =
    - basisVector ![Color.upL, Color.upL] (fun | 0 => 0 | 1 => 1)
    + basisVector ![Color.upL, Color.upL] (fun | 0 => 1 | 1 => 0)
```

**lemma leftMetric_expand_tree** [source]

```
lemma leftMetric_expand_tree : {εL | α β}^T.tensor =
    (TensorTree.add (smul (-1) (tensorNode (basisVector ![Color.upL, Color.upL]
    (fun | 0 => 0 | 1 => 1)))) <|
    (tensorNode (basisVector ![Color.upL, Color.upL] (fun | 0 => 1 | 1 => 0)))).tensor
```

**lemma altLeftMetric_expand** [source]

```
lemma altLeftMetric_expand : {εL' | α β}^T.tensor =
    basisVector ![Color.downL, Color.downL] (fun | 0 => 0 | 1 => 1)
    - basisVector ![Color.downL, Color.downL] (fun | 0 => 1 | 1 => 0)
```

**lemma altLeftMetric_expand_tree** [source]

```
lemma altLeftMetric_expand_tree : {εL' | α β}^T.tensor =
    (TensorTree.add (tensorNode (basisVector ![Color.downL, Color.downL]
    (fun | 0 => 0 | 1 => 1))) <|
    (smul (-1) (tensorNode (basisVector ![Color.downL, Color.downL]
    (fun | 0 => 1 | 1 => 0))))).tensor
```

**lemma rightMetric_expand** [source]

```
lemma rightMetric_expand : {εR | α β}^T.tensor =
    - basisVector ![Color.upR, Color.upR] (fun | 0 => 0 | 1 => 1)
    + basisVector ![Color.upR, Color.upR] (fun | 0 => 1 | 1 => 0)
```

**lemma rightMetric_expand_tree** [source]

```
lemma rightMetric_expand_tree : {εR | α β}^T.tensor =
    (TensorTree.add (smul (-1) (tensorNode (basisVector ![Color.upR, Color.upR]
    (fun | 0 => 0 | 1 => 1)))) <|
    (tensorNode (basisVector ![Color.upR, Color.upR] (fun | 0 => 1 | 1 => 0)))).tensor
```

**lemma altRightMetric_expand** [source]

```
lemma altRightMetric_expand : {εR' | α β}^T.tensor =
    basisVector ![Color.downR, Color.downR] (fun | 0 => 0 | 1 => 1)
    - basisVector ![Color.downR, Color.downR] (fun | 0 => 1 | 1 => 0)
```

**lemma altRightMetric_expand_tree** [source]

```
lemma altRightMetric_expand_tree : {εR' | α β}^T.tensor =
    (TensorTree.add (tensorNode (basisVector
    ![Color.downR, Color.downR] (fun | 0 => 0 | 1 => 1))) <|
    (smul (-1) (tensorNode (basisVector ![Color.downR, Color.downR]
    (fun | 0 => 1 | 1 => 0))))).tensor
```

## 2.30 HepLean.Tensors.ComplexLorentz.Metrics.Lemmas

HepLean/Tensors/ComplexLorentz/Metrics/Lemmas.lean on GitHub.

**informal_lemma coMetric_symm** [source]

```
informal_lemma coMetric_symm where
  math :≈ "The covariant metric is symmetric {η' | μ ν = η' | ν μ}^T"
  deps :≈ [``coMetric]
```

**informal_lemma contrMetric_symm** [source]

```
informal_lemma contrMetric_symm where
  math :≈ "The contravariant metric is symmetric {η | μ ν = η | ν μ}^T"
  deps :≈ [``contrMetric]
```

**informal_lemma leftMetric_antisymm** [source]

```
informal_lemma leftMetric_antisymm where
  math :≈ "The left metric is antisymmetric {εL | α α' = - εL | α' α}^T"
  deps :≈ [``leftMetric]
```

**informal_lemma rightMetric_antisymm** [source]

```
informal_lemma rightMetric_antisymm where
  math :≈ "The right metric is antisymmetric {εR | β β' = - εR | β' β}^T"
  deps :≈ [``rightMetric]
```

**informal_lemma altLeftMetric_antisymm** [source]

```
informal_lemma altLeftMetric_antisymm where
  math :≈ "The alt-left metric is antisymmetric {εL' | α α' = - εL' | α' α}^T"
```

```
  deps :≈ [``altLeftMetric]
```

**informal_lemma altRightMetric_antisymm** [source]

```
informal_lemma altRightMetric_antisymm where
  math :≈ "The alt-right metric is antisymmetric {εR' | β β' = - εR' | β' β}^T"
  deps :≈ [``altRightMetric]
```

**informal_lemma coMetric_contr_contrMetric** [source]

```
informal_lemma coMetric_contr_contrMetric where
  math :≈ "The contraction of the covariant metric with the contravariant metric is the unit
        {η' | μ ρ ⊗ η | ρ ν = δ' | μ ν}^T"
  deps :≈ [``coMetric, ``contrMetric, ``coContrUnit]
```

**informal_lemma contrMetric_contr_coMetric** [source]

```
informal_lemma contrMetric_contr_coMetric where
  math :≈ "The contraction of the contravariant metric with the covariant metric is the unit
        {η | μ ρ ⊗ η' | ρ ν = δ | μ ν}^T"
  deps :≈ [``contrMetric, ``coMetric, ``contrCoUnit]
```

**informal_lemma leftMetric_contr_altLeftMetric** [source]

```
informal_lemma leftMetric_contr_altLeftMetric where
  math :≈ "The contraction of the left metric with the alt-left metric is the unit
        {εL | α β ⊗ εL' | β γ = δL | α γ}^T"
  deps :≈ [``leftMetric, ``altLeftMetric, ``leftAltLeftUnit]
```

**informal_lemma rightMetric_contr_altRightMetric** [source]

```
informal_lemma rightMetric_contr_altRightMetric where
  math :≈ "The contraction of the right metric with the alt-right metric is the unit
        {εR | α β ⊗ εR' | β γ = δR | α γ}^T"
  deps :≈ [``rightMetric, ``altRightMetric, ``rightAltRightUnit]
```

**informal_lemma altLeftMetric_contr_leftMetric** [source]

```
informal_lemma altLeftMetric_contr_leftMetric where
  math :≈ "The contraction of the alt-left metric with the left metric is the unit
        {εL' | α β ⊗ εL | β γ = δL' | α γ}^T"
  deps :≈ [``altLeftMetric, ``leftMetric, ``altLeftLeftUnit]
```

**informal_lemma altRightMetric_contr_rightMetric** [source]

```
informal_lemma altRightMetric_contr_rightMetric where
  math :≈ "The contraction of the alt-right metric with the right metric is the unit
        {εR' | α β ⊗ εR | β γ = δR' | α γ}^T"
  deps :≈ [``altRightMetric, ``rightMetric, ``altRightRightUnit]
```

**def leftMetricMulRightMap** [source]

*The map to color one gets when multiplying left and right metrics.*

```
def leftMetricMulRightMap
```

**lemma leftMetric_prod_rightMetric** [source]

```
lemma leftMetric_prod_rightMetric : {εL | α α' ⊗ εR | β β'}^T.tensor
    = basisVector leftMetricMulRightMap (fun | 0 => 0 | 1 => 1 | 2 => 0 | 3 => 1)
    - basisVector leftMetricMulRightMap (fun | 0 => 0 | 1 => 1 | 2 => 1 | 3 => 0)
    - basisVector leftMetricMulRightMap (fun | 0 => 1 | 1 => 0 | 2 => 0 | 3 => 1)
    + basisVector leftMetricMulRightMap (fun | 0 => 1 | 1 => 0 | 2 => 1 | 3 => 0)
```

**lemma leftMetric_prod_rightMetric_tree** [source]

```
lemma leftMetric_prod_rightMetric_tree : {εL | α α' ⊗ εR | β β'}^T.tensor
    = (TensorTree.add (tensorNode
        (basisVector leftMetricMulRightMap (fun | 0 => 0 | 1 => 1 | 2 => 0 | 3 => 1))) <|
      TensorTree.add (TensorTree.smul (-1 : ℂ) (tensorNode
        (basisVector leftMetricMulRightMap (fun | 0 => 0 | 1 => 1 | 2 => 1 | 3 => 0)))) <|
      TensorTree.add (TensorTree.smul (-1 : ℂ) (tensorNode
        (basisVector leftMetricMulRightMap (fun | 0 => 1 | 1 => 0 | 2 => 0 | 3 => 1)))) <|
      (tensorNode
        (basisVector leftMetricMulRightMap (fun | 0 => 1 | 1 => 0 | 2 => 1 | 3 => 0)))).tensor
```

## 2.31 HepLean.Tensors.ComplexLorentz.PauliMatrices.Basic

HepLean/Tensors/ComplexLorentz/PauliMatrices/Basic.lean on GitHub.

**def pauliContr** [source]

*The Pauli matrices as the complex Lorentz tensor σ^μ^α^{dot β}.*

```
def pauliContr
```

**def pauliCo** [source]

*The Pauli matrices as the complex Lorentz tensor σ_μ^α^{dot β}.*

```
def pauliCo
```

**def pauliCoDown** [source]

*The Pauli matrices as the complex Lorentz tensor σ_μ_α_{dot β}.*

```
def pauliCoDown
```

**def pauliContrDown** [source]

*The Pauli matrices as the complex Lorentz tensor σ^μ_α_{dot β}.*

```
def pauliContrDown
```

**lemma tensorNode_pauliContr** [source]

*The definitional tensor node relation for `pauliContr`.*

```
lemma tensorNode_pauliContr : {pauliContr | μ α β}^T.tensor =
    {PauliMatrix.asConsTensor | ν α β}^T.tensor
```

**lemma tensorNode_pauliCo** [source]

*The definitional tensor node relation for `pauliCo`.*

```
lemma tensorNode_pauliCo : {pauliCo | μ α β}^T.tensor =
    {η' | μ ν ⊗ PauliMatrix.asConsTensor | ν α β}^T.tensor
```

**lemma tensorNode_pauliCoDown** [source]

*The definitional tensor node relation for `pauliCoDown`.*

```
lemma tensorNode_pauliCoDown : {pauliCoDown | μ α β}^T.tensor =
    {pauliCo | μ α β ⊗ εL' | α α' ⊗ εR' | β β'}^T.tensor
```

**lemma tensorNode_pauliContrDown** [source]

*The definitional tensor node relation for `pauliContrDown`.*

```
lemma tensorNode_pauliContrDown : {pauliContrDown | μ α β}^T.tensor =
    {pauliContr | μ α β ⊗ εL' | α α' ⊗ εR' | β β'}^T.tensor
```

**lemma pauliCoDown_eq_metric_mul_pauliCo** [source]

*A rearanging of `pauliCoDown` to place the pauli matrices on the right.*

```
lemma pauliCoDown_eq_metric_mul_pauliCo :
    {pauliCoDown | μ α' β' = εL' | α α' ⊗ εR' | β β' ⊗ pauliCo | μ α β}^T
```

**lemma pauliContrDown_eq_metric_mul_pauliContr** [source]

*A rearanging of `pauliContrDown` to place the pauli matrices on the right.*

```
lemma pauliContrDown_eq_metric_mul_pauliContr :
    {pauliContrDown | μ α' β' = εL' | α α' ⊗
    εR' | β β' ⊗ pauliContr | μ α β}^T
```

**lemma action_pauliContr** [source]

*The tensor `pauliContr` is invariant under the action of `SL(2,ℂ)`.*

```
lemma action_pauliContr (g : SL(2,ℂ)) : {g •ₐ pauliContr | μ α β}^T.tensor =
    {pauliContr | μ α β}^T.tensor
```

**lemma action_pauliCo** [source]

*The tensor `pauliCo` is invariant under the action of `SL(2,ℂ)`.*

```
lemma action_pauliCo (g : SL(2,ℂ)) : {g •ₐ pauliCo | μ α β}^{T}.tensor =
    {pauliCo | μ α β}^{T}.tensor
```

**lemma action_pauliCoDown** [source]

*The tensor `pauliCoDown` is invariant under the action of `SL(2,ℂ)`.*

```
lemma action_pauliCoDown (g : SL(2,ℂ)) : {g •ₐ pauliCoDown | μ α β}^{T}.tensor =
    {pauliCoDown | μ α β}^{T}.tensor
```

**lemma action_pauliContrDown** [source]

*The tensor `pauliContrDown` is invariant under the action of `SL(2,ℂ)`.*

```
lemma action_pauliContrDown (g : SL(2,ℂ)) : {g •ₐ pauliContrDown | μ α β}^{T}.tensor =
    {pauliContrDown | μ α β}^{T}.tensor
```

## 2.32 HepLean.Tensors.ComplexLorentz.PauliMatrices.Basis

HepLean/Tensors/ComplexLorentz/PauliMatrices/Basis.lean on GitHub.

**lemma pauliContr_in_basis** [source]

*The expansion of the Pauli matrices `σ^μ^a^{dot a}` in terms of basis vectors.*

```
lemma pauliContr_in_basis : {pauliContr | μ α β}^{T}.tensor =
    basisVector ![Color.up, Color.upL, Color.upR] (fun | 0 => 0 | 1 => 0 | 2 => 0)
    + basisVector ![Color.up, Color.upL, Color.upR] (fun | 0 => 0 | 1 => 1 | 2 => 1)
    + basisVector ![Color.up, Color.upL, Color.upR] (fun | 0 => 1 | 1 => 0 | 2 => 1)
    + basisVector ![Color.up, Color.upL, Color.upR] (fun | 0 => 1 | 1 => 1 | 2 => 0)
    - I • basisVector ![Color.up, Color.upL, Color.upR] (fun | 0 => 2 | 1 => 0 | 2 => 1)
    + I • basisVector ![Color.up, Color.upL, Color.upR] (fun | 0 => 2 | 1 => 1 | 2 => 0)
    + basisVector ![Color.up, Color.upL, Color.upR] (fun | 0 => 3 | 1 => 0 | 2 => 0)
    - basisVector ![Color.up, Color.upL, Color.upR] (fun | 0 => 3 | 1 => 1 | 2 => 1)
```

**lemma pauliContr_basis_expand_tree** [source]

```
lemma pauliContr_basis_expand_tree : {pauliContr | μ α β}^{T}.tensor =
    (TensorTree.add (tensorNode
      (basisVector ![Color.up, Color.upL, Color.upR] (fun | 0 => 0 | 1 => 0 | 2 => 0))) <|
    TensorTree.add (tensorNode
      (basisVector ![Color.up, Color.upL, Color.upR] (fun | 0 => 0 | 1 => 1 | 2 => 1))) <|
    TensorTree.add (tensorNode
      (basisVector ![Color.up, Color.upL, Color.upR] (fun | 0 => 1 | 1 => 0 | 2 => 1))) <|
    TensorTree.add (tensorNode
      (basisVector ![Color.up, Color.upL, Color.upR] (fun | 0 => 1 | 1 => 1 | 2 => 0))) <|
    TensorTree.add (smul (-I) (tensorNode
      (basisVector ![Color.up, Color.upL, Color.upR] (fun | 0 => 2 | 1 => 0 | 2 => 1)))) <|
    TensorTree.add (smul I (tensorNode
      (basisVector ![Color.up, Color.upL, Color.upR] (fun | 0 => 2 | 1 => 1 | 2 => 0)))) <|
    TensorTree.add (tensorNode
      (basisVector ![Color.up, Color.upL, Color.upR] (fun | 0 => 3 | 1 => 0 | 2 => 0))) <|
    (smul (-1) (tensorNode
      (basisVector ![Color.up, Color.upL, Color.upR]
        (fun | 0 => 3 | 1 => 1 | 2 => 1))))).tensor
```

**abbrev pauliMatrixContrMap** [source]

*The map to colors one gets when contracting with Pauli matrices on the right.*

```
abbrev pauliMatrixContrMap {n : ℕ} (c : Fin n → complexLorentzTensor.C)
```

**lemma prod_pauliMatrix_basis_tree_expand** [source]

```
lemma prod_pauliMatrix_basis_tree_expand {n : ℕ} {c : Fin n → complexLorentzTensor.C}
    (t : TensorTree complexLorentzTensor c) :
    (TensorTree.prod t (tensorNode pauliContr)).tensor = (((t.prod (tensorNode
      (basisVector ![Color.up, Color.upL, Color.upR] fun | 0 => 0 | 1 => 0 | 2 => 0)))).add
    (((t.prod (tensorNode
      (basisVector ![Color.up, Color.upL, Color.upR] fun | 0 => 0 | 1 => 1 | 2 => 1)))).add
    (((t.prod (tensorNode
      (basisVector ![Color.up, Color.upL, Color.upR] fun | 0 => 1 | 1 => 0 | 2 => 1)))).add
    (((t.prod (tensorNode
      (basisVector ![Color.up, Color.upL, Color.upR] fun | 0 => 1 | 1 => 1 | 2 => 0)))).add
    ((TensorTree.smul (-I) ((t.prod (tensorNode
      (basisVector ![Color.up, Color.upL, Color.upR] fun | 0 => 2 | 1 => 0 | 2 => 1))))).add
    ((TensorTree.smul I ((t.prod (tensorNode
      (basisVector ![Color.up, Color.upL, Color.upR] fun | 0 => 2 | 1 => 1 | 2 => 0))))).add
    ((t.prod (tensorNode
      (basisVector ![Color.up, Color.upL, Color.upR] fun | 0 => 3 | 1 => 0 | 2 => 0))).add
    (TensorTree.smul (-1) (t.prod (tensorNode
      (basisVector ![Color.up, Color.upL, Color.upR]
        fun | 0 => 3 | 1 => 1 | 2 => 1)))))))))))).tensor
```

**lemma contr_pauliMatrix_basis_tree_expand** [source]

```
lemma contr_pauliMatrix_basis_tree_expand {n : ℕ} {c : Fin n → complexLorentzTensor.C}
    (t : TensorTree complexLorentzTensor c) (i : Fin (n + 3)) (j : Fin (n +2))
    (h : (pauliMatrixContrMap c) (i.succAbove j) =
      complexLorentzTensor.τ ((pauliMatrixContrMap c) i)) :
    (contr i j h (TensorTree.prod t (tensorNode pauliContr))).tensor =
    ((contr i j h (t.prod (tensorNode
      (basisVector ![Color.up, Color.upL, Color.upR] fun | 0 => 0 | 1 => 0 | 2 => 0)))).add
    ((contr i j h (t.prod (tensorNode
      (basisVector ![Color.up, Color.upL, Color.upR] fun | 0 => 0 | 1 => 1 | 2 => 1)))).add
    ((contr i j h (t.prod (tensorNode
      (basisVector ![Color.up, Color.upL, Color.upR] fun | 0 => 1 | 1 => 0 | 2 => 1)))).add
    ((contr i j h (t.prod (tensorNode
      (basisVector ![Color.up, Color.upL, Color.upR] fun | 0 => 1 | 1 => 1 | 2 => 0)))).add
    ((TensorTree.smul (-I) (contr i j h (t.prod (tensorNode
      (basisVector ![Color.up, Color.upL, Color.upR] fun | 0 => 2 | 1 => 0 | 2 => 1))))).add
    ((TensorTree.smul I (contr i j h (t.prod (tensorNode
      (basisVector ![Color.up, Color.upL, Color.upR] fun | 0 => 2 | 1 => 1 | 2 => 0))))).add
    ((contr i j h (t.prod (tensorNode
      (basisVector ![Color.up, Color.upL, Color.upR] fun | 0 => 3 | 1 => 0 | 2 => 0)))).add
    (TensorTree.smul (-1) (contr i j h (t.prod (tensorNode
      (basisVector ![Color.up, Color.upL, Color.upR]
        fun | 0 => 3 | 1 => 1 | 2 => 1)))))))))))).tensor
```

**lemma basis_contr_pauliMatrix_basis_tree_expand'** [source]

```
lemma basis_contr_pauliMatrix_basis_tree_expand' {n : ℕ} {c : Fin n → complexLorentzTensor.C}
    (i : Fin (n + 3)) (j : Fin (n +2))
```

```
    (h : (pauliMatrixContrMap c) (i.succAbove j) = complexLorentzTensor.τ
      ((pauliMatrixContrMap c) i))
    (b : Π k, Fin (complexLorentzTensor.repDim (c k))) :
    let c'
```

**def pauliMatrixBasisProdMap** [source]

*The map to color which appears when contracting a basis vector with puali matrices.*

```
def pauliMatrixBasisProdMap
    {n : ℕ} {c : Fin n → complexLorentzTensor.C}
    (b : Π k, Fin (complexLorentzTensor.repDim (c k))) (i1 i2 i3 : Fin 4) :
    (i : Fin (n + (Nat.succ 0).succ.succ)) →
      Fin (complexLorentzTensor.repDim (Sum.elim c ![Color.up, Color.upL, Color.upR]
          (finSumFinEquiv.symm i)))
```

**def basisVectorContrPauli** [source]

*The new basis vectors which appear when contracting pauli matrices with basis vectors.*

```
def basisVectorContrPauli {n : ℕ} {c : Fin n → complexLorentzTensor.C}
    (i : Fin (n + 3)) (j : Fin (n +2))
    (b : Π k, Fin (complexLorentzTensor.repDim (c k)))
    (i1 i2 i3 : Fin 4)
```

**lemma basis_contr_pauliMatrix_basis_tree_expand** [source]

```
lemma basis_contr_pauliMatrix_basis_tree_expand {n : ℕ} {c : Fin n → complexLorentzTensor.C}
    (i : Fin (n + 3)) (j : Fin (n +2))
    (h : (pauliMatrixContrMap c) (i.succAbove j) = complexLorentzTensor.τ
      ((pauliMatrixContrMap c) i))
    (b : Π k, Fin (complexLorentzTensor.repDim (c k))) :
    let c'
```

**lemma basis_contr_pauliMatrix_basis_tree_expand_tensor** [source]

```
lemma basis_contr_pauliMatrix_basis_tree_expand_tensor {n : ℕ} {c : Fin n →
    complexLorentzTensor.C}
    (i : Fin (n + 3)) (j : Fin (n +2))
    (h : (pauliMatrixContrMap c) (i.succAbove j) = complexLorentzTensor.τ
      ((pauliMatrixContrMap c) i))
    (b : Π k, Fin (complexLorentzTensor.repDim (c k))) :
    (contr i j h (TensorTree.prod (tensorNode (basisVector c b))
    (tensorNode pauliContr))).tensor =
    (contrBasisVectorMul i j (pauliMatrixBasisProdMap b 0 0 0)) •
      (basisVectorContrPauli i j b 0 0 0)
    + (contrBasisVectorMul i j (pauliMatrixBasisProdMap b 0 1 1)) •
      (basisVectorContrPauli i j b 0 1 1)
    + (contrBasisVectorMul i j (pauliMatrixBasisProdMap b 1 0 1)) •
      (basisVectorContrPauli i j b 1 0 1)
    + (contrBasisVectorMul i j (pauliMatrixBasisProdMap b 1 1 0)) •
      (basisVectorContrPauli i j b 1 1 0)
    + (-I) • (contrBasisVectorMul i j (pauliMatrixBasisProdMap b 2 0 1)) •
      (basisVectorContrPauli i j b 2 0 1)
    + I • (contrBasisVectorMul i j (pauliMatrixBasisProdMap b 2 1 0)) •
      (basisVectorContrPauli i j b 2 1 0)
```

```
    + (contrBasisVectorMul i j (pauliMatrixBasisProdMap b 3 0 0)) •
      (basisVectorContrPauli i j b 3 0 0)
    + (-1 : ℂ) • (contrBasisVectorMul i j (pauliMatrixBasisProdMap b 3 1 1)) •
      (basisVectorContrPauli i j b 3 1 1)
```

### def pauliCoMap [source]

*The map to color one gets when lowering the indices of pauli matrices.*

```
def pauliCoMap
```

### lemma pauliMatrix_contr_down_0 [source]

```
lemma pauliMatrix_contr_down_0 :
    (contr 1 1 rfl (((tensorNode (basisVector ![Color.down, Color.down] fun x => 0)).prod
    (tensorNode pauliContr)))).tensor
    = basisVector pauliCoMap (fun | 0 => 0 | 1 => 0 | 2 => 0)
    + basisVector pauliCoMap (fun | 0 => 0 | 1 => 1 | 2 => 1)
```

### lemma pauliMatrix_contr_down_1 [source]

```
lemma pauliMatrix_contr_down_1 :
    {(basisVector ![Color.down, Color.down] fun x => 1) | ν μ ⊗
      pauliContr | μ α β}^T.tensor
    = basisVector pauliCoMap (fun | 0 => 1 | 1 => 0 | 2 => 1)
    + basisVector pauliCoMap (fun | 0 => 1 | 1 => 1 | 2 => 0)
```

### lemma pauliMatrix_contr_down_2 [source]

```
lemma pauliMatrix_contr_down_2 :
    {(basisVector ![Color.down, Color.down] fun x => 2) | μ ν ⊗
      pauliContr | ν α β}^T.tensor
    = (- I) • basisVector pauliCoMap (fun | 0 => 2 | 1 => 0 | 2 => 1)
    + (I) • basisVector pauliCoMap (fun | 0 => 2 | 1 => 1 | 2 => 0)
```

### lemma pauliMatrix_contr_down_3 [source]

```
lemma pauliMatrix_contr_down_3 :
    {(basisVector ![Color.down, Color.down] fun x => 3) | μ ν ⊗
    pauliContr | ν α β}^T.tensor
    = basisVector pauliCoMap (fun | 0 => 3 | 1 => 0 | 2 => 0)
    + (- 1 : ℂ) • basisVector pauliCoMap (fun | 0 => 3 | 1 => 1 | 2 => 1)
```

### lemma pauliCo_basis_expand [source]

*The expansion of `pauliCo` in terms of a basis.*

```
lemma pauliCo_basis_expand : pauliCo
    = basisVector pauliCoMap (fun | 0 => 0 | 1 => 0 | 2 => 0)
    + basisVector pauliCoMap (fun | 0 => 0 | 1 => 1 | 2 => 1)
    - basisVector pauliCoMap (fun | 0 => 1 | 1 => 0 | 2 => 1)
    - basisVector pauliCoMap (fun | 0 => 1 | 1 => 1 | 2 => 0)
    + I • basisVector pauliCoMap (fun | 0 => 2 | 1 => 0 | 2 => 1)
    - I • basisVector pauliCoMap (fun | 0 => 2 | 1 => 1 | 2 => 0)
```

```
    - basisVector pauliCoMap (fun | 0 => 3 | 1 => 0 | 2 => 0)
    + basisVector pauliCoMap (fun | 0 => 3 | 1 => 1 | 2 => 1)
```

**lemma pauliCo_basis_expand_tree** [source]

```
lemma pauliCo_basis_expand_tree : {pauliCo | μ α β}^T.tensor
    = (TensorTree.add (tensorNode
      (basisVector pauliCoMap (fun | 0 => 0 | 1 => 0 | 2 => 0))) <|
    TensorTree.add (tensorNode
      (basisVector pauliCoMap (fun | 0 => 0 | 1 => 1 | 2 => 1))) <|
    TensorTree.add (TensorTree.smul (-1) (tensorNode
      (basisVector pauliCoMap (fun | 0 => 1 | 1 => 0 | 2 => 1)))) <|
    TensorTree.add (TensorTree.smul (-1) (tensorNode
      (basisVector pauliCoMap (fun | 0 => 1 | 1 => 1 | 2 => 0)))) <|
    TensorTree.add (TensorTree.smul I (tensorNode
      (basisVector pauliCoMap (fun | 0 => 2 | 1 => 0 | 2 => 1)))) <|
    TensorTree.add (TensorTree.smul (-I) (tensorNode
      (basisVector pauliCoMap (fun | 0 => 2 | 1 => 1 | 2 => 0)))) <|
    TensorTree.add (TensorTree.smul (-1) (tensorNode
      (basisVector pauliCoMap (fun | 0 => 3 | 1 => 0 | 2 => 0)))) <|
      (tensorNode (basisVector pauliCoMap (fun | 0 => 3 | 1 => 1 | 2 => 1)))).tensor
```

**lemma pauliCo_prod_basis_expand** [source]

```
lemma pauliCo_prod_basis_expand {n : ℕ} {c : Fin n → complexLorentzTensor.C}
    (t : TensorTree complexLorentzTensor c) :
    (prod (tensorNode pauliCo) t).tensor =
    (((tensorNode
      (basisVector pauliCoMap fun | 0 => 0 | 1 => 0 | 2 => 0)).prod t).add
    (((tensorNode
      (basisVector pauliCoMap fun | 0 => 0 | 1 => 1 | 2 => 1)).prod t).add
    ((TensorTree.smul (-1) ((tensorNode
      (basisVector pauliCoMap fun | 0 => 1 | 1 => 0 | 2 => 1)).prod t)).add
    ((TensorTree.smul (-1) ((tensorNode
      (basisVector pauliCoMap fun | 0 => 1 | 1 => 1 | 2 => 0)).prod t)).add
    ((TensorTree.smul I ((tensorNode
      (basisVector pauliCoMap fun | 0 => 2 | 1 => 0 | 2 => 1)).prod t)).add
    ((TensorTree.smul (-I) ((tensorNode
      (basisVector pauliCoMap fun | 0 => 2 | 1 => 1 | 2 => 0)).prod t)).add
    ((TensorTree.smul (-1) ((tensorNode
      (basisVector pauliCoMap fun | 0 => 3 | 1 => 0 | 2 => 0)).prod t)).add
    ((tensorNode
      (basisVector pauliCoMap fun | 0 => 3 | 1 => 1 | 2 => 1)).prod
      t)))))))).tensor
```

## 2.33 HepLean.Tensors.ComplexLorentz.PauliMatrices.CoContractContr

HepLean/Tensors/ComplexLorentz/PauliMatrices/CoContractContr.lean on GitHub.

**def pauliMatrixContrPauliMatrixMap** [source]

*The map to colors one gets when contracting the 4-vector indices pauli matrices.*

```
def pauliMatrixContrPauliMatrixMap
```

**lemma pauliMatrix_contr_lower_0_0_0** [source]

```
lemma pauliMatrix_contr_lower_0_0_0 :
    {(basisVector pauliCoMap (fun | 0 => 0 | 1 => 0 | 2 => 0)) | μ α β ⊗
    pauliContr | μ α' β'}^T.tensor =
    basisVector pauliMatrixContrPauliMatrixMap (fun | 0 => 0 | 1 => 0 | 2 => 0 | 3 => 0)
    + basisVector pauliMatrixContrPauliMatrixMap (fun | 0 => 0 | 1 => 0 | 2 => 1 | 3 => 1)
```

**lemma pauliMatrix_contr_lower_0_1_1** [source]

```
lemma pauliMatrix_contr_lower_0_1_1 :
    {(basisVector pauliCoMap (fun | 0 => 0 | 1 => 1 | 2 => 1)) | μ α β ⊗
    pauliContr | μ α' β'}^T.tensor =
    basisVector pauliMatrixContrPauliMatrixMap (fun | 0 => 1 | 1 => 1 | 2 => 0 | 3 => 0)
    + basisVector pauliMatrixContrPauliMatrixMap (fun | 0 => 1 | 1 => 1 | 2 => 1 | 3 => 1)
```

**lemma pauliMatrix_contr_lower_1_0_1** [source]

```
lemma pauliMatrix_contr_lower_1_0_1 :
    {(basisVector pauliCoMap (fun | 0 => 1 | 1 => 0 | 2 => 1)) | μ α β ⊗
    pauliContr | μ α' β'}^T.tensor =
    basisVector pauliMatrixContrPauliMatrixMap (fun | 0 => 0 | 1 => 1 | 2 => 0 | 3 => 1)
    + basisVector pauliMatrixContrPauliMatrixMap (fun | 0 => 0 | 1 => 1 | 2 => 1 | 3 => 0)
```

**lemma pauliMatrix_contr_lower_1_1_0** [source]

```
lemma pauliMatrix_contr_lower_1_1_0 :
    {(basisVector pauliCoMap (fun | 0 => 1 | 1 => 1 | 2 => 0)) | μ α β ⊗
    pauliContr | μ α' β'}^T.tensor =
    basisVector pauliMatrixContrPauliMatrixMap (fun | 0 => 1 | 1 => 0 | 2 => 0 | 3 => 1)
    + basisVector pauliMatrixContrPauliMatrixMap (fun | 0 => 1 | 1 => 0 | 2 => 1 | 3 => 0)
```

**lemma pauliMatrix_contr_lower_2_0_1** [source]

```
lemma pauliMatrix_contr_lower_2_0_1 :
    {(basisVector pauliCoMap (fun | 0 => 2 | 1 => 0 | 2 => 1)) | μ α β ⊗
    pauliContr | μ α' β'}^T.tensor =
      (-I) • basisVector pauliMatrixContrPauliMatrixMap (fun | 0 => 0 | 1 => 1 | 2 => 0 | 3 =>
    1)
    + (I) •
      basisVector pauliMatrixContrPauliMatrixMap (fun | 0 => 0 | 1 => 1 | 2 => 1 | 3 => 0)
```

**lemma pauliMatrix_contr_lower_2_1_0** [source]

```
lemma pauliMatrix_contr_lower_2_1_0 :
    {(basisVector pauliCoMap (fun | 0 => 2 | 1 => 1 | 2 => 0)) | μ α β ⊗
    pauliContr | μ α' β'}^T.tensor =
      (-I) • basisVector pauliMatrixContrPauliMatrixMap (fun | 0 => 1 | 1 => 0 | 2 => 0 | 3 =>
    1)
    + (I) •
      basisVector pauliMatrixContrPauliMatrixMap (fun | 0 => 1 | 1 => 0 | 2 => 1 | 3 => 0)
```

**lemma pauliMatrix_contr_lower_3_0_0** [source]

```
lemma pauliMatrix_contr_lower_3_0_0 :
    {(basisVector pauliCoMap (fun | 0 => 3 | 1 => 0 | 2 => 0)) | μ α β ⊗
    pauliContr | μ α' β'}^T.tensor =
    basisVector pauliMatrixContrPauliMatrixMap (fun | 0 => 0 | 1 => 0 | 2 => 0 | 3 => 0)
    + (-1 : ℂ) • basisVector pauliMatrixContrPauliMatrixMap
    (fun | 0 => 0 | 1 => 0 | 2 => 1 | 3 => 1)
```

**lemma pauliMatrix_contr_lower_3_1_1** [source]

```
lemma pauliMatrix_contr_lower_3_1_1 :
    {(basisVector pauliCoMap (fun | 0 => 3 | 1 => 1 | 2 => 1)) | μ α β ⊗
    pauliContr | μ α' β'}^T.tensor =
    basisVector pauliMatrixContrPauliMatrixMap (fun | 0 => 1 | 1 => 1 | 2 => 0 | 3 => 0)
    + (-1 : ℂ) •
      basisVector pauliMatrixContrPauliMatrixMap (fun | 0 => 1 | 1 => 1 | 2 => 1 | 3 => 1)
```

**lemma pauliCo_prod_basis_expand'** [source]

*This lemma is exactly the same as `pauliCo_prod_basis_expand`. It is needed here for `pauliMatrix_contract_pauliMatrix_aux`. It is unclear why `pauliMatrix_lower_basis_expand_prod` does not work.*

```
private lemma pauliCo_prod_basis_expand' {n : ℕ} {c : Fin n → complexLorentzTensor.C}
    (t : TensorTree complexLorentzTensor c) :
    (TensorTree.prod (tensorNode pauliCo) t).tensor =
    ((((tensorNode
      (basisVector pauliCoMap fun | 0 => 0 | 1 => 0 | 2 => 0)).prod t).add
    (((tensorNode
      (basisVector pauliCoMap fun | 0 => 0 | 1 => 1 | 2 => 1)).prod t).add
    ((TensorTree.smul (-1) ((tensorNode
      (basisVector pauliCoMap fun | 0 => 1 | 1 => 0 | 2 => 1)).prod t)).add
    ((TensorTree.smul (-1) ((tensorNode
      (basisVector pauliCoMap fun | 0 => 1 | 1 => 1 | 2 => 0)).prod t)).add
    ((TensorTree.smul I ((tensorNode
      (basisVector pauliCoMap fun | 0 => 2 | 1 => 0 | 2 => 1)).prod t)).add
    ((TensorTree.smul (-I) ((tensorNode
      (basisVector pauliCoMap fun | 0 => 2 | 1 => 1 | 2 => 0)).prod t)).add
    ((TensorTree.smul (-1) ((tensorNode
      (basisVector pauliCoMap fun | 0 => 3 | 1 => 0 | 2 => 0)).prod t)).add
    ((tensorNode
      (basisVector pauliCoMap fun | 0 => 3 | 1 => 1 | 2 => 1)).prod
      t)))))))).tensor
```

**lemma pauliCo_contr_pauliContr_expand_aux** [source]

```
lemma pauliCo_contr_pauliContr_expand_aux :
    {pauliCo | μ α β ⊗ pauliContr | μ α' β'}^T.tensor
    = ((tensorNode
      ((basisVector pauliMatrixContrPauliMatrixMap fun | 0 => 0 | 1 => 0 | 2 => 0 | 3 => 0) +
        basisVector pauliMatrixContrPauliMatrixMap fun | 0 => 0 | 1 => 0 | 2 => 1 | 3 => 1)).
    add
    ((tensorNode
      ((basisVector pauliMatrixContrPauliMatrixMap fun | 0 => 1 | 1 => 1 | 2 => 0 | 3 => 0) +
      basisVector pauliMatrixContrPauliMatrixMap fun | 0 => 1 | 1 => 1 | 2 => 1 | 3 => 1)).add
    ((TensorTree.smul (-1) (tensorNode
```

```
    ((basisVector pauliMatrixContrPauliMatrixMap fun | 0 => 0 | 1 => 1 | 2 => 0 | 3 => 1) +
    basisVector pauliMatrixContrPauliMatrixMap fun | 0 => 0 | 1 => 1 | 2 => 1 | 3 => 0))).add
  ((TensorTree.smul (-1) (tensorNode
    ((basisVector pauliMatrixContrPauliMatrixMap fun | 0 => 1 | 1 => 0 | 2 => 0 | 3 => 1) +
    basisVector pauliMatrixContrPauliMatrixMap fun | 0 => 1 | 1 => 0 | 2 => 1 | 3 => 0))).add
  ((TensorTree.smul I (tensorNode
    ((-I • basisVector pauliMatrixContrPauliMatrixMap fun | 0 => 0 | 1 => 1 | 2 => 0 | 3 =>
  1) +
      I •
      basisVector pauliMatrixContrPauliMatrixMap fun | 0 => 0 | 1 => 1 | 2 => 1 | 3 => 0))).
  add
  ((TensorTree.smul (-I) (tensorNode
    ((-I • basisVector pauliMatrixContrPauliMatrixMap fun | 0 => 1 | 1 => 0 | 2 => 0 | 3 =>
  1) +
    I • basisVector pauliMatrixContrPauliMatrixMap fun | 0 => 1 | 1 => 0 | 2 => 1 | 3 => 0))).
  add
    ((TensorTree.smul (-1) (tensorNode
      ((basisVector pauliMatrixContrPauliMatrixMap fun | 0 => 0 | 1 => 0 | 2 => 0 | 3 => 0) +
      (-1 : ℂ) •
      basisVector pauliMatrixContrPauliMatrixMap fun | 0 => 0 | 1 => 0 | 2 => 1 | 3 => 1))).
  add
    (tensorNode
      ((basisVector pauliMatrixContrPauliMatrixMap fun | 0 => 1 | 1 => 1 | 2 => 0 | 3 => 0) +
      (-1 : ℂ) • basisVector pauliMatrixContrPauliMatrixMap
        fun | 0 => 1 | 1 => 1 | 2 => 1 | 3 => 1))))))))).tensor
```

**lemma pauliCo_contr_pauliContr_expand** [source]

```
lemma pauliCo_contr_pauliContr_expand :
    {pauliCo | ν α β ⊗ pauliContr | ν α' β'}^T.tensor =
    2 • basisVector pauliMatrixContrPauliMatrixMap (fun | 0 => 0 | 1 => 0 | 2 => 1 | 3 => 1)
    + 2 • basisVector pauliMatrixContrPauliMatrixMap (fun | 0 => 1 | 1 => 1 | 2 => 0 | 3 => 0)
    - 2 • basisVector pauliMatrixContrPauliMatrixMap (fun | 0 => 0 | 1 => 1 | 2 => 1 | 3 => 0)
    - 2 • basisVector pauliMatrixContrPauliMatrixMap (fun | 0 => 1 | 1 => 0 | 2 => 0 | 3 => 1)
```

**theorem pauliCo_contr_pauliContr** [source]

*The statement that `η_{μν} σ^{μ α dot β} σ^{ν α' dot β'} = 2 ε^{αα'} ε^{dot β dot β'}`.*

```
theorem pauliCo_contr_pauliContr :
    {pauliCo | ν α β ⊗ pauliContr | ν α' β' = 2 •ₜ εL | α α' ⊗ εR | β β'}^T
```

## 2.34 HepLean.Tensors.ComplexLorentz.Units.Basic

HepLean/Tensors/ComplexLorentz/Units/Basic.lean on GitHub.

**def coContrUnit** [source]

*The unit $\delta_i{}^i$ as a complex Lorentz tensor.*

```
def coContrUnit
```

**def contrCoUnit** [source]

*The unit $\delta^{i}{}_{i}$ as a complex Lorentz tensor.*

```
def contrCoUnit
```

**def altLeftLeftUnit** [source]

*The unit $\delta_{a}{}^{a}$ as a complex Lorentz tensor.*

```
def altLeftLeftUnit
```

**def leftAltLeftUnit** [source]

*The unit $\delta^{a}{}_{a}$ as a complex Lorentz tensor.*

```
def leftAltLeftUnit
```

**def altRightRightUnit** [source]

*The unit `δ_{dot a}^{dot a}` as a complex Lorentz tensor.*

```
def altRightRightUnit
```

**def rightAltRightUnit** [source]

*The unit `δ^{dot a}_{dot a}` as a complex Lorentz tensor.*

```
def rightAltRightUnit
```

**lemma tensorNode_coContrUnit** [source]

*The definitional tensor node relation for `coContrUnit`.*

```
lemma tensorNode_coContrUnit : {δ' | μ ν}ᵀ.tensor = (TensorTree.constTwoNodeE
    complexLorentzTensor
    Color.down Color.up Lorentz.coContrUnit).tensor
```

**lemma tensorNode_contrCoUnit** [source]

*The definitional tensor node relation for `contrCoUnit`.*

```
lemma tensorNode_contrCoUnit: {δ | μ ν}ᵀ.tensor = (TensorTree.constTwoNodeE
    complexLorentzTensor
    Color.up Color.down Lorentz.contrCoUnit).tensor
```

**lemma tensorNode_altLeftLeftUnit** [source]

*The definitional tensor node relation for `altLeftLeftUnit`.*

```
lemma tensorNode_altLeftLeftUnit : {δL' | μ ν}ᵀ.tensor = (TensorTree.constTwoNodeE
    complexLorentzTensor Color.downL Color.upL Fermion.altLeftLeftUnit).tensor
```

**lemma tensorNode_leftAltLeftUnit** [source]

*The definitional tensor node relation for `leftAltLeftUnit`.*

```
lemma tensorNode_leftAltLeftUnit : {δL | μ ν}^T.tensor = (TensorTree.constTwoNodeE
    complexLorentzTensor Color.upL Color.downL Fermion.leftAltLeftUnit).tensor
```

**lemma tensorNode_altRightRightUnit** [source]

*The definitional tensor node relation for `altRightRightUnit`.*

```
lemma tensorNode_altRightRightUnit: {δR' | μ ν}^T.tensor = (TensorTree.constTwoNodeE
    complexLorentzTensor Color.downR Color.upR Fermion.altRightRightUnit).tensor
```

**lemma tensorNode_rightAltRightUnit** [source]

*The definitional tensor node relation for `rightAltRightUnit`.*

```
lemma tensorNode_rightAltRightUnit: {δR | μ ν}^T.tensor = (TensorTree.constTwoNodeE
    complexLorentzTensor Color.upR Color.downR Fermion.rightAltRightUnit).tensor
```

**lemma action_coContrUnit** [source]

*The tensor `coContrUnit` is invariant under the action of `SL(2,ℂ)`.*

```
lemma action_coContrUnit (g : SL(2,ℂ)) : {g •ₐ δ' | μ ν}^T.tensor = {δ' | μ ν}^T.tensor
```

**lemma action_contrCoUnit** [source]

*The tensor `contrCoUnit` is invariant under the action of `SL(2,ℂ)`.*

```
lemma action_contrCoUnit (g : SL(2,ℂ)) : {g •ₐ δ | μ ν}^T.tensor = {δ | μ ν}^T.tensor
```

**lemma action_altLeftLeftUnit** [source]

*The tensor `altLeftLeftUnit` is invariant under the action of `SL(2,ℂ)`.*

```
lemma action_altLeftLeftUnit (g : SL(2,ℂ)) : {g •ₐ δL' | μ ν}^T.tensor = {δL' | μ ν}^T.tensor
```

**lemma action_leftAltLeftUnit** [source]

*The tensor `leftAltLeftUnit` is invariant under the action of `SL(2,ℂ)`.*

```
lemma action_leftAltLeftUnit (g : SL(2,ℂ)) : {g •ₐ δL | μ ν}^T.tensor = {δL | μ ν}^T.tensor
```

**lemma action_altRightRightUnit** [source]

*The tensor `altRightRightUnit` is invariant under the action of `SL(2,ℂ)`.*

```
lemma action_altRightRightUnit (g : SL(2,ℂ)) : {g •ₐ δR' | μ ν}^T.tensor = {δR' | μ ν}^T.tensor
```

**lemma action_rightAltRightUnit** [source]

*The tensor `rightAltRightUnit` is invariant under the action of `SL(2,ℂ)`.*

```
lemma action_rightAltRightUnit (g : SL(2,ℂ)) : {g •ₐ δR | μ ν}^T.tensor = {δR | μ ν}^T.tensor
```

## 2.35 HepLean.Tensors.ComplexLorentz.Units.Symm

HepLean/Tensors/ComplexLorentz/Units/Symm.lean on GitHub.

**informal_lemma coContrUnit_symm** [source]

```
informal_lemma coContrUnit_symm where
  math :≈ "Swapping indices of coContrUnit returns contrCoUnit, i.e. {δ' | μ ν = δ | ν μ}.^T"
  deps :≈ [``coContrUnit, ``contrCoUnit]
```

**informal_lemma contrCoUnit_symm** [source]

```
informal_lemma contrCoUnit_symm where
  math :≈ "Swapping indices of contrCoUnit returns coContrUnit, i.e. {δ | μ ν = δ' | ν μ}^T"
  deps :≈ [``contrCoUnit, ``coContrUnit]
```

**informal_lemma altLeftLeftUnit_symm** [source]

```
informal_lemma altLeftLeftUnit_symm where
  math :≈ "Swapping indices of altLeftLeftUnit returns leftAltLeftUnit, i.e.
    {δL' | α α' = δL | α' α}^T"
  deps :≈ [``altLeftLeftUnit, ``leftAltLeftUnit]
```

**informal_lemma leftAltLeftUnit_symm** [source]

```
informal_lemma leftAltLeftUnit_symm where
  math :≈ "Swapping indices of leftAltLeftUnit returns altLeftLeftUnit, i.e.
    {δL | α α' = δL' | α' α}^T"
  deps :≈ [``leftAltLeftUnit, ``altLeftLeftUnit]
```

**informal_lemma altRightRightUnit_symm** [source]

```
informal_lemma altRightRightUnit_symm where
  math :≈ "Swapping indices of altRightRightUnit returns rightAltRightUnit, i.e.
    {δR' | β β' = δR | β' β}^T"
  deps :≈ [``altRightRightUnit, ``rightAltRightUnit]
```

**informal_lemma rightAltRightUnit_symm** [source]

```
informal_lemma rightAltRightUnit_symm where
  math :≈ "Swapping indices of rightAltRightUnit returns altRightRightUnit, i.e.
    {δR | β β' = δR' | β' β}^T"
  deps :≈ [``rightAltRightUnit, ``altRightRightUnit]
```

## 2.36 HepLean.Tensors.OverColor.Basic

HepLean/Tensors/OverColor/Basic.lean on GitHub.

**def OverColor** [source]

*The core of the category of Types over C.*

```
def OverColor (C : Type)
```

**instance Groupoid (OverColor C)** [source]

*The instance of `OverColor C` as a groupoid.*

```
instance (C : Type) : Groupoid (OverColor C)
```

**lemma ext** [source]

```
lemma ext (m n : f ⟶ g) (h : m.hom = n.hom) : m = n
```

**lemma ext_iff** [source]

```
lemma ext_iff (m n : f ⟶ g) : (∀ x, m.hom.left x = n.hom.left x) ↔ m = n
```

**def toEquiv** [source]

*Given a hom in `OverColor C` the underlying equivalence between types.*

```
def toEquiv (m : f ⟶ g) : f.left ≃ g.left where
```

**lemma toEquiv_id** [source]

```
@[simp]
lemma toEquiv_id (f : OverColor C) : toEquiv (𝟙 f) = Equiv.refl f.left
```

**lemma toEquiv_comp** [source]

```
@[simp]
lemma toEquiv_comp (m : f ⟶ g) (n : g ⟶ h) : toEquiv (m ≫ n) = (toEquiv m).trans (toEquiv
    n)
```

**lemma toEquiv_symm_apply** [source]

```
lemma toEquiv_symm_apply (m : f ⟶ g) (i : g.left) :
    f.hom ((toEquiv m).symm i) = g.hom i
```

**lemma toEquiv_comp_hom** [source]

```
lemma toEquiv_comp_hom (m : f ⟶ g) : g.hom ∘ (toEquiv m) = f.hom
```

**lemma toEquiv_comp_inv_apply** [source]

```
lemma toEquiv_comp_inv_apply (m : f ⟶ g) (i : g.left) :
    f.hom ((OverColor.Hom.toEquiv m).symm i) = g.hom i
```

**lemma toEquiv_comp_apply** [source]

```
lemma toEquiv_comp_apply (m : f ⟶ g) (i : f.left) :
    f.hom i = g.hom ((toEquiv m) i)
```

**def toIso** [source]

*Given a morphism in `OverColor C`, the corresponding isomorphism.*

```
def toIso (m : f ⟶ g) : f ≅ g
```

**instance MonoidalCategoryStruct (OverColor C)** [source]

```
@[simps!]
instance (C : Type) : MonoidalCategoryStruct (OverColor C) where
```

**instance MonoidalCategory (OverColor C)** [source]

```
instance (C : Type) : MonoidalCategory (OverColor C) where
```

**instance BraidedCategory (OverColor C)** [source]

```
instance (C : Type) : BraidedCategory (OverColor C) where
```

**instance SymmetricCategory (OverColor C)** [source]

```
instance (C : Type) : SymmetricCategory (OverColor C) where
```

**def mk** [source]

*Make an object of `OverColor C` from a map `X → C`.*

```
def mk (f : X → C) : OverColor C
```

**lemma mk_hom** [source]

```
@[simp]
lemma mk_hom (f : X → C) : (mk f).hom = f
```

**lemma fin_ext** [source]

```
lemma Hom.fin_ext {n : ℕ} {f g : Fin n → C} (σ σ' : OverColor.mk f ⟶ OverColor.mk g)
    (h : ∀ (i : Fin n), σ.hom.left i = σ'.hom.left i) : σ = σ'
```

## 2.37 HepLean.Tensors.OverColor.Discrete

HepLean/Tensors/OverColor/Discrete.lean on GitHub.

**def pair** [source]

*The functor taking `c` to `F c ⊗ F c`.*

```
def pair : Discrete C ⥤ Rep k G
```

**def pairIso** [source]

*The isomorphism between the image of `(pair F).obj` and `(lift.obj F).obj (OverColor.mk ![c,c])`.*

```
def pairIso (c : C) : (pair F).obj (Discrete.mk c) ≅ (lift.obj F).obj (OverColor.mk ![c,c])
```

**def pairIsoSep** [source]

*The isomorphism between `F.obj (Discrete.mk c1) ⊗ F.obj (Discrete.mk c2)` and `(lift.obj F).obj (OverColor.mk ![c1,c2])` formed by the tensorate.*

```
def pairIsoSep {c1 c2 : C} : F.obj (Discrete.mk c1) ⊗ F.obj (Discrete.mk c2) ≅
    (lift.obj F).obj (OverColor.mk ![c1,c2])
```

**lemma pairIsoSep_tmul** [source]

```
lemma pairIsoSep_tmul {c1 c2 : C} (x : F.obj (Discrete.mk c1)) (y : F.obj (Discrete.mk c2)) :
    (pairIsoSep F).hom.hom (x ⊗ₜ[k] y) =
    PiTensorProduct.tprod k (Fin.cases x (Fin.cases y (fun i => Fin.elim0 i)))
```

**def tripleIsoSep** [source]

*The isomorphism between `F.obj (Discrete.mk c1) ⊗ F.obj (Discrete.mk c2) ⊗ F.obj (Discrete.mk c3)` and `(lift.obj F).obj (OverColor.mk ![c1,c2])` formed by the tensorate.*

```
def tripleIsoSep {c1 c2 c3 : C} :
    F.obj (Discrete.mk c1) ⊗ F.obj (Discrete.mk c2) ⊗ F.obj (Discrete.mk c3) ≅
    (lift.obj F).obj (OverColor.mk ![c1,c2,c3])
```

**lemma tripleIsoSep_tmul** [source]

```
lemma tripleIsoSep_tmul {c1 c2 c3 : C} (x : F.obj (Discrete.mk c1)) (y : F.obj (Discrete.mk c2)
    )
    (z : F.obj (Discrete.mk c3)) :
    (tripleIsoSep F).hom.hom (x ⊗ₜ[k] y ⊗ₜ[k] z) = PiTensorProduct.tprod k
      (fun | (0 : Fin 3) => x | (1 : Fin 3) => y | (2 : Fin 3) => z)
```

**def pairτ** [source]

*The functor taking `c` to `F c ⊗ F (τ c)`.*

```
def pairτ (τ : C → C) : Discrete C ⥤ Rep k G
```

**lemma pairτ_tmul** [source]

```
lemma pairτ_tmul {c : C} (x : F.obj (Discrete.mk c))
    (y : ↑(((Action.functorCategoryEquivalence (ModuleCat k) (MonCat.of G)).symm.inverse.obj
    ((Discrete.functor (Discrete.mk ∘ τ) ⋙ F).obj { as := c })).obj PUnit.unit)) (h : c = c1)
     :
    ((pairτ F τ).map (Discrete.eqToHom h)).hom (x ⊗ₜ[k] y)= ((F.map (Discrete.eqToHom h)).hom x
    )
    ⊗ₜ[k] ((F.map (Discrete.eqToHom (by simp [h]))).hom y)
```

**def τPair** [source]

*The functor taking `c` to `F (τ c) ⊗ F c`.*

```
def τPair (τ : C → C) : Discrete C ⥤ Rep k G
```

## 2.38 HepLean.Tensors.OverColor.Functors

HepLean/Tensors/OverColor/Functors.lean on GitHub.

**def map** [source]

*The monoidal functor from `OverColor C` to `OverColor D` constructed from a map `f : C → D`.*

```
def map {C D : Type} (f : C → D) : MonoidalFunctor (OverColor C) (OverColor D) where
```

**def tensor** [source]

*The tensor product on `OverColor C` as a monoidal functor.*

```
def tensor (C : Type) : MonoidalFunctor (OverColor C × OverColor C) (OverColor C) where
```

**def diag** [source]

*The monoidal functor from `OverColor C` to `OverColor C × OverColor C` landing on the diagonal.*

```
def diag (C : Type) : MonoidalFunctor (OverColor C) (OverColor C × OverColor C)
```

**def const** [source]

*The constant monoidal functor from `OverColor C` to itself landing on `𝟙_ (OverColor C)`.*

```
def const (C : Type) : MonoidalFunctor (OverColor C) (OverColor C) where
```

**def contrPair** [source]

*The monoidal functor from `OverColor C` to `OverColor C` taking `f` to `f ⊗ τ_* f`.*

```
def contrPair (C : Type) (τ : C → C) : MonoidalFunctor (OverColor C) (OverColor C)
```

## 2.39 HepLean.Tensors.OverColor.Iso

HepLean/Tensors/OverColor/Iso.lean on GitHub.

### def equivToIso [source]

*The isomorphism between `c : X → C` and `c ∘ e.symm` as objects in `OverColor C` for an equivalence `e`.*

```
def equivToIso {c : X → C} (e : X ≃ Y) : mk c ≅ mk (c ∘ e.symm)
```

### lemma equivToIso_homToEquiv [source]

```
@[simp]
lemma equivToIso_homToEquiv {c : X → C} (e : X ≃ Y) :
    Hom.toEquiv (equivToIso (c := c) e).hom = e
```

### def equivToHom [source]

*The homomorphism between `c : X → C` and `c ∘ e.symm` as objects in `OverColor C` for an equivalence `e`.*

```
def equivToHom {c : X → C} (e : X ≃ Y) : mk c ⟶ mk (c ∘ e.symm)
```

### def mkSum [source]

*Given a map `X ⊕ Y → C`, the isomorphism `mk c ≅ mk (c ∘ Sum.inl) ⊗ mk (c ∘ Sum.inr)`.*

```
def mkSum (c : X ⊕ Y → C) : mk c ≅ mk (c ∘ Sum.inl) ⊗ mk (c ∘ Sum.inr)
```

### lemma mkSum_homToEquiv [source]

```
@[simp]
lemma mkSum_homToEquiv {c : X ⊕ Y → C}:
    Hom.toEquiv (mkSum c).hom = (Equiv.refl _)
```

### lemma mkSum_inv_homToEquiv [source]

```
@[simp]
lemma mkSum_inv_homToEquiv {c : X ⊕ Y → C}:
    Hom.toEquiv (mkSum c).inv = (Equiv.refl _)
```

### def mkIso [source]

*The isomorphism between objects in `OverColor C` given equality of maps.*

```
def mkIso {c1 c2 : X → C} (h : c1 = c2) : mk c1 ≅ mk c2
```

### lemma mkIso_refl_hom [source]

```
lemma mkIso_refl_hom {c : X → C} : (mkIso (by rfl : c =c)).hom = 𝟙 _
```

**lemma equivToIso_mkIso_hom** [source]

```
@[simp]
lemma equivToIso_mkIso_hom {c1 c2 : X → C} (h : c1 = c2) :
    Hom.toEquiv (mkIso h).hom = Equiv.refl _
```

**lemma equivToIso_mkIso_inv** [source]

```
@[simp]
lemma equivToIso_mkIso_inv {c1 c2 : X → C} (h : c1 = c2) :
    Hom.toEquiv (mkIso h).inv = Equiv.refl _
```

**def equivToHomEq** [source]

*The homorophism from `mk c` to `mk c1` obtaied by an equivalence and an equality lemma.*

```
def equivToHomEq {c : X → C} {c1 : Y → C} (e : X ≃ Y)
    (h : ∀ x, c1 x = (c ∘ e.symm) x := by decide) : mk c ⟶ mk c1
```

**def fin2Iso** [source]

*The isomorphism splitting a `mk c` for `Fin 2 → C` into the tensor product of the `Fin 1 → C` maps defined by `c 0` and `c 1`.*

```
def fin2Iso {c : Fin 2 → C} : mk c ≅ mk ![c 0] ⊗ mk ![c 1]
```

**def fin3Iso** [source]

*The isomorphism splitting a `mk c` for `c : Fin 3 → C` into the tensor product of a `Fin 1 → C` map `![c 0]` and a `Fin 2 → C` map `![c 1, c 2]`.*

```
def fin3Iso {c : Fin 3 → C} : mk c ≅ mk ![c 0] ⊗ mk ![c 1, c 2]
```

**def fin3Iso'** [source]

*The isomorphism splitting a `mk ![c1, c2, c3]` into the tensor product of a `Fin 1 → C` map `fun _ => c1` and a `Fin 2 → C` map `![c 1, c 2]`.*

```
def fin3Iso' {c1 c2 c3 : C} : mk ![c1, c2, c3] ≅ mk (fun (_ : Fin 1) => c1) ⊗ mk ![c2, c3]
```

**def extractOne** [source]

*Removes a given `i : Fin n.succ.succ` from a morphism in `OverColor C`.*

```
def extractOne {n : ℕ} (i : Fin n.succ.succ)
    {c1 c2 : Fin n.succ.succ → C} (σ : mk c1 ⟶ mk c2) :
    mk (c1 ∘ Fin.succAbove ((Hom.toEquiv σ).symm i)) ⟶ mk (c2 ∘ Fin.succAbove i)
```

**lemma extractOne_homToEquiv** [source]

```
@[simp]
lemma extractOne_homToEquiv {n : ℕ} (i : Fin n.succ.succ)
    {c1 c2 : Fin n.succ.succ → C} (σ : mk c1 ⟶ mk c2) : Hom.toEquiv (extractOne i σ) =
```

```
    (finExtractOnePerm ((Hom.toEquiv σ).symm i) (Hom.toEquiv σ))
```

**def extractTwo** [source]

*Removes a given `i : Fin n.succ.succ` and `j : Fin n.succ` from a morphism in `OverColor C`.*

```
def extractTwo {n : ℕ} (i : Fin n.succ.succ) (j : Fin n.succ)
    {c1 c2 : Fin n.succ.succ → C} (σ : mk c1 ⟶ mk c2) :
    mk (c1 ∘ Fin.succAbove ((Hom.toEquiv σ).symm i) ∘
      Fin.succAbove (((Hom.toEquiv (extractOne i σ))).symm j)) ⟶
    mk (c2 ∘ Fin.succAbove i ∘ Fin.succAbove j)
```

**def extractTwoAux** [source]

*Removes a given `i : Fin n.succ.succ` and `j : Fin n.succ` from a morphism in `OverColor C`. This is from and to different (by equivalent) objects to `extractTwo`.*

```
def extractTwoAux {n : ℕ} (i : Fin n.succ.succ) (j : Fin n.succ)
    {c c1 : Fin n.succ.succ → C} (σ : mk c ⟶ mk c1) :
    mk ((c ∘ ⇑(finExtractTwo ((Hom.toEquiv σ).symm i)
    ((Hom.toEquiv (extractOne i σ)).symm j)).symm) ∘ Sum.inr) ⟶
    mk ((c1 ∘ ⇑(finExtractTwo i j).symm) ∘ Sum.inr)
```

**def extractTwoAux'** [source]

*Given a morphism `mk c ⟶ mk c1` the corresponding morphism on the `Fin 1 ⊕ Fin 1` maps obtained by extracting `i` and `j`.*

```
def extractTwoAux' {n : ℕ} (i : Fin n.succ.succ) (j : Fin n.succ)
    {c c1 : Fin n.succ.succ → C} (σ : mk c ⟶ mk c1) :
  mk ((c ∘ ⇑(finExtractTwo ((Hom.toEquiv σ).symm i)
  ((Hom.toEquiv (extractOne i σ)).symm j)).symm) ∘ Sum.inl) ⟶
  mk ((c1 ∘ ⇑(finExtractTwo i j).symm) ∘ Sum.inl)
```

**lemma extractTwo_finExtractTwo_succ** [source]

```
lemma extractTwo_finExtractTwo_succ {n : ℕ} (i : Fin n.succ.succ.succ) (j : Fin n.succ.succ)
    {c c1 : Fin n.succ.succ.succ → C} (σ : mk c ⟶ mk c1) :
    σ ≫ (equivToIso (HepLean.Fin.finExtractTwo i j)).hom ≫
    (mkSum (c1 ∘ ⇑(HepLean.Fin.finExtractTwo i j).symm)).hom =
    (equivToIso (HepLean.Fin.finExtractTwo ((Hom.toEquiv σ).symm i)
    (((Hom.toEquiv (extractOne i σ))).symm j))).hom
    ≫ (mkSum (c ∘ ⇑(HepLean.Fin.finExtractTwo ((Hom.toEquiv σ).symm i)
    (((Hom.toEquiv (extractOne i σ))).symm j)).symm)).hom
    ≫ ((extractTwoAux' i j σ) ⊗ (extractTwoAux i j σ))
```

**lemma extractTwo_finExtractTwo** [source]

```
lemma extractTwo_finExtractTwo {n : ℕ} (i : Fin n.succ.succ) (j : Fin n.succ)
    {c c1 : Fin n.succ.succ → C} (σ : mk c ⟶ mk c1) :
    σ ≫ (equivToIso (HepLean.Fin.finExtractTwo i j)).hom ≫
    (mkSum (c1 ∘ ⇑(HepLean.Fin.finExtractTwo i j).symm)).hom =
    (equivToIso (HepLean.Fin.finExtractTwo ((Hom.toEquiv σ).symm i)
    (((Hom.toEquiv (extractOne i σ))).symm j))).hom
```

```
    ≫ (mkSum (c ∘ ⇑(HepLean.Fin.finExtractTwo ((Hom.toEquiv σ).symm i)
    (((Hom.toEquiv (extractOne i σ))).symm j)).symm)).hom
    ≫ ((extractTwoAux' i j σ) ⊗ (extractTwoAux i j σ))
```

**def contrPairFin1Fin1** [source]

*The isomorphism between a `Fin 1 ⊕ Fin 1 → C` satisfying the condition `c (Sum.inr 0) = τ (c (Sum.inl 0))` and an object in the image of `contrPair`.*

```
def contrPairFin1Fin1 (τ : C → C) (c : Fin 1 ⊕ Fin 1 → C)
    (h : c (Sum.inr 0) = τ (c (Sum.inl 0))) :
    OverColor.mk c ≅ (contrPair C τ).obj (OverColor.mk (fun (_ : Fin 1) => c (Sum.inl 0)))
```

**def contrPairEquiv** [source]

*The Isomorphism between a `Fin n.succ.succ → C` and the product containing an object in the image of `contrPair` based on the given values.*

```
def contrPairEquiv {n : ℕ} (τ : C → C) (c : Fin n.succ.succ → C) (i : Fin n.succ.succ)
    (j : Fin n.succ) (h : c (i.succAbove j) = τ (c i)) :
    OverColor.mk c ≅ ((contrPair C τ).obj (Over.mk (fun (_ : Fin 1) => c i))) ⊗
    (OverColor.mk (c ∘ i.succAbove ∘ j.succAbove))
```

**def permFinOne** [source]

*Given a function `c` from `Fin 1` to `C`, this function returns a morphism from `mk c` to `mk ![c 0]`. -*

```
def permFinOne (c : Fin 1 → C) : mk c ⟶ mk ![c 0]
```

**def permFinTwo** [source]

*This a function that takes a function `c` from `Fin 2` to `C` and returns a morphism from `mk c` to `mk ![c 0, c 1]`. -*

```
def permFinTwo (c : Fin 2 → C) : mk c ⟶ mk ![c 0, c 1]
```

**def permFinThree** [source]

*This is a function that takes a function `c` from `Fin 3` to `C` and returns a morphism from `mk c` to `mk ![c 0, c 1, c 2]`. -*

```
def permFinThree (c : Fin 3 → C) : mk c ⟶ mk ![c 0, c 1, c 2]
```

## 2.40 HepLean.Tensors.OverColor.Lift

HepLean/Tensors/OverColor/Lift.lean on GitHub.

**def homToIso** [source]

*Takes a homorphism in `Discrete C` to an isomorphism built on the same objects.*

```
def homToIso {c1 c2 : Discrete C} (h : c1 ⟶ c2) : c1 ≅ c2
```

**def discreteFunctorMapIso** [source]

*Takes a homorphisms of `Discrete C` to an isomorphism `F.obj c1 ≅ F.obj c2`.*

```
def discreteFunctorMapIso {c1 c2 : Discrete C} (h : c1 ⟶ c2) :
    F.obj c1 ≅ F.obj c2
```

**lemma discreteFun_hom_trans** [source]

```
lemma discreteFun_hom_trans {c1 c2 c3 : Discrete C} (h1 : c1 = c2) (h2 : c2 = c3)
    (v : F.obj c1) : (F.map (eqToHom h2)).hom ((F.map (eqToHom h1)).hom v)
    = (F.map (eqToHom (h1.trans h2))).hom v
```

**def discreteFunctorMapEqIso** [source]

*The linear equivalence between `(F.obj c1).V ≃ₗ[k] (F.obj c2).V` induced by an equality of `c1` and `c2`.*

```
def discreteFunctorMapEqIso {c1 c2 : Discrete C} (h : c1.as = c2.as) :
    (F.obj c1).V ≃ₗ[k] (F.obj c2).V
```

**lemma discreteFunctorMapEqIso_comm_ρ** [source]

```
lemma discreteFunctorMapEqIso_comm_ρ {c1 c2 : Discrete C} (h : c1.as = c2.as) (M : G)
    (x : F.obj c1) : discreteFunctorMapEqIso F h ((F.obj c1).ρ M x) =
    (F.obj c2).ρ M (discreteFunctorMapEqIso F h x)
```

**def objObj'** [source]

*Given a object in `OverColor Color` the correpsonding tensor product of representations.*

```
def objObj' (f : OverColor C) : Rep k G
```

**lemma objObj'_ρ** [source]

```
lemma objObj'_ρ (f : OverColor C) (M : G) : (objObj' F f).ρ M =
    PiTensorProduct.map (fun x => (F.obj (Discrete.mk (f.hom x))).ρ M)
```

**lemma objObj'_ρ_tprod** [source]

```
lemma objObj'_ρ_tprod (f : OverColor C) (M : G) (x : (i : f.left) → F.obj (Discrete.mk (f.hom i
    ))) :
    (objObj' F f).ρ M (PiTensorProduct.tprod k x) =
    PiTensorProduct.tprod k fun i => (F.obj (Discrete.mk (f.hom i))).ρ M (x i)
```

**lemma objObj'_ρ_empty** [source]

```
@[simp]
lemma objObj'_ρ_empty (g : G) : (objObj' F (𝟙_ (OverColor C))).ρ g = LinearMap.id
```

**lemma objObj'_V_carrier** [source]

```
@[simp]
lemma objObj'_V_carrier (f : OverColor C) :
    (objObj' F f).V = ⨂[k] (i : f.left), F.obj (Discrete.mk (f.hom i))
```

**def mapToLinearEquiv'** [source]

*Given a morphism in `OverColor C` the corresopnding linear equivalence between `obj' _` induced by reindexing.*

```
def mapToLinearEquiv' {f g : OverColor C} (m : f ⟶ g) : (objObj' F f).V ≃ₗ[k] (objObj' F g).V
```

**lemma mapToLinearEquiv'_tprod** [source]

```
lemma mapToLinearEquiv'_tprod {f g : OverColor C} (m : f ⟶ g)
    (x : (i : f.left) → (F.obj (Discrete.mk (f.hom i)))) :
    mapToLinearEquiv' F m (PiTensorProduct.tprod k x) =
    PiTensorProduct.tprod k (fun i => (discreteFunctorMapEqIso F (Hom.toEquiv_symm_apply m i))
    (x ((OverColor.Hom.toEquiv m).symm i)))
```

**def objMap'** [source]

*Given a morphism in `OverColor C` the corresopnding map of representations induced by reindexing.*

```
def objMap' {f g : OverColor C} (m : f ⟶ g) : objObj' F f ⟶ objObj' F g where
```

**lemma objMap'_tprod** [source]

```
lemma objMap'_tprod {X Y : OverColor C} (p : (i : X.left) → F.obj (Discrete.mk (X.hom i)))
    (f : X ⟶ Y) : (objMap' F f).hom (PiTensorProduct.tprod k p) =
    PiTensorProduct.tprod k fun (i : Y.left) => discreteFunctorMapEqIso F
    (OverColor.Hom.toEquiv_comp_inv_apply f i) (p ((OverColor.Hom.toEquiv f).symm i))
```

**def ε** [source]

*The unit natural isomorphism.*

```
def ε : 𝟙_ (Rep k G) ≅ objObj' F (𝟙_ (OverColor C))
```

**def discreteSumEquiv** [source]

*An auxillary equivalence, and trivial, of modules needed to define `μModEquiv`.*

```
def discreteSumEquiv {X Y : OverColor C} (i : X.left ⊕ Y.left) :
    Sum.elim (fun i => F.obj (Discrete.mk (X.hom i)))
    (fun i => F.obj (Discrete.mk (Y.hom i))) i ≃ₗ[k] F.obj (Discrete.mk ((X ⊗ Y).hom i))
```

**def discreteSumEquiv'** [source]

*An equivalence used in the lemma of `μ_tmul_tprod_mk`. Identical to `μModEquiv` except with arguments based on maps instead of elements of `OverColor C`.*

```
def discreteSumEquiv' {X Y : Type} {cX : X → C} {cY : Y → C} (i : X ⊕ Y) :
    Sum.elim (fun i => F.obj (Discrete.mk (cX i)))
    (fun i => F.obj (Discrete.mk (cY i))) i ≃ₗ[k] F.obj (Discrete.mk ((Sum.elim cX cY) i))
```

**def μModEquiv** [source]

*The equivalence of modules corresonding to the tensorate.*

```
def μModEquiv (X Y : OverColor C) :
    (objObj' F X ⊗ objObj' F Y).V ≃ₗ[k] objObj' F (X ⊗ Y)
```

**lemma μModEquiv_tmul_tprod** [source]

```
lemma μModEquiv_tmul_tprod {X Y : OverColor C}
    (p : (i : X.left) → F.obj (Discrete.mk (X.hom i)))
    (q : (i : Y.left) → F.obj (Discrete.mk (Y.hom i))) :
    μModEquiv F X Y (PiTensorProduct.tprod k p ⊗ₜ[k] PiTensorProduct.tprod k q) =
    PiTensorProduct.tprod k fun i =>
    (discreteSumEquiv F i) (HepLean.PiTensorProduct.elimPureTensor p q i)
```

**def μ** [source]

*The natural isomorphism corresponding to the tensorate.*

```
def μ (X Y : OverColor C) : objObj' F X ⊗ objObj' F Y ≅ objObj' F (X ⊗ Y)
```

**lemma μ_tmul_tprod** [source]

```
lemma μ_tmul_tprod {X Y : OverColor C} (p : (i : X.left) → F.obj (Discrete.mk <| X.hom i))
    (q : (i : Y.left) → (F.obj <| Discrete.mk (Y.hom i))) :
    (μ F X Y).hom.hom (PiTensorProduct.tprod k p ⊗ₜ[k] PiTensorProduct.tprod k q) =
    (PiTensorProduct.tprod k) fun i =>
    discreteSumEquiv F i (HepLean.PiTensorProduct.elimPureTensor p q i)
```

**lemma μ_tmul_tprod_mk** [source]

```
lemma μ_tmul_tprod_mk {X Y : Type} {cX : X → C} {cY : Y → C}
    (p : (i : X) → F.obj (Discrete.mk <| cX i))
    (q : (i : Y) → (F.obj <| Discrete.mk (cY i))) :
    (μ F (OverColor.mk cX) (OverColor.mk cY)).hom.hom
    (PiTensorProduct.tprod k p ⊗ₜ[k] PiTensorProduct.tprod k q)
    = (PiTensorProduct.tprod k) fun i =>
    discreteSumEquiv' F i (HepLean.PiTensorProduct.elimPureTensor p q i)
```

**lemma μ_natural_left** [source]

```
lemma μ_natural_left {X Y : OverColor C} (f : X ⟶ Y) (Z : OverColor C) :
    MonoidalCategory.whiskerRight (objMap' F f) (objObj' F Z) ≫ (μ F Y Z).hom =
    (μ F X Z).hom ≫ objMap' F (MonoidalCategory.whiskerRight f Z)
```

**lemma μ_natural_right** [source]

```
lemma μ_natural_right {X Y : OverColor C} (X' : OverColor C) (f : X ⟶ Y) :
    MonoidalCategory.whiskerLeft (objObj' F X') (objMap' F f) ≫ (μ F X' Y).hom =
    (μ F X' X).hom ≫ objMap' F (MonoidalCategory.whiskerLeft X' f)
```

**lemma associativity** [source]

```
lemma associativity (X Y Z : OverColor C) :
    whiskerRight (μ F X Y).hom (objObj' F Z) ≫
    (μ F (X ⊗ Y) Z).hom ≫ objMap' F (associator X Y Z).hom =
    (associator (objObj' F X) (objObj' F Y) (objObj' F Z)).hom ≫
    whiskerLeft (objObj' F X) (μ F Y Z).hom ≫ (μ F X (Y ⊗ Z)).hom
```

**lemma left_unitality** [source]

```
lemma left_unitality (X : OverColor C) : (leftUnitor (objObj' F X)).hom =
    whiskerRight (ε F).hom (objObj' F X) ≫
    (μ F (𝟙_ (OverColor C)) X).hom ≫ objMap' F (leftUnitor X).hom
```

**lemma right_unitality** [source]

```
lemma right_unitality (X : OverColor C) : (rightUnitor (objObj' F X)).hom =
    whiskerLeft (objObj' F X) (ε F).hom ≫
    (μ F X (𝟙_ (OverColor C))).hom ≫ objMap' F (rightUnitor X).hom
```

**lemma braided'** [source]

```
lemma braided' (X Y : OverColor C) : (μ F X Y).hom ≫ (objMap' F) (β_ X Y).hom =
    (β_ (objObj' F X) (objObj' F Y)).hom ≫ (μ F Y X).hom
```

**lemma braided** [source]

```
lemma braided (X Y : OverColor C) :
    (objMap' F) (β_ X Y).hom = (μ F X Y).inv ≫
    ((β_ (objObj' F X) (objObj' F Y)).hom ≫ (μ F Y X).hom)
```

**lemma objMap'_id** [source]

```
lemma objMap'_id (X : OverColor C) : objMap' F (𝟙 X) = 𝟙 _
```

**lemma objMap'_comp** [source]

```
lemma objMap'_comp {X Y Z : OverColor C} (f : X ⟶ Y) (g : Y ⟶ Z) :
    objMap' F (f ≫ g) = objMap' F f ≫ objMap' F g
```

**def obj'** [source]

*The `BraidedFunctor (OverColor C) (Rep k G)` from a functor `Discrete C ⥤ Rep k G`.*

```
def obj' : BraidedFunctor (OverColor C) (Rep k G) where
```

**def mapApp'** [source]

*The maps for a natural transformation between `obj' F` and `obj' F'`.*

```
def mapApp' (X : OverColor C) : (obj' F).obj X ⟶ (obj' F').obj X where
```

**lemma mapApp'_tprod** [source]

```
lemma mapApp'_tprod (X : OverColor C) (p : (i : X.left) → F.obj (Discrete.mk (X.hom i))) :
    (mapApp' η X).hom (PiTensorProduct.tprod k p) =
    PiTensorProduct.tprod k fun i => (η.app (Discrete.mk (X.hom i))).hom (p i)
```

**lemma mapApp'_naturality** [source]

```
lemma mapApp'_naturality {X Y : OverColor C} (f : X ⟶ Y) :
    (obj' F).map f ≫ mapApp' η Y = mapApp' η X ≫ (obj' F').map f
```

**lemma mapApp'_unit** [source]

```
lemma mapApp'_unit : (obj' F).ε ≫ mapApp' η (𝟙_ (OverColor C)) = (obj' F').ε
```

**lemma mapApp'_tensor** [source]

```
lemma mapApp'_tensor (X Y : OverColor C) :
    (obj' F).μ X Y ≫ mapApp' η (X ⊗ Y) =
    (mapApp' η X ⊗ mapApp' η Y) ≫ (obj' F').μ X Y
```

**def map'** [source]

*Given a natural transformation between `F F' : Discrete C ⥤ Rep k G` the monoidal natural transformation between `obj' F` and `obj' F'`.*

```
def map' : obj' F ⟶ obj' F' where
```

**def lift** [source]

*The functor taking functors in `Discrete C ⥤ Rep k G` to monoidal functors in `BraidedFunctor (OverColor C) (Rep k G)`, built on the PiTensorProduct.*

```
noncomputable def lift : (Discrete C ⥤ Rep k G) ⥤ BraidedFunctor (OverColor C) (Rep k G)
    where
```

**lemma map_tprod** [source]

```
lemma map_tprod (F : Discrete C ⥤ Rep k G) {X Y : OverColor C} (f : X ⟶ Y)
    (p : (i : X.left) → F.obj (Discrete.mk <| X.hom i)) :
    ((lift.obj F).map f).hom (PiTensorProduct.tprod k p) =
    PiTensorProduct.tprod k fun (i : Y.left) => discreteFunctorMapEqIso F
    (OverColor.Hom.toEquiv_comp_inv_apply f i) (p ((OverColor.Hom.toEquiv f).symm i))
```

**lemma obj_μ_tprod_tmul** [source]

```
lemma obj_μ_tprod_tmul (F : Discrete C ⥤ Rep k G) (X Y : OverColor C)
    (p : (i : X.left) → (F.obj (Discrete.mk <| X.hom i)))
    (q : (i : Y.left) → F.obj (Discrete.mk <| Y.hom i)) :
    ((lift.obj F).μ X Y).hom (PiTensorProduct.tprod k p ⊗ₜ[k] PiTensorProduct.tprod k q) =
    (PiTensorProduct.tprod k) fun i =>
    discreteSumEquiv F i (HepLean.PiTensorProduct.elimPureTensor p q i)
```

**lemma μIso_inv_tprod** [source]

```
lemma μIso_inv_tprod (F : Discrete C ⥤ Rep k G) (X Y : OverColor C)
    (p : (i : (X ⊗ Y).left) → F.obj (Discrete.mk <| (X ⊗ Y).hom i)) :
    ((lift.obj F).μIso X Y).inv.hom (PiTensorProduct.tprod k p) =
    (PiTensorProduct.tprod k (fun i => p (Sum.inl i))) ⊗ₜ[k]
    (PiTensorProduct.tprod k (fun i => p (Sum.inr i)))
```

**lemma inv_μ** [source]

```
@[simp]
lemma inv_μ (X Y : OverColor C) : inv ((lift.obj F).μ X Y) =
    (lift.μ F X Y).inv
```

**def incl** [source]

*The natural inclusion of `Discrete C` into `OverColor C`.*

```
def incl : Discrete C ⥤ OverColor C
```

**def forget** [source]

*The forgetful map from `BraidedFunctor (OverColor C) (Rep k G)` to `Discrete C ⥤ Rep k G` built on the inclusion `incl` and forgetting the monoidal structure.*

```
def forget : BraidedFunctor (OverColor C) (Rep k G) ⥤ (Discrete C ⥤ Rep k G) where
```

**def forgetLiftAppV** [source]

*The `forgetLiftAppV` function takes an object `c` of type `C` and returns a linear equivalence between the vector space obtained by applying the lift of `F` and that obtained by applying `F`. -*

```
def forgetLiftAppV (c : C) : ((lift.obj F).obj (OverColor.mk (fun (_ : Fin 1) => c))).V ≃ₗ[k]
    (F.obj (Discrete.mk c)).V
```

**lemma forgetLiftAppV_symm_apply** [source]

```
@[simp]
lemma forgetLiftAppV_symm_apply (c : C) (x : (F.obj (Discrete.mk c)).V) :
    (forgetLiftAppV F c).symm x = PiTensorProduct.tprod k (fun _ => x)
```

**def forgetLiftApp** [source]

*The `forgetLiftAppV` function takes an object `c` of type `C` and returns a isomorphism between the objects obtained by applying the lift of `F` and that obtained by applying `F`.*

```
def forgetLiftApp (c : C) : (lift.obj F).obj (OverColor.mk (fun (_ : Fin 1) => c))
    ≅ F.obj (Discrete.mk c)
```

**lemma forgetLiftApp_hom_hom_apply_eq** [source]

```
lemma forgetLiftApp_hom_hom_apply_eq (c : C)
    (x : (lift.obj F).obj (OverColor.mk (fun (_ : Fin 1) => c)))
    (y : (F.obj (Discrete.mk c)).V) :
    (forgetLiftApp F c).hom.hom x = y ↔ x = PiTensorProduct.tprod k (fun _ => y)
```

**informal_definition forgetLift** [source]

```
informal_definition forgetLift where
  math :≈ "The natural isomorphism between `lift (C := C) ⋙ forget` and
    `Functor.id (Discrete C ⥤ Rep k G)`."
  deps :≈ [``forget, ``lift]
```

## 2.41 HepLean.Tensors.Tree.Basic

HepLean/Tensors/Tree/Basic.lean on GitHub.

**structure TensorSpecies** [source]

*The sturcture of a type of tensors e.g. Lorentz tensors, Einstien tensors, complex Lorentz tensors.*

```
structure TensorSpecies where
  C : Type
  G : Type
  G_group : Group G
  k : Type
  k_commRing : CommRing k
  FDiscrete : Discrete C ⥤ Rep k G
  τ : C → C
  τ_involution : Function.Involutive τ
  contr : OverColor.Discrete.pairτ FDiscrete τ ⟶ 𝟙_ (Discrete C ⥤ Rep k G)
  metric : 𝟙_ (Discrete C ⥤ Rep k G) ⟶ OverColor.Discrete.pair FDiscrete
  unit : 𝟙_ (Discrete C ⥤ Rep k G) ⟶ OverColor.Discrete.τPair FDiscrete τ
  repDim : C → ℕ
  repDim_neZero (c : C) : NeZero (repDim c)
  basis : (c : C) → Basis (Fin (repDim c)) k (FDiscrete.obj (Discrete.mk c)).V
  contr_tmul_symm (c : C) (x : FDiscrete.obj (Discrete.mk c))
      (y : FDiscrete.obj (Discrete.mk (τ c))) :
    (contr.app (Discrete.mk c)).hom (x ⊗ₜ[k] y) = (contr.app (Discrete.mk (τ c))).hom
      (y ⊗ₜ (FDiscrete.map (Discrete.eqToHom (τ_involution c).symm)).hom x)
  contr_unit (c : C) (x : FDiscrete.obj (Discrete.mk (c))) :
    (λ_ (FDiscrete.obj (Discrete.mk (c)))).hom.hom
    (((contr.app (Discrete.mk c)) ▷ (FDiscrete.obj (Discrete.mk (c)))).hom
    ((α_ _ _ (FDiscrete.obj (Discrete.mk (c)))).inv.hom
    (x ⊗ₜ[k] (unit.app (Discrete.mk c)).hom (1 : k)))) = x
  unit_symm (c : C) :
```

```
    ((unit.app (Discrete.mk c)).hom (1 : k)) =
    ((FDiscrete.obj (Discrete.mk (τ (c)))) ◁
      (FDiscrete.map (Discrete.eqToHom (τ_involution c)))).hom
    ((β_ (FDiscrete.obj (Discrete.mk (τ (τ c)))) (FDiscrete.obj (Discrete.mk (τ (c))))).hom.hom
    ((unit.app (Discrete.mk (τ c))).hom (1 : k)))
  contr_metric (c : C) :
    (β_ (FDiscrete.obj (Discrete.mk c)) (FDiscrete.obj (Discrete.mk (τ c)))).hom.hom
    (((FDiscrete.obj (Discrete.mk c)) ◁ (λ_ (FDiscrete.obj (Discrete.mk (τ c))))).hom).hom
    (((FDiscrete.obj (Discrete.mk c)) ◁ ((contr.app (Discrete.mk c)) ▷
    (FDiscrete.obj (Discrete.mk (τ c))))).hom
    (((FDiscrete.obj (Discrete.mk c)) ◁ (α_ (FDiscrete.obj (Discrete.mk (c)))
      (FDiscrete.obj (Discrete.mk (τ c))) (FDiscrete.obj (Discrete.mk (τ c)))).inv).hom
    ((α_ (FDiscrete.obj (Discrete.mk (c))) (FDiscrete.obj (Discrete.mk (c)))
      (FDiscrete.obj (Discrete.mk (τ c)) ⊗ FDiscrete.obj (Discrete.mk (τ c)))).hom.hom
    ((metric.app (Discrete.mk c)).hom (1 : k) ⊗ₜ[k]
```

**instance CommRing S.k** [source]

```
instance : CommRing S.k
```

**instance Group S.G** [source]

```
instance : Group S.G
```

**instance NeZero (S.repDim c)** [source]

```
instance (c : S.C) : NeZero (S.repDim c)
```

**def F** [source]

*The lift of the functor `S.F` to a monoidal functor.*

```
def F : BraidedFunctor (OverColor S.C) (Rep S.k S.G)
```

**lemma F_def** [source]

```
lemma F_def : F S = (OverColor.lift).obj S.FDiscrete
```

**lemma perm_contr_cond** [source]

```
lemma perm_contr_cond {n : ℕ} {c : Fin n.succ.succ → S.C} {c1 : Fin n.succ.succ → S.C}
    {i : Fin n.succ.succ} {j : Fin n.succ}
    (h : c1 (i.succAbove j) = S.τ (c1 i)) (σ : (OverColor.mk c) ⟶ (OverColor.mk c1)) :
    c (Fin.succAbove ((Hom.toEquiv σ).symm i) ((Hom.toEquiv (extractOne i σ)).symm j)) =
    S.τ (c ((Hom.toEquiv σ).symm i))
```

**def contrFin1Fin1** [source]

*The isomorphism between the image of a map `Fin 1 ⊕ Fin 1 → S.C` contructed by `finExtractTwo` under `S.F.obj`, and an object in the image of `OverColor.Discrete.pairτ S.FDiscrete`.*

```
def contrFin1Fin1 {n : ℕ} (c : Fin n.succ.succ → S.C)
    (i : Fin n.succ.succ) (j : Fin n.succ) (h : c (i.succAbove j) = S.τ (c i)) :
    S.F.obj (OverColor.mk ((c ∘ ⇑(HepLean.Fin.finExtractTwo i j).symm) ∘ Sum.inl)) ≅
    (OverColor.Discrete.pairτ S.FDiscrete S.τ).obj { as := c i }
```

**lemma contrFin1Fin1_inv_tmul** [source]

```
lemma contrFin1Fin1_inv_tmul {n : ℕ} (c : Fin n.succ.succ → S.C)
    (i : Fin n.succ.succ) (j : Fin n.succ) (h : c (i.succAbove j) = S.τ (c i))
    (x : S.FDiscrete.obj { as := c i })
    (y : S.FDiscrete.obj { as := S.τ (c i) }) :
    (S.contrFin1Fin1 c i j h).inv.hom (x ⊗ₜ[S.k] y) =
    PiTensorProduct.tprod S.k (fun k =>
    match k with | Sum.inl 0 => x | Sum.inr 0 => (S.FDiscrete.map
    (eqToHom (by simp [h]))).hom y)
```

**lemma contrFin1Fin1_hom_hom_tprod** [source]

```
lemma contrFin1Fin1_hom_hom_tprod {n : ℕ} (c : Fin n.succ.succ → S.C)
    (i : Fin n.succ.succ) (j : Fin n.succ) (h : c (i.succAbove j) = S.τ (c i))
    (x : (k : Fin 1 ⊕ Fin 1) → (S.FDiscrete.obj
      { as := (OverColor.mk ((c ∘ ⇑(HepLean.Fin.finExtractTwo i j).symm) ∘ Sum.inl)).hom k }))
    :
    (S.contrFin1Fin1 c i j h).hom.hom (PiTensorProduct.tprod S.k x) =
    x (Sum.inl 0) ⊗ₜ[S.k] ((S.FDiscrete.map (eqToHom (by simp [h]))).hom (x (Sum.inr 0)))
```

**def contrIso** [source]

*The isomorphism of objects in `Rep S.k S.G` given an `i` in `Fin n.succ.succ` and a `j` in `Fin n.succ` allowing us to undertake contraction.*

```
def contrIso {n : ℕ} (c : Fin n.succ.succ → S.C)
    (i : Fin n.succ.succ) (j : Fin n.succ) (h : c (i.succAbove j) = S.τ (c i)) :
    S.F.obj (OverColor.mk c) ≅ ((OverColor.Discrete.pairτ S.FDiscrete S.τ).obj
      (Discrete.mk (c i))) ⊗
      (OverColor.lift.obj S.FDiscrete).obj (OverColor.mk (c ∘ i.succAbove ∘ j.succAbove))
```

**lemma contrIso_hom_hom** [source]

```
lemma contrIso_hom_hom {n : ℕ} {c1 : Fin n.succ.succ → S.C}
    {i : Fin n.succ.succ} {j : Fin n.succ} {h : c1 (i.succAbove j) = S.τ (c1 i)} :
    (S.contrIso c1 i j h).hom.hom =
    (S.F.map (equivToIso (HepLean.Fin.finExtractTwo i j)).hom).hom ≫
    (S.F.map (mkSum (c1 ∘ ⇑(HepLean.Fin.finExtractTwo i j).symm)).hom).hom ≫
    (S.F.μIso (OverColor.mk ((c1 ∘ ⇑(HepLean.Fin.finExtractTwo i j).symm) ∘ Sum.inl))
    (OverColor.mk ((c1 ∘ ⇑(HepLean.Fin.finExtractTwo i j).symm) ∘ Sum.inr))).inv.hom ≫
    ((S.contrFin1Fin1 c1 i j h).hom.hom ⊗
    (S.F.map (mkIso (contrIso.proof_1 S c1 i j)).hom).hom)
```

**def TensorSpecies.contrMap** [source]

*`contrMap` is a function that takes a natural number `n`, a function `c` from `Fin n.succ.succ` to `S.C`, an index `i` of type `Fin n.succ.succ`, an index `j` of type `Fin n.succ`, and a*

*proof `h` that `c (i.succAbove j) = S.τ (c i)`. It returns a morphism corresponding to the contraction of the `i`th index with the `i.succAbove j` index. -*

```
def contrMap {n : ℕ} (c : Fin n.succ.succ → S.C)
    (i : Fin n.succ.succ) (j : Fin n.succ) (h : c (i.succAbove j) = S.τ (c i)) :
    S.F.obj (OverColor.mk c) ⟶
    S.F.obj (OverColor.mk (c ∘ i.succAbove ∘ j.succAbove))
```

**def castToField** [source]

*Casts an element of the monoidal unit of `Rep S.k S.G` to the field `S.k`.*

```
def castToField (v : (↑((𝟙_ (Discrete S.C ⥤ Rep S.k S.G)).obj { as := c }).V)) : S.k
```

**def castFin0ToField** [source]

*Casts an element of `(S.F.obj (OverColor.mk c)).V` for `c` a map from `Fin 0` to an element of the field.*

```
def castFin0ToField {c : Fin 0 → S.C} : (S.F.obj (OverColor.mk c)).V →ₗ[S.k] S.k
```

**lemma castFin0ToField_tprod** [source]

```
lemma castFin0ToField_tprod {c : Fin 0 → S.C}
    (x : (i : Fin 0) → S.FDiscrete.obj (Discrete.mk (c i))) :
    castFin0ToField S (PiTensorProduct.tprod S.k x) = 1
```

**lemma contrMap_tprod** [source]

```
lemma contrMap_tprod {n : ℕ} (c : Fin n.succ.succ → S.C)
    (i : Fin n.succ.succ) (j : Fin n.succ) (h : c (i.succAbove j) = S.τ (c i))
    (x : (i : Fin n.succ.succ) → S.FDiscrete.obj (Discrete.mk (c i))) :
    (S.contrMap c i j h).hom (PiTensorProduct.tprod S.k x) =
    (S.castToField ((S.contr.app (Discrete.mk (c i))).hom ((x i) ⊗ₜ[S.k]
    (S.FDiscrete.map (Discrete.eqToHom h)).hom (x (i.succAbove j))))) : S.k)
    • (PiTensorProduct.tprod S.k (fun k => x (i.succAbove (j.succAbove k))) :
    S.F.obj (OverColor.mk (c ∘ i.succAbove ∘ j.succAbove)))
```

**def evalIso** [source]

*The isomorphism of objects in `Rep S.k S.G` given an `i` in `Fin n.succ` allowing us to undertake evaluation.*

```
def evalIso {n : ℕ} (c : Fin n.succ → S.C)
    (i : Fin n.succ) : S.F.obj (OverColor.mk c) ≅ (S.FDiscrete.obj (Discrete.mk (c i))) ⊗
      (OverColor.lift.obj S.FDiscrete).obj (OverColor.mk (c ∘ i.succAbove))
```

**lemma evalIso_tprod** [source]

```
lemma evalIso_tprod {n : ℕ} {c : Fin n.succ → S.C} (i : Fin n.succ)
    (x : (i : Fin n.succ) → S.FDiscrete.obj (Discrete.mk (c i))) :
    (S.evalIso c i).hom.hom (PiTensorProduct.tprod S.k x) =
    x i ⊗ₜ[S.k] (PiTensorProduct.tprod S.k (fun k => x (i.succAbove k)))
```

**def evalLinearMap** [source]

*The linear map giving the coordinate of a vector with respect to the given basis. Important Note: This is not a morphism in the category of representations. In general, it cannot be lifted thereto.*

```
def evalLinearMap {n : ℕ} {c : Fin n.succ → S.C} (i : Fin n.succ) (e : Fin (S.repDim (c i))) :
    S.FDiscrete.obj { as := c i } →ₗ[S.k] S.k where
```

**def evalMap** [source]

*The evaluation map, used to evaluate indices of tensors. Important Note: The evaluation map is in general, not equivariant with respect to group actions. It is a morphism in the underlying module category, not the category of representations.*

```
def evalMap {n : ℕ} {c : Fin n.succ → S.C} (i : Fin n.succ) (e : Fin (S.repDim (c i))) :
    (S.F.obj (OverColor.mk c)).V ⟶ (S.F.obj (OverColor.mk (c ∘ i.succAbove))).V
```

**lemma evalMap_tprod** [source]

```
lemma evalMap_tprod {n : ℕ} {c : Fin n.succ → S.C} (i : Fin n.succ) (e : Fin (S.repDim (c i)))
    (x : (i : Fin n.succ) → S.FDiscrete.obj (Discrete.mk (c i))) :
    (S.evalMap i e) (PiTensorProduct.tprod S.k x) =
    (((S.basis (c i)).repr (x i) e) : S.k) •
    (PiTensorProduct.tprod S.k
    (fun k => x (i.succAbove k)) : S.F.obj (OverColor.mk (c ∘ i.succAbove)))
```

**inductive TensorTree** [source]

*A syntax tree for tensor expressions.*

```
inductive TensorTree (S : TensorSpecies) : {n : ℕ} → (Fin n → S.C) → Type where
  | tensorNode {n : ℕ} {c : Fin n → S.C} (T : S.F.obj (OverColor.mk c)) : TensorTree S c
  | add {n : ℕ} {c : Fin n → S.C} : TensorTree S c → TensorTree S c → TensorTree S c
  | perm {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C}
      (σ : (OverColor.mk c) ⟶ (OverColor.mk c1)) (t : TensorTree S c) : TensorTree S c1
  | prod {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C}
    (t : TensorTree S c) (t1 : TensorTree S c1) : TensorTree S (Sum.elim c c1 ∘ finSumFinEquiv.
    symm)
  | smul {n : ℕ} {c : Fin n → S.C} : S.k → TensorTree S c → TensorTree S c
  | neg {n : ℕ} {c : Fin n → S.C} : TensorTree S c → TensorTree S c
  | contr {n : ℕ} {c : Fin n.succ.succ → S.C} : (i : Fin n.succ.succ) →
    (j : Fin n.succ) → (h : c (i.succAbove j) = S.τ (c i)) → TensorTree S c →
    TensorTree S (c ∘ Fin.succAbove i ∘ Fin.succAbove j)
  | action {n : ℕ} {c : Fin n → S.C} : S.G → TensorTree S c → TensorTree S c
  | eval {n : ℕ} {c : Fin n.succ → S.C} : (i : Fin n.succ) → (x : ℕ) → TensorTree S c →
    TensorTree S (c ∘ Fin.succAbove i)
```

**def vecNode** [source]

*A node consisting of a single vector.*

```
def vecNode {c : S.C} (v : S.FDiscrete.obj (Discrete.mk c)) : TensorTree S ![c]
```

**abbrev vecNodeE** [source]

*The node `vecNode` of a tensor tree, with all arguments explicit.*

```
abbrev vecNodeE (S : TensorSpecies) (c1 : S.C)
    (v : (S.FDiscrete.obj (Discrete.mk c1)).V) :
    TensorTree S ![c1]
```

**def twoNode** [source]

*A node consisting of a two tensor.*

```
def twoNode {c1 c2 : S.C} (t : (S.FDiscrete.obj (Discrete.mk c1) ⊗
    S.FDiscrete.obj (Discrete.mk c2)).V) :
    TensorTree S ![c1, c2]
```

**abbrev twoNodeE** [source]

*The node `twoNode` of a tensor tree, with all arguments explicit.*

```
abbrev twoNodeE (S : TensorSpecies) (c1 c2 : S.C)
    (v : (S.FDiscrete.obj (Discrete.mk c1) ⊗ S.FDiscrete.obj (Discrete.mk c2)).V) :
    TensorTree S ![c1, c2]
```

**def threeNode** [source]

*A node consisting of a three tensor.*

```
def threeNode {c1 c2 c3 : S.C} (t : (S.FDiscrete.obj (Discrete.mk c1) ⊗
    S.FDiscrete.obj (Discrete.mk c2) ⊗ S.FDiscrete.obj (Discrete.mk c3)).V) :
    TensorTree S ![c1, c2, c3]
```

**abbrev threeNodeE** [source]

*The node `threeNode` of a tensor tree, with all arguments explicit.*

```
abbrev threeNodeE (S : TensorSpecies) (c1 c2 c3 : S.C)
    (v : (S.FDiscrete.obj (Discrete.mk c1) ⊗ S.FDiscrete.obj (Discrete.mk c2) ⊗
    S.FDiscrete.obj (Discrete.mk c3)).V) :
    TensorTree S ![c1, c2, c3]
```

**def constNode** [source]

*A general constant node.*

```
def constNode {n : ℕ} {c : Fin n → S.C} (T : 𝓚_ (Rep S.k S.G) ⟶ S.F.obj (OverColor.mk c)) :
    TensorTree S c
```

**def constVecNode** [source]

*A constant vector.*

```
def constVecNode {c : S.C} (v : 𝓚_ (Rep S.k S.G) ⟶ S.FDiscrete.obj (Discrete.mk c)) :
    TensorTree S ![c]
```

**def constTwoNode** [source]

*A constant two tensor (e.g. metric and unit).*

```
def constTwoNode {c1 c2 : S.C}
    (v : 𝟙_ (Rep S.k S.G) ⟶ S.FDiscrete.obj (Discrete.mk c1) ⊗ S.FDiscrete.obj (Discrete.mk
    c2)) :
    TensorTree S ![c1, c2]
```

**abbrev constTwoNodeE** [source]

*The node `constTwoNode` of a tensor tree, with all arguments explicit.*

```
abbrev constTwoNodeE (S : TensorSpecies) (c1 c2 : S.C)
    (v : 𝟙_ (Rep S.k S.G) ⟶ S.FDiscrete.obj (Discrete.mk c1) ⊗ S.FDiscrete.obj (Discrete.mk
    c2)) :
    TensorTree S ![c1, c2]
```

**def constThreeNode** [source]

*A constant three tensor (e.g. Pauli matrices).*

```
def constThreeNode {c1 c2 c3 : S.C}
    (v : 𝟙_ (Rep S.k S.G) ⟶ S.FDiscrete.obj (Discrete.mk c1) ⊗ S.FDiscrete.obj (Discrete.mk
    c2) ⊗
    S.FDiscrete.obj (Discrete.mk c3)) : TensorTree S ![c1, c2, c3]
```

**abbrev constThreeNodeE** [source]

*The node `constThreeNode` of a tensor tree, with all arguments explicit.*

```
abbrev constThreeNodeE (S : TensorSpecies) (c1 c2 c3 : S.C)
    (v : 𝟙_ (Rep S.k S.G) ⟶ S.FDiscrete.obj (Discrete.mk c1) ⊗ S.FDiscrete.obj (Discrete.mk
    c2) ⊗
    S.FDiscrete.obj (Discrete.mk c3)) : TensorTree S ![c1, c2, c3]
```

**def size** [source]

*The number of nodes in a tensor tree.*

```
def size : ∀ {n : ℕ} {c : Fin n → S.C}, TensorTree S c → ℕ
```

**def tensor** [source]

*The underlying tensor a tensor tree corresponds to.*

```
def tensor : ∀ {n : ℕ} {c : Fin n → S.C}, TensorTree S c → S.F.obj (OverColor.mk c)
```

**def field** [source]

*Takes a tensor tree based on `Fin 0`, into the field `S.k`.*

```
def field {c : Fin 0 → S.C} (t : TensorTree S c) : S.k
```

**lemma tensorNode_tensor** [source]

```
@[simp]
lemma tensorNode_tensor {c : Fin n → S.C} (T : S.F.obj (OverColor.mk c)) :
    (tensorNode T).tensor = T
```

**lemma constTwoNode_tensor** [source]

```
@[simp]
lemma constTwoNode_tensor {c1 c2 : S.C}
    (v : 𝟙_ (Rep S.k S.G) ⟶ S.FDiscrete.obj (Discrete.mk c1) ⊗ S.FDiscrete.obj (Discrete.mk
    c2)) :
    (constTwoNode v).tensor =
    (OverColor.Discrete.pairIsoSep S.FDiscrete).hom.hom (v.hom (1 : S.k))
```

**lemma constThreeNode_tensor** [source]

```
@[simp]
lemma constThreeNode_tensor {c1 c2 c3 : S.C}
    (v : 𝟙_ (Rep S.k S.G) ⟶ S.FDiscrete.obj (Discrete.mk c1) ⊗ S.FDiscrete.obj (Discrete.mk
    c2) ⊗
    S.FDiscrete.obj (Discrete.mk c3)) :
    (constThreeNode v).tensor =
    (OverColor.Discrete.tripleIsoSep S.FDiscrete).hom.hom (v.hom (1 : S.k))
```

**lemma prod_tensor** [source]

```
lemma prod_tensor {c1 : Fin n → S.C} {c2 : Fin m → S.C} (t1 : TensorTree S c1)
    (t2 : TensorTree S c2) :
    (prod t1 t2).tensor = (S.F.map (OverColor.equivToIso finSumFinEquiv).hom).hom
    ((S.F.μ _ _).hom (t1.tensor ⊗ₜ t2.tensor))
```

**lemma add_tensor** [source]

```
lemma add_tensor (t1 t2 : TensorTree S c) : (add t1 t2).tensor = t1.tensor + t2.tensor
```

**lemma perm_tensor** [source]

```
lemma perm_tensor (σ : (OverColor.mk c) ⟶ (OverColor.mk c1)) (t : TensorTree S c) :
    (perm σ t).tensor = (S.F.map σ).hom t.tensor
```

**lemma contr_tensor** [source]

```
lemma contr_tensor {n : ℕ} {c : Fin n.succ.succ → S.C} {i : Fin n.succ.succ} {j : Fin n.succ}
    {h : c (i.succAbove j) = S.τ (c i)} (t : TensorTree S c) :
    (contr i j h t).tensor = (S.contrMap c i j h).hom t.tensor
```

**lemma neg_tensor** [source]

```
lemma neg_tensor (t : TensorTree S c) : (neg t).tensor = - t.tensor
```

**lemma eval_tensor** [source]

```
lemma eval_tensor {n : ℕ} {c : Fin n.succ → S.C} (i : Fin n.succ) (e : ℕ) (t : TensorTree S c)
    :
    (eval i e t).tensor = (S.evalMap i (Fin.ofNat' e Fin.size_pos')) t.tensor
```

**lemma smul_tensor** [source]

```
lemma smul_tensor {c : Fin n → S.C} (a : S.k) (T : TensorTree S c) :
    (smul a T).tensor = a • T.tensor
```

**lemma action_tensor** [source]

```
lemma action_tensor {c : Fin n → S.C} (g : S.G) (T : TensorTree S c) :
    (action g T).tensor = (S.F.obj (OverColor.mk c)).ρ g T.tensor
```

**lemma contr_tensor_eq** [source]

```
lemma contr_tensor_eq {n : ℕ} {c : Fin n.succ.succ → S.C} {T1 T2 : TensorTree S c}
    (h : T1.tensor = T2.tensor) {i : Fin n.succ.succ} {j : Fin n.succ}
    {h' : c (i.succAbove j) = S.τ (c i)} :
    (contr i j h' T1).tensor = (contr i j h' T2).tensor
```

**lemma prod_tensor_eq_fst** [source]

```
lemma prod_tensor_eq_fst {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C}
    {T1 T1' : TensorTree S c} { T2 : TensorTree S c1}
    (h : T1.tensor = T1'.tensor) :
    (prod T1 T2).tensor = (prod T1' T2).tensor
```

**lemma prod_tensor_eq_snd** [source]

```
lemma prod_tensor_eq_snd {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C}
    {T1 : TensorTree S c} {T2 T2' : TensorTree S c1}
    (h : T2.tensor = T2'.tensor) :
    (prod T1 T2).tensor = (prod T1 T2').tensor
```

**lemma perm_tensor_eq** [source]

```
lemma perm_tensor_eq {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C}
    {σ : (OverColor.mk c) ⟶ (OverColor.mk c1)} {T1 T2 : TensorTree S c}
    (h : T1.tensor = T2.tensor) :
    (perm σ T1).tensor = (perm σ T2).tensor
```

**lemma add_tensor_eq_fst** [source]

```
lemma add_tensor_eq_fst {T1 T1' T2 : TensorTree S c} (h : T1.tensor = T1'.tensor) :
    (add T1 T2).tensor = (add T1' T2).tensor
```

**lemma add_tensor_eq_snd** [source]

```
lemma add_tensor_eq_snd {T1 T2 T2' : TensorTree S c} (h : T2.tensor = T2'.tensor) :
    (add T1 T2).tensor = (add T1 T2').tensor
```

**lemma add_tensor_eq** [source]

```
lemma add_tensor_eq {T1 T1' T2 T2' : TensorTree S c} (h1 : T1.tensor = T1'.tensor)
    (h2 : T2.tensor = T2'.tensor) :
    (add T1 T2).tensor = (add T1' T2').tensor
```

**lemma neg_tensor_eq** [source]

```
lemma neg_tensor_eq {T1 T2 : TensorTree S c} (h : T1.tensor = T2.tensor) :
    (neg T1).tensor = (neg T2).tensor
```

**lemma smul_tensor_eq** [source]

```
lemma smul_tensor_eq {T1 T2 : TensorTree S c} {a : S.k} (h : T1.tensor = T2.tensor) :
    (smul a T1).tensor = (smul a T2).tensor
```

**lemma action_tensor_eq** [source]

```
lemma action_tensor_eq {T1 T2 : TensorTree S c} {g : S.G} (h : T1.tensor = T2.tensor) :
    (action g T1).tensor = (action g T2).tensor
```

**lemma smul_mul_eq** [source]

```
lemma smul_mul_eq {T1 : TensorTree S c} {a b : S.k} (h : a = b) :
    (smul a T1).tensor = (smul b T1).tensor
```

**lemma eq_tensorNode_of_eq_tensor** [source]

```
lemma eq_tensorNode_of_eq_tensor {T1 : TensorTree S c} {t : S.F.obj (OverColor.mk c)}
    (h : T1.tensor = t) : T1.tensor = (tensorNode t).tensor
```

**def zeroTree** [source]

*The zero tensor tree.*

```
def zeroTree {n : ℕ} {c : Fin n → S.C} : TensorTree S c
```

**lemma zeroTree_tensor** [source]

```
@[simp]
lemma zeroTree_tensor {n : ℕ} {c : Fin n → S.C} : (zeroTree (c := c)).tensor = 0
```

**lemma zero_smul** [source]

```
lemma zero_smul {T1 : TensorTree S c} :
    (smul 0 T1).tensor = zeroTree.tensor
```

**lemma smul_zero** [source]

```
lemma smul_zero {a : S.k} : (smul a (zeroTree (c := c))).tensor = zeroTree.tensor
```

**lemma zero_add** [source]

```
lemma zero_add {T1 : TensorTree S c} : (add zeroTree T1).tensor = T1.tensor
```

**lemma add_zero** [source]

```
lemma add_zero {T1 : TensorTree S c} : (add T1 zeroTree).tensor = T1.tensor
```

**lemma perm_zero** [source]

```
lemma perm_zero {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C} (σ : (OverColor.mk c) ⟶
    (OverColor.mk c1)) : (perm σ zeroTree).tensor = zeroTree.tensor
```

**lemma neg_zero** [source]

```
lemma neg_zero : (neg (zeroTree (c := c))).tensor = zeroTree.tensor
```

**lemma contr_zero** [source]

```
lemma contr_zero {n : ℕ} {c : Fin n.succ.succ → S.C} {i : Fin n.succ.succ} {j : Fin n.succ}
    {h : c (i.succAbove j) = S.τ (c i)} : (contr i j h zeroTree).tensor = zeroTree.tensor
```

**lemma zero_prod** [source]

```
lemma zero_prod {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C} (t : TensorTree S c1) :
    (prod (zeroTree (c := c)) t).tensor = zeroTree.tensor
```

**lemma prod_zero** [source]

```
lemma prod_zero {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C} (t : TensorTree S c) :
    (prod t (zeroTree (c := c1))).tensor = zeroTree.tensor
```

**structure ContrPair** [source]

*A structure containing a pair of indices (i, j) to be contracted in a tensor. This is used in some proofs of node identities for tensor trees.*

```
structure ContrPair {n : ℕ} (c : Fin n.succ.succ → S.C) where
  i : Fin n.succ.succ
  j : Fin n.succ
  h : c (i.succAbove j) = S.τ (c i)
```

**lemma ext** [source]

```
lemma ext (hi : q.i = q'.i) (hj : q.j = q'.j) : q = q'
```

**def ContrPair.contrMap** [source]

*The contraction map for a pair of indices.*

```
def contrMap
```

## 2.42 HepLean.Tensors.Tree.Dot

HepLean/Tensors/Tree/Dot.lean on GitHub.

**def dotString** [source]

*Turns a nodes of tensor trees into nodes and edges of a dot file.*

```
def dotString (m : ℕ) (nt : ℕ) : ∀ {n : ℕ} {c : Fin n → S.C}, TensorTree S c → String
```

**def dot** [source]

*Used to form a dot graph from a tensor tree. use e.g.* `#tensor_dot {T4 | i j l d ⊗ T5 | i j k a b}ᵀ.dot`. *Dot files can be viewed by copying and pasting into: https://dreampuf.github.io/GraphvizOnline.*

```
def dot {n : ℕ} {c : Fin n → S.C} (t : TensorTree S c) : String
```

**def dotElab** [source]

*Adapted from* `Lean.Elab.Command.elabReduce` *in file copyright Microsoft Corporation.*

```
unsafe def dotElab (term : Syntax) : CommandElabM Unit
```

**def elabTensorDot** [source]

*Elaborator for the syntax tensorDot.*

```
@[command_elab tensorDot]
unsafe def elabTensorDot: CommandElab
  | `(#tensor_dot $term) => dotElab term
  | _ => throwUnsupportedSyntax
```

## 2.43 HepLean.Tensors.Tree.Elab

HepLean/Tensors/Tree/Elab.lean on GitHub.

**def indexExprIsNum** [source]

*Bool which is ture if an index is a num.*

```
def indexExprIsNum (stx : Syntax) : Bool
```

**def indexToNum** [source]

*If an index is a num - the undelrying natural number.*

```
def indexToNum (stx : Syntax) : TermElabM Nat
```

**def indexToIdent** [source]

*When an index is not a num, the corresponding ident.*

```
def indexToIdent (stx : Syntax) : TermElabM Ident
```

**def indexPosEq** [source]

*Takes a pair `a b : ℕ × TSyntax `indexExpr`. If `a.1 < b.1` and `a.2 = b.2` then outputs `some (a.1, b.1)`, otherwise `none`.*

```
def indexPosEq (a b : ℕ × TSyntax `indexExpr) : TermElabM (Option (ℕ × ℕ))
```

**def indexToDual** [source]

*Bool which is true if an index is of the form $\tau(i)$ that is, to be dualed.*

```
def indexToDual (stx : Syntax) : Bool
```

**def TensorNode.getIndices** [source]

*The indices of a tensor node. Before contraction, and evaluation.*

```
partial def getIndices (stx : Syntax) : TermElabM (List (TSyntax `indexExpr))
```

**def getNoIndicesExact** [source]

*Uses the structure of the tensor to get the number of indices.*

```
def getNoIndicesExact (stx : Syntax) : TermElabM ℕ
```

**def stringToType** [source]

*The construction of an expression corresponding to the type of a given string once parsed.*

```
def stringToType (str : String) : TermElabM (Option Expr)
```

**def stringToTerm** [source]

*The construction of an expression corresponding to the type of a given string once parsed.*

```
def stringToTerm (str : String) : TermElabM Term
```

**def specialTypes** [source]

*Specific types of tensors which appear which we want to elaborate in specific ways.*

```
def specialTypes : List (String × (Term → Term))
```

**def termNodeSyntax** [source]

*The syntax associated with a terminal node of a tensor tree.*

```
def termNodeSyntax (T : Term) : TermElabM Term
```

**def evalAdjustPos** [source]

*Adjusts a list `List ℕ` by subtracting from each natrual number the number of elements before it in the list which are less then itself. This is used to form a list of pairs which can be used for evaluating indices.*

```
def evalAdjustPos (l : List ℕ) : List ℕ
```

**def getEvalPos** [source]

*The positions in getIndicesNode which get evaluated, and the value they take.*

```
partial def getEvalPos (stx : Syntax) : TermElabM (List (ℕ × ℕ))
```

**def evalSyntax** [source]

*For each element of `l : List (ℕ × ℕ)` applies `TensorTree.eval` to the given term.*

```
def evalSyntax (l : List (ℕ × ℕ)) (T : Term) : Term
```

**def TensorNode.getContrPos** [source]

*The pairs of positions in getIndicesNode which get contracted.*

```
partial def getContrPos (stx : Syntax) : TermElabM (List (ℕ × ℕ))
```

**def TensorNode.withoutContr** [source]

*The list of indices after contraction or evaluation.*

```
def withoutContr (ind : List (TSyntax `indexExpr)) : TermElabM (List (TSyntax `indexExpr))
```

**def toPairs** [source]

*Takes a list and puts conseutive elements into pairs. e.g. [0, 1, 2, 3] becomes [(0, 1), (2, 3)].*

```
def toPairs (l : List ℕ) : List (ℕ × ℕ)
```

**def contrListAdjust** [source]

*Adjusts a list `List (ℕ × ℕ)` by subtracting from each natrual number the number of elements before it in the list which are less then itself. This is used to form a list of pairs which can be used for contracting indices.*

```
def contrListAdjust (l : List (ℕ × ℕ)) : List (ℕ × ℕ)
```

**def contrSyntax** [source]

*For each element of `l : List (ℕ × ℕ)` applies `TensorTree.contr` to the given term.*

```
def contrSyntax (l : List (ℕ × ℕ)) (T : Term) : Term
```

**def ProdNode.getIndices** [source]

*Gets the indices associated with a product node.*

```
partial def getIndices (stx : Syntax) : TermElabM (List (TSyntax `indexExpr))
```

**def ProdNode.getContrPos** [source]

*The pairs of positions in getIndicesNode which get contracted.*

```
partial def getContrPos (stx : Syntax) : TermElabM (List (ℕ × ℕ))
```

**def ProdNode.withoutContr** [source]

*The list of indices after contraction.*

```
def withoutContr (stx : Syntax) : TermElabM (List (TSyntax `indexExpr))
```

**def prodSyntax** [source]

*The syntax associated with a product of tensors.*

```
def prodSyntax (T1 T2 : Term) : Term
```

**def getPermutation** [source]

*Given two lists of indices returns the `List (ℕ)` representing the how one list permutes into the other.*

```
def getPermutation (l1 l2 : List (TSyntax `indexExpr)) : TermElabM (List (ℕ))
```

**def finMapToEquiv** [source]

*Takes two maps `Fin n → Fin n` and returns the equivelance they form.*

```
def finMapToEquiv (f1 : Fin n → Fin m) (f2 : Fin m → Fin n)
    (h : ∀ x, f1 (f2 x) = x := by decide)
    (h' : ∀ x, f2 (f1 x) = x := by decide) : Fin n ≃ Fin m where
```

**def getPermutationSyntax** [source]

*Given two lists of indices returns the permutation between them based on `finMapToEquiv`.*

```
def getPermutationSyntax (l1 l2 : List (TSyntax `indexExpr)) : TermElabM Term
```

**def negSyntax** [source]

*The syntax associated with a product of tensors.*

```
def negSyntax (T1 : Term) : Term
```

**def getIndicesFull** [source]

*Returns the full list of indices after contraction. TODO: Include evaluation.*

```
partial def getIndicesFull (stx : Syntax) : TermElabM (List (TSyntax `indexExpr))
```

**def smulSyntax** [source]

*The syntax associated with the scalar multiplication of tensors.*

```
def smulSyntax (c T : Term) : Term
```

**def actionSyntax** [source]

*The syntax associated with the group action of tensors.*

```
def actionSyntax (c T : Term) : Term
```

**def Add.getIndicesLeft** [source]

*Gets the indices associated with the LHS of an addition.*

```
partial def getIndicesLeft (stx : Syntax) : TermElabM (List (TSyntax `indexExpr))
```

**def Add.getIndicesRight** [source]

*Gets the indices associated with the RHS of an addition.*

```
partial def getIndicesRight (stx : Syntax) : TermElabM (List (TSyntax `indexExpr))
```

**def addSyntax** [source]

*The syntax for a equality of tensor trees.*

```
def addSyntax (permSyntax : Term) (T1 T2 : Term) : TermElabM Term
```

**def Equality.getIndicesLeft** [source]

*Gets the indices associated with the LHS of an equality.*

```
partial def getIndicesLeft (stx : Syntax) : TermElabM (List (TSyntax `indexExpr))
```

**def Equality.getIndicesRight** [source]

*Gets the indices associated with the RHS of an equality.*

```
partial def getIndicesRight (stx : Syntax) : TermElabM (List (TSyntax `indexExpr))
```

**def equalSyntax** [source]

*The syntax for a equality of tensor trees.*

```
def equalSyntax (permSyntax : Term) (T1 T2 : Term) : TermElabM Term
```

**def syntaxFull** [source]

*Creates the syntax associated with a tensor node.*

```
partial def syntaxFull (stx : Syntax) : TermElabM Term
```

**def elaborateTensorNode** [source]

*An elaborator for tensor nodes. This is to be generalized.*

```
def elaborateTensorNode (stx : Syntax) : TermElabM Expr
```

## 2.44 HepLean.Tensors.Tree.NodeIdentities.Basic

HepLean/Tensors/Tree/NodeIdentities/Basic.lean on GitHub.

**informal_lemma constVecNode_eq_vecNode** [source]

```
informal_lemma constVecNode_eq_vecNode where
  math :≈ "A constVecNode has equal tensor to the vecNode with the map evaluated at 1."
  deps :≈ [``constVecNode, ``vecNode]
```

**informal_lemma constTwoNode_eq_twoNode** [source]

```
informal_lemma constTwoNode_eq_twoNode where
  math :≈ "A constTwoNode has equal tensor to the twoNode with the map evaluated at 1."
  deps :≈ [``constTwoNode, ``twoNode]
```

**lemma neg_neg** [source]

```
@[simp]
lemma neg_neg (t : TensorTree S c) : (neg (neg t)).tensor = t.tensor
```

**lemma neg_fst_prod** [source]

```
@[simp]
lemma neg_fst_prod {c1 : Fin n → S.C} {c2 : Fin m → S.C} (T1 : TensorTree S c1)
    (T2 : TensorTree S c2) :
    (prod (neg T1) T2).tensor = (neg (prod T1 T2)).tensor
```

**lemma neg_snd_prod** [source]

```
@[simp]
lemma neg_snd_prod {c1 : Fin n → S.C} {c2 : Fin m → S.C} (T1 : TensorTree S c1)
    (T2 : TensorTree S c2) :
    (prod T1 (neg T2)).tensor = (neg (prod T1 T2)).tensor
```

**lemma neg_contr** [source]

```
@[simp]
lemma neg_contr {n : ℕ} {c : Fin n.succ.succ → S.C} {i : Fin n.succ.succ} {j : Fin n.succ}
    {h : c (i.succAbove j) = S.τ (c i)}
    (t : TensorTree S c) : (contr i j h (neg t)).tensor = (neg (contr i j h t)).tensor
```

**lemma neg_perm** [source]

```
lemma neg_perm {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C}
    (σ : (OverColor.mk c) ⟶ (OverColor.mk c1)) (t : TensorTree S c) :
    (perm σ (neg t)).tensor = (neg (perm σ t)).tensor
```

**lemma neg_add** [source]

```
@[simp]
lemma neg_add (t : TensorTree S c) : (add (neg t) t).tensor = 0
```

**lemma add_neg** [source]

```
@[simp]
lemma add_neg (t : TensorTree S c) : (add t (neg t)).tensor = 0
```

**lemma perm_perm** [source]

*Applying two permutations is the same as applying the transitive permutation.*

```
lemma perm_perm {n n1 n2 : ℕ} {c : Fin n → S.C} {c1 : Fin n1 → S.C} {c2 : Fin n2 → S.C}
    (σ : (OverColor.mk c) ⟶ (OverColor.mk c1)) (σ2 : (OverColor.mk c1) ⟶ (OverColor.mk c2)
    )
    (t : TensorTree S c) : (perm σ2 (perm σ t)).tensor = (perm (σ ≫ σ2) t).tensor
```

**lemma perm_id** [source]

*Applying the identity permutation is the same as not applying a permutation.*

```
lemma perm_id (t : TensorTree S c) : (perm (𝟙 (OverColor.mk c)) t).tensor = t.tensor
```

**lemma perm_eq_id** [source]

*Applying a permutation which is equal to the identity permutation is the same as not applying a permutation.*

```
lemma perm_eq_id {n : ℕ} {c : Fin n → S.C} (σ : (OverColor.mk c) ⟶ (OverColor.mk c))
    (h : σ = 𝟙 _) (t : TensorTree S c) : (perm σ t).tensor = t.tensor
```

**lemma perm_eq_of_eq_perm** [source]

```
lemma perm_eq_of_eq_perm {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C}
    (σ : (OverColor.mk c) ≅ (OverColor.mk c1))
    {t : TensorTree S c} {t2 : TensorTree S c1} (h : (perm σ.hom t).tensor = t2.tensor) :
    t.tensor = (perm σ.inv t2).tensor
```

**lemma perm_eq_iff_eq_perm** [source]

```
lemma perm_eq_iff_eq_perm {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C}
    (σ : (OverColor.mk c) ⟶ (OverColor.mk c1))
    {t : TensorTree S c} {t2 : TensorTree S c1} :
    (perm σ t).tensor = t2.tensor ↔ t.tensor =
    (perm (equivToHomEq (Hom.toEquiv σ).symm (fun x => Hom.toEquiv_comp_apply σ x)) t2).tensor
```

**lemma smul_smul** [source]

```
lemma smul_smul (t : TensorTree S c) (a b : S.k) :
    (smul a (smul b t)).tensor = (smul (a * b) t).tensor
```

**lemma smul_one** [source]

```
lemma smul_one (t : TensorTree S c) :
    (smul 1 t).tensor = t.tensor
```

**lemma smul_eq_one** [source]

```
lemma smul_eq_one (t : TensorTree S c) (a : S.k) (h : a = 1) :
    (smul a t).tensor = t.tensor
```

**lemma add_comm** [source]

*The addition node is commutative.*

```
lemma add_comm (t1 t2 : TensorTree S c) : (add t1 t2).tensor = (add t2 t1).tensor
```

**lemma add_assoc** [source]

*The addition node is associative.*

```
lemma add_assoc (t1 t2 t3 : TensorTree S c) :
    (add (add t1 t2) t3).tensor = (add t1 (add t2 t3)).tensor
```

**lemma add_perm** [source]

*When the same permutation acts on both arguments of an addition, the permutation can be moved out of the addition.*

```
lemma add_perm {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C}
    (σ : (OverColor.mk c) ⟶ (OverColor.mk c1)) (t t1 : TensorTree S c) :
    (add (perm σ t) (perm σ t1)).tensor = (perm σ (add t t1)).tensor
```

**lemma perm_add** [source]

```
lemma perm_add {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C}
    (σ : (OverColor.mk c) ⟶ (OverColor.mk c1)) (t t1 : TensorTree S c) :
    (perm σ (add t t1)).tensor = (add (perm σ t) (perm σ t1)).tensor
```

**lemma perm_smul** [source]

```
lemma perm_smul {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C}
    (σ : (OverColor.mk c) ⟶ (OverColor.mk c1)) (a : S.k) (t : TensorTree S c) :
    (perm σ (smul a t)).tensor = (smul a (perm σ t)).tensor
```

**lemma add_eval** [source]

*When the same evaluation acts on both arguments of an addition, the evaluation can be moved out of the addition.*

```
lemma add_eval {n : ℕ} {c : Fin n.succ → S.C} (i : Fin n.succ) (e : ℕ) (t t1 : TensorTree S c)
    :
    (add (eval i e t) (eval i e t1)).tensor = (eval i e (add t t1)).tensor
```

**lemma contr_add** [source]

```
lemma contr_add {n : ℕ} {c : Fin n.succ.succ → S.C} {i : Fin n.succ.succ} {j : Fin n.succ}
    {h : c (i.succAbove j) = S.τ (c i)} (t1 t2 : TensorTree S c) :
  (contr i j h (add t1 t2)).tensor = (add (contr i j h t1) (contr i j h t2)).tensor
```

**lemma contr_smul** [source]

```
lemma contr_smul {n : ℕ} {c : Fin n.succ.succ → S.C} {i : Fin n.succ.succ} {j : Fin n.succ}
    {h : c (i.succAbove j) = S.τ (c i)} (a : S.k) (t : TensorTree S c) :
    (contr i j h (smul a t)).tensor = (smul a (contr i j h t)).tensor
```

**lemma add_prod** [source]

```
@[simp]
lemma add_prod {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C}
    (t1 t2 : TensorTree S c) (t3 : TensorTree S c1) :
    (prod (add t1 t2) t3).tensor = (add (prod t1 t3) (prod t2 t3)).tensor
```

**lemma prod_add** [source]

```
@[simp]
lemma prod_add {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C}
    (t1 : TensorTree S c) (t2 t3 : TensorTree S c1) :
    (prod t1 (add t2 t3)).tensor = (add (prod t1 t2) (prod t1 t3)).tensor
```

**lemma smul_prod** [source]

```
lemma smul_prod {n m: ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C}
    (a : S.k) (t1 : TensorTree S c) (t2 : TensorTree S c1) :
    ((prod (smul a t1) t2)).tensor = (smul a (prod t1 t2)).tensor
```

**lemma prod_smul** [source]

```
lemma prod_smul {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C}
    (a : S.k) (t1 : TensorTree S c) (t2 : TensorTree S c1) :
    (prod t1 (smul a t2)).tensor = (smul a (prod t1 t2)).tensor
```

**lemma prod_add_both** [source]

```
lemma prod_add_both {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C}
    (t1 t2 : TensorTree S c) (t3 t4 : TensorTree S c1) :
    (prod (add t1 t2) (add t3 t4)).tensor = (add (add (prod t1 t3) (prod t1 t4))
    (add (prod t2 t3) (prod t2 t4))).tensor
```

**lemma smul_action** [source]

```
lemma smul_action {n : ℕ} {c : Fin n → S.C} (g : S.G) (a : S.k) (t : TensorTree S c) :
    (smul a (action g t)).tensor = (action g (smul a t)).tensor
```

**lemma contr_action** [source]

```
lemma contr_action {n : ℕ} {c : Fin n.succ.succ → S.C} {i : Fin n.succ.succ} {j : Fin n.succ}
    {h : c (i.succAbove j) = S.τ (c i)} (g : S.G) (t : TensorTree S c) :
    (contr i j h (action g t)).tensor = (action g (contr i j h t)).tensor
```

**lemma prod_action** [source]

```
lemma prod_action {n n1 : ℕ} {c : Fin n → S.C} {c1 : Fin n1 → S.C} (g : S.G)
    (t : TensorTree S c) (t1 : TensorTree S c1) :
    (prod (action g t) (action g t1)).tensor = (action g (prod t t1)).tensor
```

**lemma add_action** [source]

```
lemma add_action {n : ℕ} {c : Fin n → S.C} (g : S.G) (t t1 : TensorTree S c) :
    (add (action g t) (action g t1)).tensor = (action g (add t t1)).tensor
```

**lemma perm_action** [source]

```
lemma perm_action {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C}
    (σ : (OverColor.mk c) ⟶ (OverColor.mk c1)) (g : S.G) (t : TensorTree S c) :
    (perm σ (action g t)).tensor = (action g (perm σ t)).tensor
```

**lemma neg_action** [source]

```
lemma neg_action {n : ℕ} {c : Fin n → S.C} (g : S.G) (t : TensorTree S c) :
    (neg (action g t)).tensor = (action g (neg t)).tensor
```

**lemma action_action** [source]

```
lemma action_action {n : ℕ} {c : Fin n → S.C} (g h : S.G) (t : TensorTree S c) :
    (action g (action h t)).tensor = (action (g * h) t).tensor
```

**lemma action_id** [source]

```
lemma action_id {n : ℕ} {c : Fin n → S.C} (t : TensorTree S c) :
    (action 1 t).tensor = t.tensor
```

**lemma action_constTwoNode** [source]

```
lemma action_constTwoNode {c1 c2 : S.C}
    (v : 𝕂[_] (Rep S.k S.G) ⟶ S.FDiscrete.obj (Discrete.mk c1) ⊗ S.FDiscrete.obj (Discrete.mk
    c2))
    (g : S.G) : (action g (constTwoNode v)).tensor = (constTwoNode v).tensor
```

**lemma action_constThreeNode** [source]

```
lemma action_constThreeNode {c1 c2 c3 : S.C}
    (v : 𝕂[_] (Rep S.k S.G) ⟶ S.FDiscrete.obj (Discrete.mk c1) ⊗ S.FDiscrete.obj (Discrete.mk
    c2) ⊗
      S.FDiscrete.obj (Discrete.mk c3))
    (g : S.G) : (action g (constThreeNode v)).tensor = (constThreeNode v).tensor
```

## 2.45 HepLean.Tensors.Tree.NodeIdentities.Congr

HepLean/Tensors/Tree/NodeIdentities/Congr.lean on GitHub.

**lemma perm_congr** [source]

```
lemma perm_congr {c1 : Fin n → S.C} {c2 : Fin m → S.C} {T T' : TensorTree S c1}
    {σ σ' : OverColor.mk c1 ⟶ OverColor.mk c2}
    (h : σ = σ') (hT : T.tensor = T'.tensor) :
    (perm σ T).tensor = (perm σ' T').tensor
```

**lemma perm_update** [source]

```
lemma perm_update {c1 : Fin n → S.C} {c2 : Fin m → S.C} {T : TensorTree S c1}
    {σ : OverColor.mk c1 ⟶ OverColor.mk c2} (σ' : OverColor.mk c1 ⟶ OverColor.mk c2)
    (h : σ = σ') :
    (perm σ T).tensor = (perm σ' T).tensor
```

**lemma contr_congr** [source]

```
lemma contr_congr {n : ℕ} {c : Fin n.succ.succ → S.C} {i : Fin n.succ.succ}
    (i' : Fin n.succ.succ) {j : Fin n.succ} (j' : Fin n.succ){h : c (i.succAbove j) = S.τ (c i)
    }
    {t : TensorTree S c} (hi : i = i' := by decide) (hj : j = j' := by decide) :
    (contr i j h t).tensor =
    (perm (mkIso (by rw [hi, hj])).hom (contr i' j' (by rw [← hi, ← hj, h]) t)).tensor
```

## 2.46 HepLean.Tensors.Tree.NodeIdentities.ContrContr

HepLean/Tensors/Tree/NodeIdentities/ContrContr.lean on GitHub.

**structure ContrQuartet** [source]

*A structure containing two pairs of indices (i, j) and (k, l) to be sequentially contracted in a tensor.*

```
structure ContrQuartet {n : ℕ} (c : Fin n.succ.succ.succ.succ → S.C) where
  i : Fin n.succ.succ.succ.succ
  j : Fin n.succ.succ.succ
```

```
  k : Fin n.succ.succ
  l : Fin n.succ
  hij : c (i.succAbove j) = S.τ (c i)
  hkl : (c ∘ i.succAbove ∘ j.succAbove) (k.succAbove l) = S.τ ((c ∘ i.succAbove ∘ j.succAbove)
    k)
```

**def swapI** [source]

*On swapping the order of contraction (notionally `(i, j) - (k, l)` vs `(k, l) - (i, j)`), this is the new `i` index.*

```
def swapI : Fin n.succ.succ.succ.succ
```

**def swapJ** [source]

*On swapping the order of contraction (notionally `(i, j) - (k, l)` vs `(k, l) - (i, j)`), this is the new `j` index.*

```
def swapJ : Fin n.succ.succ.succ
```

**def swapK** [source]

*On swapping the order of contraction (notionally `(i, j) - (k, l)` vs `(k, l) - (i, j)`), this is the new `k` index.*

```
def swapK : Fin n.succ.succ
```

**def swapL** [source]

*On swapping the order of contraction (notionally `(i, j) - (k, l)` vs `(k, l) - (i, j)`), this is the new `l` index.*

```
def swapL : Fin n.succ
```

**lemma swap_map_eq** [source]

```
lemma swap_map_eq (x : Fin n) : (q.swapI.succAbove (q.swapJ.succAbove
    (q.swapK.succAbove (q.swapL.succAbove x)))) =
    (q.i.succAbove (q.j.succAbove (q.k.succAbove (q.l.succAbove x))))
```

**lemma swapI_neq_i** [source]

```
@[simp]
lemma swapI_neq_i : ¬ q.swapI = q.i
```

**lemma swapI_neq_succAbove** [source]

```
@[simp]
lemma swapI_neq_succAbove : ¬ q.swapI = q.i.succAbove q.j
```

**lemma swapI_neq_i_j_k_l_succAbove** [source]

```
@[simp]
lemma swapI_neq_i_j_k_l_succAbove :
    ¬ q.swapI = q.i.succAbove (q.j.succAbove (q.k.succAbove q.l))
```

**lemma swapJ_swapI_succAbove** [source]

```
lemma swapJ_swapI_succAbove : q.swapI.succAbove q.swapJ = q.i.succAbove
    (q.j.succAbove (q.k.succAbove q.l))
```

**lemma swapJ_eq_swapI_predAbove** [source]

```
lemma swapJ_eq_swapI_predAbove : q.swapJ = predAboveI q.swapI (q.i.succAbove
    (q.j.succAbove (q.k.succAbove q.l)))
```

**lemma swapK_swapJ_succAbove** [source]

```
@[simp]
lemma swapK_swapJ_succAbove : (q.swapJ.succAbove q.swapK) = predAboveI q.swapI q.i
```

**lemma swapK_swapJ_swapI_succAbove** [source]

```
@[simp]
lemma swapK_swapJ_swapI_succAbove : (q.swapI).succAbove (predAboveI q.swapI q.i) = q.i
```

**lemma swapL_swapK_swapJ_swapI_succAbove** [source]

```
@[simp]
lemma swapL_swapK_swapJ_swapI_succAbove :
    q.swapI.succAbove (q.swapJ.succAbove (q.swapK.succAbove q.swapL)) = q.i.succAbove q.j
```

**def swap** [source]

*The `ContrQuartet` corresponding to swapping the order of contraction (notionally `(i, j) - (k, l)` vs `(k, l) - (i, j)`).*

```
def swap : ContrQuartet c where
```

**def contrMapFst** [source]

*The contraction map for the first pair of indices.*

```
def contrMapFst
```

**def contrMapSnd** [source]

*The contractoin map for the second pair of indices.*

```
def contrMapSnd
```

**def contrSwapHom** [source]

*The homomorphism one must apply on swapping the order of contractions.*

```
def contrSwapHom : (OverColor.mk ((c ∘ q.swap.i.succAbove ∘ q.swap.j.succAbove) ∘
    q.swap.k.succAbove ∘ q.swap.l.succAbove)) ⟶
    (OverColor.mk fun x => c (q.i.succAbove (q.j.succAbove (q.k.succAbove (q.l.succAbove x)))))
```

**lemma contrSwapHom_contrMapSnd_tprod** [source]

```
lemma contrSwapHom_contrMapSnd_tprod (x : (i : (𝟙 Type).obj (OverColor.mk c).left) →
    CoeSort.coe (S.FDiscrete.obj { as := (OverColor.mk c).hom i })) :
    ((lift.obj S.FDiscrete).map q.contrSwapHom).hom
    (q.swap.contrMapSnd.hom ((PiTensorProduct.tprod S.k) fun k =>
    x (q.swap.i.succAbove (q.swap.j.succAbove k)))) = ((S.castToField
    ((S.contr.app { as := (c ∘ q.swap.i.succAbove ∘ q.swap.j.succAbove) q.swap.k }).hom
    (x (q.swap.i.succAbove (q.swap.j.succAbove q.swap.k)) ⊗ₜ[S.k]
    (S.FDiscrete.map (Discrete.eqToHom q.swap.hkl)).hom
    (x (q.swap.i.succAbove (q.swap.j.succAbove (q.swap.k.succAbove q.swap.l)))))) •
    ((lift.obj S.FDiscrete).map q.contrSwapHom).hom ((PiTensorProduct.tprod S.k) fun k =>
    x (q.swap.i.succAbove (q.swap.j.succAbove (q.swap.k.succAbove (q.swap.l.succAbove k))))))
```

**lemma contrSwapHom_tprod** [source]

```
lemma contrSwapHom_tprod (x : (i : (𝟙 Type).obj (OverColor.mk c).left) →
    CoeSort.coe (S.FDiscrete.obj { as := (OverColor.mk c).hom i })) :
    ((PiTensorProduct.tprod S.k)
    fun k => x (q.i.succAbove (q.j.succAbove (q.k.succAbove (q.l.succAbove k))))) =
    ((lift.obj S.FDiscrete).map q.contrSwapHom).hom
    ((PiTensorProduct.tprod S.k) fun k =>
      x (q.swap.i.succAbove (q.swap.j.succAbove (q.swap.k.succAbove (q.swap.l.succAbove k)))))
```

**lemma contrMapFst_contrMapSnd_swap** [source]

```
lemma contrMapFst_contrMapSnd_swap :
    q.contrMapFst ≫ q.contrMapSnd = q.swap.contrMapFst ≫ q.swap.contrMapSnd ≫
    S.F.map q.contrSwapHom
```

**lemma ContrQuartet.contr_contr** [source]

```
lemma contr_contr (t : TensorTree S c) :
    (contr q.k q.l q.hkl (contr q.i q.j q.hij t)).tensor = (perm q.contrSwapHom
    (contr q.swapK q.swapL q.swap.hkl (contr q.swapI q.swapJ q.swap.hij t))).tensor
```

**def contrContrPerm** [source]

*The homomorphism one must apply on swapping the order of contractions. This is identical to `ContrQuartet.contrSwapHom` except manifestly between the correct types.*

```
def contrContrPerm {n : ℕ} {c : Fin n.succ.succ.succ.succ → S.C} {i : Fin n.succ.succ.succ.succ
    }
    {j : Fin n.succ.succ.succ} {k : Fin n.succ.succ} {l : Fin n.succ}
    (hij : c (i.succAbove j) = S.τ (c i)) (hkl : (c ∘ i.succAbove ∘ j.succAbove) (k.succAbove l
    ) =
    S.τ ((c ∘ i.succAbove ∘ j.succAbove) k)) :
```

```
    OverColor.mk ((c ∘ (ContrQuartet.mk i j k l hij hkl).swapI.succAbove ∘
      (ContrQuartet.mk i j k l hij hkl).swapJ.succAbove) ∘
      (ContrQuartet.mk i j k l hij hkl).swapK.succAbove ∘
      (ContrQuartet.mk i j k l hij hkl).swapL.succAbove) ⟶
    OverColor.mk
      ((c ∘ i.succAbove ∘ j.succAbove) ∘ k.succAbove ∘ l.succAbove)
```

**theorem TensorTree.contr_contr** [source]

*Contraction nodes commute on adjusting indices.*

```
theorem contr_contr {n : ℕ} {c : Fin n.succ.succ.succ.succ → S.C} {i : Fin n.succ.succ.succ.
    succ}
    {j : Fin n.succ.succ.succ} {k : Fin n.succ.succ} {l : Fin n.succ}
    (hij : c (i.succAbove j) = S.τ (c i)) (hkl : (c ∘ i.succAbove ∘ j.succAbove) (k.succAbove l
    ) =
    S.τ ((c ∘ i.succAbove ∘ j.succAbove) k))
    (t : TensorTree S c) :
    (contr k l hkl (contr i j hij t)).tensor =
    (perm (contrContrPerm hij hkl)
    (contr (ContrQuartet.mk i j k l hij hkl).swapK (ContrQuartet.mk i j k l hij hkl).swapL
    (ContrQuartet.mk i j k l hij hkl).swap.hkl (contr (ContrQuartet.mk i j k l hij hkl).swapI
    (ContrQuartet.mk i j k l hij hkl).swapJ
    (ContrQuartet.mk i j k l hij hkl).swap.hij t))).tensor
```

## 2.47 HepLean.Tensors.Tree.NodeIdentities.ContrSwap

HepLean/Tensors/Tree/NodeIdentities/ContrSwap.lean on GitHub.

**def swapI** [source]

*On swapping the indices of a contraction (notionally `(i, j)` vs `(j, i)`), this is the new `i` index.*

```
def swapI : Fin n.succ.succ
```

**def swapJ** [source]

*On swapping the indices of a contraction (notionally `(i, j)` vs `(j, i)`), this is the new `j` index.*

```
def swapJ : Fin n.succ
```

**lemma swap_map_eq** [source]

```
lemma swap_map_eq (x : Fin n) : (q.swapI.succAbove (q.swapJ.succAbove x)) =
    (q.i.succAbove (q.j.succAbove x))
```

**lemma swapJ_swapI_succAbove** [source]

```
@[simp]
lemma swapJ_swapI_succAbove : q.swapI.succAbove q.swapJ = q.i
```

**def swap** [source]

*The `ContrPair` corresponding to swapping the indices, notionally `(i, j)` vs `(j, i)`.*

```
def swap : ContrPair c where
```

**lemma swapI_color** [source]

```
lemma swapI_color : c q.swapI = S.τ (c (q.i))
```

**lemma predAboveI_i_swapI** [source]

```
@[simp]
lemma predAboveI_i_swapI : predAboveI q.i q.swapI = q.j
```

**lemma swap_swap** [source]

```
lemma swap_swap : q.swap.swap = q
```

**def contrSwapHom** [source]

*The homomorphism one must apply on swapping indices in a contraction.*

```
def contrSwapHom : (OverColor.mk (c ∘ q.swap.i.succAbove ∘ q.swap.j.succAbove)) ⟶
    (OverColor.mk (c ∘ q.i.succAbove ∘ q.j.succAbove))
```

**lemma contrMap_swap** [source]

```
lemma contrMap_swap : q.contrMap = q.swap.contrMap ≫ S.F.map q.contrSwapHom
```

**lemma ContrPair.contr_swap** [source]

```
lemma contr_swap (t : TensorTree S c) :
    (contr q.i q.j q.h t).tensor = (perm q.contrSwapHom
    (contr q.swapI q.swapJ q.swap.h t)).tensor
```

**theorem TensorTree.contr_swap** [source]

*Swapping the nodes of a contraction.*

```
theorem contr_swap {n : ℕ} {c : Fin n.succ.succ → S.C} {i : Fin n.succ.succ}
    {j : Fin n.succ} (hij : c (i.succAbove j) = S.τ (c i))
    (t : TensorTree S c) :
    (contr i j hij t).tensor = (perm (ContrPair.mk i j hij).contrSwapHom
    (contr (ContrPair.mk i j hij).swapI (ContrPair.mk i j hij).swapJ
    (ContrPair.mk i j hij).swap.h t)).tensor
```

## 2.48 HepLean.Tensors.Tree.NodeIdentities.PermContr

HepLean/Tensors/Tree/NodeIdentities/PermContr.lean on GitHub.

**lemma contrFin1Fin1_naturality** [source]

```
lemma contrFin1Fin1_naturality {n : ℕ} {c c1 : Fin n.succ.succ → S.C}
    {i : Fin n.succ.succ} {j : Fin n.succ} (h : c1 (i.succAbove j) = S.τ (c1 i))
    (σ : (OverColor.mk c) ⟶ (OverColor.mk c1)) :
    (S.F.map (extractTwoAux' i j σ)).hom ≫ (S.contrFin1Fin1 c1 i j h).hom.hom
    = (S.contrFin1Fin1 c ((Hom.toEquiv σ).symm i)
      ((HepLean.Fin.finExtractOnePerm ((Hom.toEquiv σ).symm i) (Hom.toEquiv σ)).symm j)
      (perm_contr_cond S h σ)).hom.hom
    ≫ ((Discrete.pairτ S.FDiscrete S.τ).map (Discrete.eqToHom (Hom.toEquiv_comp_inv_apply σ i)
     :
      (Discrete.mk (c ((Hom.toEquiv σ).symm i))) ⟶ (Discrete.mk (c1 i)))).hom
```

**lemma contrIso_comm_aux_1** [source]

```
lemma contrIso_comm_aux_1 {n : ℕ} {c c1 : Fin n.succ.succ → S.C}
    {i : Fin n.succ.succ} {j : Fin n.succ}
    (σ : (OverColor.mk c) ⟶ (OverColor.mk c1)) :
    ((S.F.map σ).hom ≫ (S.F.map (equivToIso (HepLean.Fin.finExtractTwo i j)).hom).hom) ≫
        (S.F.map (mkSum (c1 ∘ ⇑(HepLean.Fin.finExtractTwo i j).symm)).hom).hom =
    (S.F.map (equivToIso (HepLean.Fin.finExtractTwo ((Hom.toEquiv σ).symm i)
    ((HepLean.Fin.finExtractOnePerm ((Hom.toEquiv σ).symm i) (Hom.toEquiv σ)).symm j))).hom).
    hom ≫
    (S.F.map (mkSum (c ∘ ⇑(HepLean.Fin.finExtractTwo ((Hom.toEquiv σ).symm i)
    ((HepLean.Fin.finExtractOnePerm
    ((Hom.toEquiv σ).symm i) (Hom.toEquiv σ)).symm j)).symm)).hom).hom
    ≫ (S.F.map (extractTwoAux' i j σ ⊗ extractTwoAux i j σ)).hom
```

**lemma contrIso_comm_aux_2** [source]

```
lemma contrIso_comm_aux_2 {n : ℕ} {c c1 : Fin n.succ.succ → S.C}
    {i : Fin n.succ.succ} {j : Fin n.succ}
    (σ : (OverColor.mk c) ⟶ (OverColor.mk c1)) :
    (S.F.map (extractTwoAux' i j σ ⊗ extractTwoAux i j σ)).hom ≫
    (S.F.μIso (OverColor.mk ((c1 ∘ ⇑(HepLean.Fin.finExtractTwo i j).symm) ∘ Sum.inl))
    (OverColor.mk ((c1 ∘ ⇑(HepLean.Fin.finExtractTwo i j).symm) ∘ Sum.inr))).inv.hom =
    (S.F.μIso _ _).inv.hom ≫
    (S.F.map (extractTwoAux' i j σ) ⊗ S.F.map (extractTwoAux i j σ)).hom
```

**lemma contrIso_comm_aux_3** [source]

```
lemma contrIso_comm_aux_3 {n : ℕ} {c c1 : Fin n.succ.succ → S.C}
    {i : Fin n.succ.succ} {j : Fin n.succ}
    (σ : (OverColor.mk c) ⟶ (OverColor.mk c1)) :
      ((Action.functorCategoryEquivalence (ModuleCat S.k) (MonCat.of S.G)).symm.inverse.map
                  (S.F.map (extractTwoAux i j σ))).app
              PUnit.unit ≫
            (S.F.map (mkIso (contrIso.proof_1 S c1 i j)).hom).hom
    = (S.F.map (mkIso (contrIso.proof_1 S c ((Hom.toEquiv σ).symm i)
    ((HepLean.Fin.finExtractOnePerm ((Hom.toEquiv σ).symm i) (Hom.toEquiv σ)).symm j))).hom).
    hom ≫
    (S.F.map (extractTwo i j σ)).hom
```

**def contrIsoComm** [source]

*A helper function used to proof the relation between perm and contr.*

```
def contrIsoComm {n : ℕ} {c c1 : Fin n.succ.succ → S.C}
    {i : Fin n.succ.succ} {j : Fin n.succ} (σ : (OverColor.mk c) ⟶ (OverColor.mk c1))
```

**lemma contrIso_comm_aux_5** [source]

```
lemma contrIso_comm_aux_5 {n : ℕ} {c c1 : Fin n.succ.succ → S.C}
    {i : Fin n.succ.succ} {j : Fin n.succ} (h : c1 (i.succAbove j) = S.τ (c1 i))
    (σ : (OverColor.mk c) ⟶ (OverColor.mk c1)) :
    (S.F.map (extractTwoAux' i j σ) ⊗ S.F.map (extractTwoAux i j σ)).hom ≫
    ((S.contrFin1Fin1 c1 i j h).hom.hom ⊗ (S.F.map (mkIso (contrIso.proof_1 S c1 i j)).hom).hom
    )
    = ((S.contrFin1Fin1 c ((Hom.toEquiv σ).symm i)
    ((HepLean.Fin.finExtractOnePerm ((Hom.toEquiv σ).symm i) (Hom.toEquiv σ)).symm j)
    (perm_contr_cond S h σ)).hom.hom ⊗
    (S.F.map (mkIso (contrIso.proof_1 S c ((Hom.toEquiv σ).symm i)
    ((HepLean.Fin.finExtractOnePerm ((Hom.toEquiv σ).symm i) (Hom.toEquiv σ)).symm j))).hom).
    hom)
    ≫ (S.contrIsoComm σ).hom
```

**lemma contrIso_comm_map** [source]

```
lemma contrIso_comm_map {n : ℕ} {c c1 : Fin n.succ.succ → S.C}
    {i : Fin n.succ.succ} {j : Fin n.succ}
    {h : c1 (i.succAbove j) = S.τ (c1 i)}
    (σ : (OverColor.mk c) ⟶ (OverColor.mk c1)) :
  (S.F.map σ) ≫ (S.contrIso c1 i j h).hom =
  (S.contrIso c ((OverColor.Hom.toEquiv σ).symm i)
    (((Hom.toEquiv (extractOne i σ))).symm j) (S.perm_contr_cond h σ)).hom ≫
    contrIsoComm S σ
```

**lemma contrMap_naturality** [source]

*Contraction commutes with `S.F.map σ` on removing corresponding indices from `σ`.*

```
lemma contrMap_naturality {n : ℕ} {c c1 : Fin n.succ.succ → S.C}
    {i : Fin n.succ.succ} {j : Fin n.succ} {h : c1 (i.succAbove j) = S.τ (c1 i)}
    (σ : (OverColor.mk c) ⟶ (OverColor.mk c1)) :
    (S.F.map σ) ≫ (S.contrMap c1 i j h) =
    (S.contrMap c ((OverColor.Hom.toEquiv σ).symm i)
    (((Hom.toEquiv (extractOne i σ))).symm j) (S.perm_contr_cond h σ)) ≫
    (S.F.map (extractTwo i j σ))
```

**lemma perm_contr** [source]

*Permuting indices, and then contracting is equivalent to contracting and then permuting, once care is taking about ensuring one is contracting the same idices.*

```
lemma perm_contr {n : ℕ} {c : Fin n.succ.succ → S.C} {c1 : Fin n.succ.succ → S.C}
    {i : Fin n.succ.succ} {j : Fin n.succ}
    {h : c1 (i.succAbove j) = S.τ (c1 i)} (t : TensorTree S c)
    (σ : (OverColor.mk c) ⟶ (OverColor.mk c1)) :
    (contr i j h (perm σ t)).tensor
    = (perm (extractTwo i j σ) (contr ((Hom.toEquiv σ).symm i)
    (((Hom.toEquiv (extractOne i σ))).symm j) (S.perm_contr_cond h σ) t)).tensor
```

**lemma perm_contr_congr_mkIso_cond** [source]

```
lemma perm_contr_congr_mkIso_cond {n : ℕ} {c : Fin n.succ.succ → S.C} {c1 : Fin n.succ.succ → S.
    C}
    {i : Fin n.succ.succ} {j : Fin n.succ}
    {σ : (OverColor.mk c) ⟶ (OverColor.mk c1)}
    {i' : Fin n.succ.succ} {j' : Fin n.succ}
    (hi : i' = ((Hom.toEquiv σ).symm i))
    (hj : j' = (((Hom.toEquiv (extractOne i σ))).symm j)) :
    c ∘ i'.succAbove ∘ j'.succAbove = c ∘ Fin.succAbove ((Hom.toEquiv σ).symm i) ∘
    Fin.succAbove ((Hom.toEquiv (extractOne i σ)).symm j)
```

**lemma perm_contr_congr_contr_cond** [source]

```
lemma perm_contr_congr_contr_cond {n : ℕ} {c : Fin n.succ.succ → S.C} {c1 : Fin n.succ.succ → S.
    C}
    {i : Fin n.succ.succ} {j : Fin n.succ}
    (h : c1 (i.succAbove j) = S.τ (c1 i))
    {σ : (OverColor.mk c) ⟶ (OverColor.mk c1)}
    {i' : Fin n.succ.succ} {j' : Fin n.succ}
    (hi : i' = ((Hom.toEquiv σ).symm i))
    (hj : j' = (((Hom.toEquiv (extractOne i σ))).symm j)) :
    c (i'.succAbove j') = S.τ (c i')
```

**lemma perm_contr_congr** [source]

```
lemma perm_contr_congr {n : ℕ} {c : Fin n.succ.succ → S.C} {c1 : Fin n.succ.succ → S.C}
    {i : Fin n.succ.succ} {j : Fin n.succ}
    {h : c1 (i.succAbove j) = S.τ (c1 i)} {t : TensorTree S c}
    {σ : (OverColor.mk c) ⟶ (OverColor.mk c1)}
    (i' : Fin n.succ.succ) (j' : Fin n.succ)
    (hi : i' = ((Hom.toEquiv σ).symm i) := by decide)
    (hj : j' = (((Hom.toEquiv (extractOne i σ))).symm j) := by decide) :
  (contr i j h (perm σ t)).tensor = (perm ((mkIso (perm_contr_congr_mkIso_cond hi hj)).hom ≫
    extractTwo i j σ) (contr i' j' (perm_contr_congr_contr_cond h hi hj) t)).tensor
```

## 2.49 HepLean.Tensors.Tree.NodeIdentities.PermProd

HepLean/Tensors/Tree/NodeIdentities/PermProd.lean on GitHub.

**def permProdLeft** [source]

*The permutation that arises when moving a `perm` node in the left entry through a `prod` node. This permutation is defined using right-whiskering and composition with `finSumFinEquiv` based-isomorphisms.*

```
def permProdLeft
```

**def permProdRight** [source]

*The permutation that arises when moving a `perm` node in the right entry through a `prod` node. This permutation is defined using left-whiskering and composition with `finSumFinEquiv` based-isomorphisms.*

```
def permProdRight
```

**theorem prod_perm_left** [source]

*When a `prod` acts on a `perm` node in the left entry, the `perm` node can be moved through the `prod` node via right-whiskering.*

```
theorem prod_perm_left (t : TensorTree S c) (t2 : TensorTree S c2) :
    (prod (perm σ t) t2).tensor = (perm (permProdLeft c2 σ) (prod t t2)).tensor
```

**theorem prod_perm_right** [source]

*When a `prod` acts on a `perm` node in the right entry, the `perm` node can be moved through the `prod` node via left-whiskering.*

```
theorem prod_perm_right (t2 : TensorTree S c2) (t : TensorTree S c) :
    (prod t2 (perm σ t)).tensor = (perm (permProdRight c2 σ) (prod t2 t)).tensor
```

## 2.50 HepLean.Tensors.Tree.NodeIdentities.ProdAssoc

HepLean/Tensors/Tree/NodeIdentities/ProdAssoc.lean on GitHub.

**def assocPerm** [source]

*The permutation that arises from assocativity of `prod` node. This permutation is defined using braiding and composition with `finSumFinEquiv` based-isomorphisms.*

```
def assocPerm : OverColor.mk
    (Sum.elim (Sum.elim c c2 ∘ ⇑finSumFinEquiv.symm) c3 ∘ ⇑finSumFinEquiv.symm) ≅
    OverColor.mk (Sum.elim c (Sum.elim c2 c3 ∘ ⇑finSumFinEquiv.symm) ∘ ⇑finSumFinEquiv.symm)
```

**lemma finSumFinEquiv_comp_assocPerm** [source]

```
lemma finSumFinEquiv_comp_assocPerm :
    (equivToIso finSumFinEquiv).hom ≫ (assocPerm c c2 c3).hom =
    (whiskerRightIso (equivToIso finSumFinEquiv).symm (OverColor.mk c3)).hom ≫
    (α_ (OverColor.mk c) (OverColor.mk c2) (OverColor.mk c3)).hom ≫
    (whiskerLeftIso (OverColor.mk c) (equivToIso finSumFinEquiv)).hom
    ≫ (equivToIso finSumFinEquiv).hom
```

**theorem prod_assoc** [source]

*The `prod` obeys associativity.*

```
theorem prod_assoc (t : TensorTree S c) (t2 : TensorTree S c2) (t3 : TensorTree S c3) :
    (prod t (prod t2 t3)).tensor = (perm (assocPerm c c2 c3).hom (prod (prod t t2) t3)).tensor
```

**lemma prod_assoc'** [source]

*The alternative version of associativity for `prod` where the permutation is on the opposite side.*

```
lemma prod_assoc' (t : TensorTree S c) (t2 : TensorTree S c2) (t3 : TensorTree S c3) :
    (prod (prod t t2) t3).tensor = (perm (assocPerm c c2 c3).inv (prod t (prod t2 t3))).tensor
```

## 2.51 HepLean.Tensors.Tree.NodeIdentities.ProdComm

HepLean/Tensors/Tree/NodeIdentities/ProdComm.lean on GitHub.

**def braidPerm** [source]

*The permutation that arises when moving a commuting terms in a `prod` node. This permutation is defined using braiding and composition with `finSumFinEquiv` based-isomorphisms.*

```
def braidPerm : OverColor.mk (Sum.elim c2 c ∘ ⇑finSumFinEquiv.symm) ⟶
    OverColor.mk (Sum.elim c c2 ∘ ⇑finSumFinEquiv.symm)
```

**lemma finSumFinEquiv_comp_braidPerm** [source]

```
lemma finSumFinEquiv_comp_braidPerm :
    (equivToIso finSumFinEquiv).hom ≫ braidPerm c c2 =
    (β_ (OverColor.mk c2) (OverColor.mk c)).hom
    ≫ (equivToIso finSumFinEquiv).hom
```

**theorem prod_comm** [source]

*The arguments of a `prod` node can be commuted using braiding.*

```
theorem prod_comm (t : TensorTree S c) (t2 : TensorTree S c2) :
    (prod t t2).tensor = (perm (braidPerm c c2) (prod t2 t)).tensor
```

## 2.52 HepLean.Tensors.Tree.NodeIdentities.ProdContr

HepLean/Tensors/Tree/NodeIdentities/ProdContr.lean on GitHub.

**def leftContrEquivSuccSucc** [source]

*An equivalence needed to perform contraction. For specified `n` and `n1` this reduces to an identity.*

```
def leftContrEquivSuccSucc : Fin (n.succ.succ + n1) ≃ Fin ((n + n1).succ.succ)
```

**def leftContrEquivSucc** [source]

*An equivalence needed to perform contraction. For specified `n` and `n1` this reduces to an identity.*

```
def leftContrEquivSucc : Fin (n.succ + n1) ≃ Fin ((n + n1).succ)
```

**def leftContrI** [source]

*The new fst index of a contraction obtained on moving a contraction in the LHS of a product through the product.*

```
def leftContrI (n1 : ℕ) : Fin ((n + n1).succ.succ)
```

**def leftContrJ** [source]

*The new snd index of a contraction obtained on moving a contraction in the LHS of a product through the product.*

```
def leftContrJ (n1 : ℕ) : Fin ((n + n1).succ)
```

**lemma leftContrJ_succAbove_leftContrI** [source]

```
@[simp]
lemma leftContrJ_succAbove_leftContrI : (q.leftContrI n1).succAbove (q.leftContrJ n1)
    = leftContrEquivSuccSucc (Fin.castAdd n1 (q.i.succAbove q.j))
```

**lemma succAbove_leftContrJ_leftContrI_castAdd** [source]

```
lemma succAbove_leftContrJ_leftContrI_castAdd (x : Fin n) :
    (q.leftContrI n1).succAbove ((q.leftContrJ n1).succAbove (Fin.castAdd n1 x)) =
    leftContrEquivSuccSucc (Fin.castAdd n1 (q.i.succAbove (q.j.succAbove x)))
```

**lemma succAbove_leftContrJ_leftContrI_natAdd** [source]

```
lemma succAbove_leftContrJ_leftContrI_natAdd (x : Fin n1) :
    (q.leftContrI n1).succAbove ((q.leftContrJ n1).succAbove (Fin.natAdd n x)) =
    leftContrEquivSuccSucc (Fin.natAdd n.succ.succ x)
```

**def leftContr** [source]

*The pair of contraction indices obtained on moving a contraction in the LHS of a product through the product.*

```
def leftContr : ContrPair ((Sum.elim c c1 ∘ (@finSumFinEquiv n.succ.succ n1).symm.toFun) ∘
    leftContrEquivSuccSucc.symm) where
```

**lemma leftContr_map_eq** [source]

```
lemma leftContr_map_eq : ((Sum.elim c (OverColor.mk c1).hom ∘ finSumFinEquiv.symm.toFun) ∘
    ⇑leftContrEquivSuccSucc.symm) ∘ (q.leftContr (c1 := c1)).i.succAbove ∘
    (q.leftContr (c1 := c1)).j.succAbove =
    Sum.elim (OverColor.mk (c ∘ q.i.succAbove ∘ q.j.succAbove)).hom (OverColor.mk c1).hom ∘
    ⇑finSumFinEquiv.symm
```

**lemma sum_inl_succAbove_leftContrI_leftContrJ** [source]

```
lemma sum_inl_succAbove_leftContrI_leftContrJ (k : Fin n) : finSumFinEquiv.symm
    (leftContrEquivSuccSucc.symm
    ((q.leftContr (c1 := c1)).i.succAbove
    ((q.leftContr (c1 := c1)).j.succAbove
    ((finSumFinEquiv (Sum.inl k))))))) =
  Sum.map (q.i.succAbove ∘ q.j.succAbove) id (Sum.inl k)
```

**lemma sum_inr_succAbove_leftContrI_leftContrJ** [source]

```
lemma sum_inr_succAbove_leftContrI_leftContrJ (k : Fin n1) : finSumFinEquiv.symm
    (leftContrEquivSuccSucc.symm ((q.leftContr (c1 := c1)).i.succAbove
    ((q.leftContr (c1 := c1)).j.succAbove ((finSumFinEquiv (Sum.inr k))))))) =
    Sum.map (q.i.succAbove ∘ q.j.succAbove) id (Sum.inr k)
```

**lemma contrMap_prod_tprod** [source]

```
lemma contrMap_prod_tprod (p : (i : (𝟭 Type).obj (OverColor.mk c).left) →
    CoeSort.coe (S.FDiscrete.obj { as := (OverColor.mk c).hom i }))
    (q' : (i : (𝟭 Type).obj (OverColor.mk c1).left) →
      CoeSort.coe (S.FDiscrete.obj { as := (OverColor.mk c1).hom i })) :
    (S.F.map (equivToIso finSumFinEquiv).hom).hom
    ((S.F.μ (OverColor.mk (c ∘ q.i.succAbove ∘ q.j.succAbove)) (OverColor.mk c1)).hom
    ((q.contrMap.hom (PiTensorProduct.tprod S.k p)) ⊗ₜ[S.k] (PiTensorProduct.tprod S.k) q'))
    = (S.F.map (mkIso (by exact leftContr_map_eq q)).hom).hom
    (q.leftContr.contrMap.hom
    ((S.F.map (equivToIso (@leftContrEquivSuccSucc n n1)).hom).hom
    ((S.F.map (equivToIso finSumFinEquiv).hom).hom
    ((S.F.μ (OverColor.mk c) (OverColor.mk c1)).hom
    ((PiTensorProduct.tprod S.k) p ⊗ₜ[S.k] (PiTensorProduct.tprod S.k) q')))))
```

**lemma contrMap_prod** [source]

```
lemma contrMap_prod :
    (q.contrMap ▷ S.F.obj (OverColor.mk c1)) ≫ (S.F.μ _ ((OverColor.mk c1))) ≫
    S.F.map (OverColor.equivToIso finSumFinEquiv).hom =
    (S.F.μ ((OverColor.mk c)) ((OverColor.mk c1))) ≫
    S.F.map (OverColor.equivToIso finSumFinEquiv).hom ≫
    S.F.map (OverColor.equivToIso leftContrEquivSuccSucc).hom ≫ q.leftContr.contrMap
    ≫ S.F.map (OverColor.mkIso (q.leftContr_map_eq)).hom
```

**lemma ContrPair.contr_prod** [source]

```
lemma contr_prod
    (t : TensorTree S c) (t1 : TensorTree S c1) :
    (prod (contr q.i q.j q.h t) t1).tensor = ((perm (OverColor.mkIso q.leftContr_map_eq).hom
    (contr (q.leftContrI n1) (q.leftContrJ n1)
    q.leftContr.h
    (perm (OverColor.equivToIso ContrPair.leftContrEquivSuccSucc).hom (prod t t1)))).tensor)
```

**def rightContrI** [source]

*The new fst index of a contraction obtained on moving a contraction in the RHS of a product through the product.*

```
def rightContrI (n1 : ℕ) : Fin ((n1 + n).succ.succ)
```

**def rightContrJ** [source]

*The new snd index of a contraction obtained on moving a contraction in the RHS of a product through the product.*

```
def rightContrJ (n1 : ℕ) : Fin ((n1 + n).succ)
```

**lemma rightContrJ_succAbove_rightContrI** [source]

```
@[simp]
lemma rightContrJ_succAbove_rightContrI : (q.rightContrI n1).succAbove (q.rightContrJ n1)
    = (Fin.natAdd n1 (q.i.succAbove q.j))
```

**lemma succAbove_rightContrJ_rightContrI_castAdd** [source]

```
lemma succAbove_rightContrJ_rightContrI_castAdd (x : Fin n1) :
    (q.rightContrI n1).succAbove ((q.rightContrJ n1).succAbove (Fin.castAdd n x)) =
    (Fin.castAdd n.succ.succ x)
```

**lemma succAbove_rightContrJ_rightContrI_natAdd** [source]

```
lemma succAbove_rightContrJ_rightContrI_natAdd (x : Fin n) :
    (q.rightContrI n1).succAbove ((q.rightContrJ n1).succAbove (Fin.natAdd n1 x)) =
    (Fin.natAdd n1 ((q.i.succAbove) (q.j.succAbove x)))
```

**def rightContr** [source]

*The new pair of indices obtained on moving a contraction in the RHS of a product through the product.*

```
def rightContr : ContrPair ((Sum.elim c1 c ∘ (@finSumFinEquiv n1 n.succ.succ).symm.toFun))
    where
```

**lemma rightContr_map_eq** [source]

```
lemma rightContr_map_eq : ((Sum.elim c1 (OverColor.mk c).hom ∘ finSumFinEquiv.symm.toFun)) ∘
    (q.rightContr (c1 := c1)).i.succAbove ∘ (q.rightContr (c1 := c1)).j.succAbove =
    Sum.elim (OverColor.mk c1).hom (OverColor.mk (c ∘ q.i.succAbove ∘ q.j.succAbove)).hom ∘
    ⇑finSumFinEquiv.symm
```

**lemma sum_inl_succAbove_rightContrI_rightContrJ** [source]

```
lemma sum_inl_succAbove_rightContrI_rightContrJ (k : Fin n1) : (@finSumFinEquiv n1 n.succ.succ).
    symm
    ((q.rightContr (c1 := c1)).i.succAbove
      ((q.rightContr (c1 := c1)).j.succAbove (((@finSumFinEquiv n1 n) (Sum.inl k))))) =
    Sum.map id (q.i.succAbove ∘ q.j.succAbove)
    (finSumFinEquiv.symm (finSumFinEquiv (Sum.inl k)))
```

**lemma sum_inr_succAbove_rightContrI_rightContrJ** [source]

```
lemma sum_inr_succAbove_rightContrI_rightContrJ (k : Fin n) : (@finSumFinEquiv n1 n.succ.succ).
    symm
    ((q.rightContr (c1 := c1)).i.succAbove
    ((q.rightContr (c1 := c1)).j.succAbove ((finSumFinEquiv (Sum.inr k))))) =
    Sum.map id (q.i.succAbove ∘ q.j.succAbove)
    (finSumFinEquiv.symm (finSumFinEquiv (Sum.inr k)))
```

**lemma prod_contrMap_tprod** [source]

```
lemma prod_contrMap_tprod (p : (i : (𝟙 Type).obj (OverColor.mk c1).left) →
    CoeSort.coe (S.FDiscrete.obj { as := (OverColor.mk c1).hom i }))
    (q' : (i : (𝟙 Type).obj (OverColor.mk c).left) →
      CoeSort.coe (S.FDiscrete.obj { as := (OverColor.mk c).hom i })) :
    (S.F.map (equivToIso finSumFinEquiv).hom).hom
    ((S.F.μ (OverColor.mk c1) (OverColor.mk (c ∘ q.i.succAbove ∘ q.j.succAbove))).hom
    ((PiTensorProduct.tprod S.k) p ⊗ₜ[S.k] (q.contrMap.hom (PiTensorProduct.tprod S.k q')))) =
    (S.F.map (mkIso (by exact (rightContr_map_eq q))).hom).hom
    (q.rightContr.contrMap.hom
    (((S.F.map (equivToIso finSumFinEquiv).hom).hom
    ((S.F.μ (OverColor.mk c1) (OverColor.mk c)).hom
    ((PiTensorProduct.tprod S.k) p ⊗ₜ[S.k] (PiTensorProduct.tprod S.k) q')))))
```

**lemma prod_contrMap** [source]

```
lemma prod_contrMap :
    (S.F.obj (OverColor.mk c1) ◁ q.contrMap) ≫ (S.F.μ ((OverColor.mk c1)) _) ≫
    S.F.map (OverColor.equivToIso finSumFinEquiv).hom =
    (S.F.μ ((OverColor.mk c1)) ((OverColor.mk c))) ≫
    S.F.map (OverColor.equivToIso finSumFinEquiv).hom ≫
    q.rightContr.contrMap ≫ S.F.map (OverColor.mkIso (q.rightContr_map_eq)).hom
```

**lemma ContrPair.prod_contr** [source]

```
lemma prod_contr (t1 : TensorTree S c1) (t : TensorTree S c) :
    (prod t1 (contr q.i q.j q.h t)).tensor = ((perm (OverColor.mkIso q.rightContr_map_eq).hom
    (contr (q.rightContrI n1) (q.rightContrJ n1)
    q.rightContr.h (prod t1 t))).tensor)
```

**theorem TensorTree.contr_prod** [source]

*Contraction in the LHS of a product can be moved out of that product.*

```
theorem contr_prod {n n1 : ℕ} {c : Fin n.succ.succ → S.C} {c1 : Fin n1 → S.C} {i : Fin n.succ.
    succ}
    {j : Fin n.succ} (hij : c (i.succAbove j) = S.τ (c i))
    (t : TensorTree S c) (t1 : TensorTree S c1) :
    (prod (contr i j hij t) t1).tensor =
    ((perm (OverColor.mkIso (ContrPair.mk i j hij).leftContr_map_eq).hom
    (contr ((ContrPair.mk i j hij).leftContrI n1) ((ContrPair.mk i j hij).leftContrJ n1)
    (ContrPair.mk i j hij).leftContr.h
    (perm (OverColor.equivToIso ContrPair.leftContrEquivSuccSucc).hom (prod t t1)))).tensor)
```

**theorem TensorTree.prod_contr** [source]

*Contraction in the RHS of a product can be moved out of that product.*

```
theorem prod_contr {n n1 : ℕ} {c : Fin n.succ.succ → S.C} {c1 : Fin n1 → S.C} {i : Fin n.succ.
    succ}
    {j : Fin n.succ} (hij : c (i.succAbove j) = S.τ (c i))
    (t1 : TensorTree S c1) (t : TensorTree S c) :
    (prod t1 (contr i j hij t)).tensor =
    ((perm (OverColor.mkIso (ContrPair.mk i j hij).rightContr_map_eq).hom
    (contr ((ContrPair.mk i j hij).rightContrI n1) ((ContrPair.mk i j hij).rightContrJ n1)
```

```
    (ContrPair.mk i j hij).rightContr.h (prod t1 t))).tensor)
```

## 2.53 scripts.hepLean_style_lint

scripts/hepLean_style_lint.lean on GitHub.

### def longLineLinter [source]

```
def longLineLinter : HepLeanTextLinter
```

### def endLineLinter [source]

```
def endLineLinter (s : String) : HepLeanTextLinter
```